



Republic of Tunisia
Ministry of Higher Education
and Scientific Research
Private Higher School of Engineering and Technology
TEK-UP



FINAL YEAR PROJECT REPORT

Submitted in partial fulfillment of the requirements for the
National Engineering Degree in Applied Sciences and Technologies
Speciality: Software Engineering and Information Systems

Prepared by

Jabberi Mariem

Design and Development of a Secure Human Resources Information System Based on Microservices with CI/CD and DevSecOps Integration

Industrial Supervisor: **Mr. Blel Dhyaddine**

Software Engineer

Academic Supervisor: **Mrs. Zrelli Siwar**

Software Engineer

I authorize the student to submit this internship report for defense.

Industrial Supervisor, **Mr. Blel Dhyaddine**

Signature and stamp

I authorize the student to submit this internship report for defense.

Academic Supervisor, **Mrs. Zrelli Siwar**

Signature

Dedication

I dedicate this humble work with great love, sincerity, and pride.

To my beloved parents, whose endless love, sacrifices, guidance, and encouragement have always been my greatest source of strength. Your unwavering support has inspired me throughout every stage of my academic journey.

To my brothers and my sister, whose affection, understanding, and constant encouragement have always motivated me to strive for excellence. I hope this achievement fulfills your expectations and makes you proud.

To all the members of my family, my professors, and everyone who supported and encouraged me throughout this journey, I express my deepest gratitude and appreciation.

To my friends, who stood by my side, offered their help, encouragement, and friendship whenever I needed it most.

Finally, I sincerely thank everyone who has contributed, directly or indirectly, to the successful completion of this work.

May God bless you all with health, happiness, and success.

Acknowledgement

First and foremost, I would like to express my deepest gratitude to Almighty Allah for granting me the strength, patience, and perseverance to successfully complete this work.

I would like to express my sincere appreciation to my industrial supervisor, **Mr. DhiaEddin Blel**, for his continuous guidance, valuable advice, constructive feedback, and constant encouragement throughout this project. His expertise and insightful recommendations greatly contributed to improving both the technical quality and the organization of this work.

My sincere thanks also go to my academic supervisor, **Mrs. Siwar Zrelli**, for her support, availability, valuable guidance, and constructive comments throughout the realization of this project.

I would also like to extend my sincere gratitude to the members of the examination committee for accepting to review and evaluate this work. It is a great honor to benefit from their expertise and valuable remarks.

My appreciation also goes to the entire educational staff of **TEK-UP Private Higher School of Engineering and Technology** for the quality of education, knowledge, and support they have provided throughout my engineering studies.

Finally, I would like to express my heartfelt thanks to everyone who contributed, directly or indirectly, to the successful completion of this project. Their support, encouragement, and confidence have been invaluable throughout this journey.

Table of Contents

General Introduction	1
1 Project Framework	3
Introduction	4
1.1 Overview of the Host Organization	4
1.1.1 General Presentation	4
1.1.2 Activities	4
1.1.3 Organizational Structure	5
1.2 Internship Context	5
1.2.1 Study of the Existing	5
1.2.2 Critique of the Existing	5
1.3 Problem Statement	6
1.4 Proposed Solution	6
1.4.1 System Overview	6
1.4.2 AI Integration in the Evaluation Service	7
1.4.3 Security Architecture	7
1.4.4 Infrastructure and Service Discovery	8
1.4.5 DevSecOps Approach	8
1.4.6 High-Level Solution Workflow	8
1.5 Project Objectives	9
1.6 Project Management Methodology	10
1.6.1 Scrum Methodology	10
1.6.2 Scrum Roles	11
1.6.3 Scrum Ceremonies	11
Conclusion	12
2 Analysis and Specification of Requirements	13
Introduction	14
2.1 Requirements Specification	14
2.1.1 Identification of Actors	14
2.1.2 Functional Requirements	14
2.1.3 Non-Functional Requirements	17

2.2	System Modeling	17
2.2.1	Global Use-Case Diagram	17
2.2.2	Employee Management Domain Model	18
2.2.3	Evaluation Management Domain Model	19
2.2.4	Artificial Intelligence Module	19
2.3	Project Planning	20
2.3.1	Project Team and Roles	20
2.3.2	Global Product Backlog	21
2.3.3	Release Planning	27
2.4	System Architecture	28
2.4.1	Logical Architecture	28
2.4.2	Physical Architecture	29
2.5	Development Environment	30
2.5.1	Hardware Environment	30
2.5.2	Software Environment	30
	Conclusion	36
3	Release 1: Foundation and Employee Management	37
	Introduction	38
3.1	Sprint 1: Authentication and System Foundation	38
3.1.1	Sprint 1 Backlog	38
3.1.2	Sprint 1 Analysis and Design	39
3.1.3	Realization and Configuration of Sprint 1	43
3.2	Sprint 2: Employee Management	50
3.2.1	Sprint 2 Backlog	50
3.2.2	Sprint 2 Analysis and Design	51
3.2.3	Realization of Sprint 2	55
	Conclusion	65
4	Release 2: Evaluation Management and Decision Support	66
	Introduction	67
4.1	Sprint 3: Evaluation Management	67
4.1.1	Sprint 3 Backlog	67
4.1.2	Sprint 3 Analysis and Design	68
4.1.3	Realization of Sprint 3	72

4.2	Sprint 4: Development and Decision Support	84
4.2.1	Sprint 4 Backlog	84
4.2.2	Sprint 4 Analysis and Design	86
4.2.3	Realization of Sprint 4	91
	Conclusion	99
5	Release 3: DevSecOps and Deployment	101
	Introduction	102
5.1	Sprint 5: Docker Containerization and Local Orchestration	102
5.1.1	Sprint 5 Backlog	102
5.1.2	Sprint 5 Analysis and Design	103
5.1.3	Realization of Sprint 5	105
5.2	Sprint 6: CI/CD Automation, Security and Observability	109
5.2.1	Sprint 6 Backlog	109
5.2.2	Sprint 6 Analysis and Design	110
5.2.3	Realization of Sprint 6	112
	Conclusion	121
	General Conclusion and Perspectives	122
	Webography	124

List of Figures

1.1	ARABSOFT company logo.	4
1.2	High-level architecture of the proposed SIRH platform.	9
1.3	Scrum process adopted for the development of the SIRH platform	10
2.1	Global use-case diagram of the SIRH platform.	18
2.2	Class diagram of the Employee Service domain.	18
2.3	Class diagram of the Evaluation Service domain.	19
2.4	Logical architecture of the proposed SIRH platform.	29
2.5	Physical deployment architecture.	29
2.6	Spring Boot – Logo	31
2.7	Angular – Logo	31
2.8	Keycloak – Logo	31
2.9	Spring Cloud Gateway – Logo	32
2.10	FastAPI – Logo	32
2.11	Python – Logo	32
2.12	MySQL – Logo	33
2.13	Docker – Logo	33
2.14	Docker Compose – Logo	34
2.15	IntelliJ IDEA – Logo	34
2.16	Visual Studio Code – Logo	34
2.17	Swagger – Logo	35
2.18	GitHub – Logo	35
2.19	SonarQube – Logo	35
2.20	Trivy – Logo	35
3.1	Use-case diagram of Sprint 1	39
3.2	Sequence diagram of the authentication workflow	41
3.3	Sprint 1 authentication and security architecture	42
3.4	Eureka Server dashboard showing registered microservices	44
3.5	Spring Cloud Gateway route and security configuration	45
3.6	Realm Configuration (A)	46
3.7	Realm Configuration (B)	46

3.8	Realm Configuration (C)	46
3.9	Realm Configuration (D)	46
3.10	Keycloak Client (A): General settings	47
3.11	Keycloak Client (B): Capability configuration	47
3.12	Keycloak Client (C): Login settings	47
3.13	Realm Role (A): Creating a new role	47
3.14	Realm Role (B): Configured SIRH roles	47
3.15	Keycloak Users (A)	48
3.16	Keycloak Users (B)	48
3.17	JWT Authentication Validation (A)	49
3.18	JWT Authentication Validation (B)	49
3.19	Use-case diagram of Sprint 2	52
3.20	Sequence diagram of employee creation with Keycloak account provisioning	53
3.21	Sequence diagram of employee document upload	54
3.22	Sprint 2 Employee Management architecture	55
3.23	Employee list interface	56
3.24	Employee creation interface	56
3.25	Employee update interface	57
3.26	Welcome email containing temporary credentials	58
3.27	Department list interface	58
3.28	Department creation interface	59
3.29	Department update interface	59
3.30	Service list interface	60
3.31	Service creation interface	60
3.32	Service update interface	61
3.33	Organizational structure visualization	61
3.34	Contract creation interface	62
3.35	Employee contracts list	62
3.36	Contract update interface	63
3.37	Document upload interface	64
3.38	Employee documents list	64
3.39	Document update interface	65
4.1	Use-case diagram of Sprint 3	68

4.2	Sequence diagram of the evaluation campaign workflow	70
4.3	Sequence diagram of the self-evaluation workflow	71
4.4	Sprint 3 architecture of the Evaluation Service	72
4.5	Skills management interface	73
4.6	Skill creation interface	73
4.7	Skill update interface	74
4.8	Evaluation campaigns interface	74
4.9	Evaluation campaign creation interface	75
4.10	Evaluation campaign update interface	75
4.11	Department and service assignment interface	76
4.12	Campaign scope consultation interface	76
4.13	Campaign evaluations interface	77
4.14	Evaluations of a selected campaign	77
4.15	Competency evaluations interface	78
4.16	All competency evaluations interface	78
4.17	Competency questions and scores interface	78
4.18	Employee evaluations interface	79
4.19	Employee competency evaluations interface	79
4.20	Employee competency questions interface	79
4.21	Employee question assessment interface	80
4.22	Manager team evaluations interface	80
4.23	Manager competency evaluations interface	81
4.24	Manager evaluation competencies interface	81
4.25	Manager competency questions interface	81
4.26	Manager question assessment interface	82
4.27	Manager objectives interface	82
4.28	Objective creation interface	83
4.29	Objective update interface	83
4.30	HR objectives follow-up interface	83
4.31	Employee objectives interface	84
4.32	Objective checklist update interface	84
4.33	Use-case diagram of Sprint 4	86
4.34	Sequence diagram of AI recommendation generation	88
4.35	Sequence diagram of assigning approved trainings	89

4.36	Sprint 4 architecture	90
4.37	Training catalogue interface	91
4.38	Top AI training recommendations for a validated evaluation	92
4.39	Manager assignment of approved AI trainings	92
4.40	Manager development plans interface	93
4.41	Employee development plans interface	93
4.42	Employee development plan progress update	94
4.43	HR Admin development plan monitoring	94
4.44	HR Admin Dashboard (A)	96
4.45	HR Admin Dashboard (B)	96
4.46	Manager Dashboard	97
4.47	Employee Dashboard	98
4.48	Reports Dashboard (A): Human Resources Analytics	99
4.49	Reports Dashboard (B): Evaluation and Executive Analytics	99
5.1	Sprint 5 containerized architecture	103
5.2	Infrastructure organization of the SIRH project	106
5.3	Successful Docker Compose execution of the SIRH platform	109
5.4	CI/CD and DevSecOps pipeline of the SIRH platform	111
5.5	GitHub Actions CI/CD workflow	112
5.6	Test and Quality job execution (A)	113
5.7	Generated pipeline artifacts (B)	113
5.8	SonarQube Cloud project summary	114
5.9	OWASP database update workflow (A)	115
5.10	OWASP database update workflow (B)	115
5.11	GitLeaks secret detection workflow showing that no leaks were detected	116
5.12	Docker images of the SIRH platform published to Docker Hub	117
5.13	Kubernetes deployments (A)	118
5.14	Kubernetes services (B)	118
5.15	Running Kubernetes pods (C)	118
5.16	Prometheus monitoring targets reporting the UP state	119
5.17	Grafana <i>SIRH Overview</i> observability dashboard	120
5.18	Grafana Explore displaying centralized logs collected by Loki	121

List of Tables

2.1	Scrum Team Roles and Responsibilities	20
2.2	Global Product Backlog	21
2.3	Release planning of the SIRH project	27
2.4	Hardware environment used during development	30
3.1	Sprint 1 backlog	38
3.2	Role-based access control model	42
3.3	Sprint 2 backlog	50
4.1	Sprint 3 backlog	67
4.2	Evaluation status lifecycle	71
4.3	Sprint 4 backlog	85
4.4	AI recommendation status lifecycle	87
5.1	Sprint 5 backlog	102
5.2	Containerized components of the SIRH platform	104
5.3	Sprint 6 backlog	109

List of Abbreviations

- **AI** = Artificial Intelligence
- **API** = Application Programming Interface
- **AWS** = Amazon Web Services
- **CI/CD** = Continuous Integration / Continuous Deployment
- **CORS** = Cross-Origin Resource Sharing
- **CPU** = Central Processing Unit
- **CRUD** = Create, Read, Update, and Delete
- **CSV** = Comma-Separated Values
- **DTO** = Data Transfer Object
- **DevOps** = Development and Operations
- **DevSecOps** = Development, Security, and Operations
- **HR** = Human Resources
- **HRIS** = Human Resources Information System
- **HTML** = HyperText Markup Language
- **HTTP** = HyperText Transfer Protocol
- **IAM** = Identity and Access Management
- **IDE** = Integrated Development Environment
- **JPA** = Java Persistence API
- **JSON** = JavaScript Object Notation
- **JVM** = Java Virtual Machine
- **JWT** = JSON Web Token
- **JaCoCo** = Java Code Coverage
- **LCOV** = Linux Test Coverage

- **LTS** = Long-Term Support
- **MAE** = Mean Absolute Error
- **ML** = Machine Learning
- **MVC** = Model–View–Controller
- **NVD** = National Vulnerability Database
- **OAuth 2.0** = Open Authorization 2.0
- **OIDC** = OpenID Connect
- **ORM** = Object-Relational Mapping
- **OWASP** = Open Worldwide Application Security Project
- **PDF** = Portable Document Format
- **PDP** = Personal Development Plan
- **RBAC** = Role-Based Access Control
- **REST** = Representational State Transfer
- **RMSE** = Root Mean Square Error
- **SAST** = Static Application Security Testing
- **SBOM** = Software Bill of Materials
- **SCA** = Software Composition Analysis
- **SHA** = Secure Hash Algorithm
- **SIRH** = Système d’Information des Ressources Humaines
- **SMTP** = Simple Mail Transfer Protocol
- **SPA** = Single-Page Application
- **SQL** = Structured Query Language
- **SaaS** = Software as a Service
- **TF-IDF** = Term Frequency–Inverse Document Frequency

- **UI** = User Interface
- **UML** = Unified Modeling Language
- **URL** = Uniform Resource Locator
- **UUID** = Universally Unique Identifier
- **XGBoost** = Extreme Gradient Boosting
- **XML** = eXtensible Markup Language
- **YAML** = YAML Ain't Markup Language

General Introduction

The rapid evolution of information technologies has profoundly transformed the way organizations manage their business processes. In an increasingly digital environment, companies seek software solutions that are scalable, secure, intelligent, and easy to maintain. Among the different organizational functions, Human Resources Management plays a strategic role by supporting employee administration, performance evaluation, competency development, and decision-making.

Traditional Human Resources Information Systems (HRIS) are often implemented as monolithic applications, making them difficult to evolve, scale, and secure. At the same time, modern software engineering practices such as microservices architecture, cloud-native applications, DevSecOps, and Artificial Intelligence have become essential for developing enterprise software capable of meeting today's operational and security requirements.

This final-year project was carried out at ARABSOFIT and aims to design and implement a secure Human Resources Information System based on a microservices architecture. The proposed solution integrates Spring Boot microservices, an Angular frontend, centralized authentication using Keycloak, and an Artificial Intelligence module that generates personalized training recommendations according to employee competencies and evaluation results.

Beyond software development, this project also adopts a complete DevSecOps approach by automating software quality verification, security analysis, containerization, continuous integration, and deployment using Docker, GitHub Actions, SonarQube Cloud, OWASP Dependency-Check, GitLeaks, Trivy, and Kubernetes. Monitoring and observability are ensured through Prometheus, Grafana, Alloy, and Loki, providing continuous visibility into system health and application logs.

The project was developed following the Agile Scrum methodology through six sprints grouped into three releases, allowing the progressive delivery and validation of the platform while ensuring software quality throughout the development lifecycle.

This report is organized as follows:

- Chapter 1 presents the project framework, the host company, the project context, and the adopted development methodology.
- Chapter 2 describes the requirement analysis, functional and non-functional specifications, UML analysis, software architecture, project planning, and development environment.
- Chapter 3 presents Release 1, dedicated to authentication, security, and employee management.
- Chapter 4 presents Release 2, which focuses on performance evaluation, competency management,

Artificial Intelligence recommendations, development plans, dashboards, and reports.

- Chapter 5 presents Release 3, dedicated to Docker containerization, DevSecOps automation, CI/CD pipelines, security scanning, and deployment validation.

PROJECT FRAMEWORK

Plan

Introduction	4
1 Overview of the Host Organization	4
2 Internship Context	5
3 Problem Statement	6
4 Proposed Solution	6
5 Project Objectives	9
6 Project Management Methodology	10
Conclusion	12

Introduction

This chapter introduces the general context of the project and presents the host organization. It also describes the internship framework, analyzes the limitations of existing Human Resources systems, and motivates the need for a modern solution. Finally, it presents the proposed system and the adopted development methodology.

1.1 Overview of the Host Organization

1.1.1 General Presentation

The internship was carried out at **ARABSOFT**, a leading Tunisian IT services company specializing in the development and integration of enterprise information systems.

Founded in 1984, ARABSOFT plays an important role in supporting digital transformation initiatives for public administrations and private organizations. The company provides high-value technological solutions in several domains, including enterprise resource planning, e-government systems, and custom software development.**W2**



Figure 1.1: ARABSOFT company logo.

1.1.2 Activities

The main activities of ARABSOFT include:

- Design and development of enterprise information systems.
- Implementation of ERP solutions.
- Digital transformation of public services.
- IT consulting and system integration.

- Maintenance and evolution of software platforms.

1.1.3 Organizational Structure

ARABSOFT is structured into multiple departments and technical teams that collaborate to deliver complex IT solutions. These include development teams, project management units, infrastructure teams, and quality assurance departments.

This structured organization enables efficient management of large-scale projects and ensures high-quality service delivery.

1.2 Internship Context

1.2.1 Study of the Existing

Within organizations such as ARABSOFT, Human Resources management systems occupy a central role in handling employee records, contracts, organizational structure, and related administrative processes. The efficiency, reliability, and security of these systems directly impact productivity and decision-making at every level of the organization.

However, like many legacy enterprise systems, existing HR solutions commonly suffer from architectural and functional limitations that make them difficult to scale, secure, and maintain. As organizations grow and evolve, the need for a modern, integrated, and extensible Human Resources Information System becomes increasingly critical.

1.2.2 Critique of the Existing

A thorough analysis of conventional HR systems reveals several recurring limitations:

- **Decentralized and fragmented data:** Employee information, contracts, documents, and evaluation data are often spread across disconnected modules or systems, leading to data redundancy, inconsistency, and costly reconciliation efforts.
- **Weak security mechanisms:** Traditional systems frequently lack modern authentication and fine-grained authorization strategies, exposing sensitive HR data to unauthorized access and compliance risks.
- **Monolithic architecture:** Legacy systems are typically built as monolithic applications, making them difficult to scale horizontally, hard to maintain, and prone to cascading failures when a single component is modified.

- **Limited evaluation support:** Performance evaluation, objectives, competencies, and development plans are often managed manually or through disconnected tools, reducing traceability and efficiency.
- **Absence of automation:** Deployment, testing, and quality assurance processes are largely manual, introducing risk and slowing the delivery of new features or critical fixes.
- **Limited adaptability:** Responding to evolving business requirements, such as adding new modules, integrating third-party services, or supporting new roles, is costly and time-consuming in rigid architectures.
- **Poor user experience:** Outdated interfaces hinder adoption and productivity, particularly for HR administrators, managers, and employees who must navigate complex workflows daily.

These limitations collectively highlight the need for a modern HRIS that guarantees scalability, security, modularity, and an intuitive user experience.

1.3 Problem Statement

Based on the challenges identified above, the central problem addressed in this project can be formulated as follows:

How can we design and implement a secure, scalable, modular, and intelligent Human Resources Information System that efficiently manages employee data, organizational structure, contracts, documents, and evaluation processes while integrating modern authentication mechanisms, AI-assisted features, and DevSecOps practices?

1.4 Proposed Solution

To address this problem, we propose the design and development of a modern **Human Resources Information System (SIRH)** built on a **microservices architecture**. The system provides a complete HR management platform accessible through a responsive web interface, secured by a centralized identity management solution, and organized around clearly separated business services.

1.4.1 System Overview

The proposed Human Resources Information System adopts a microservices architecture organized around two main business services and several infrastructure components.

- **Employee Service:** Responsible for all employee administration functionalities, including employee management, department management, service management, contract management, document management, and organizational structure management.
- **Evaluation Service:** Responsible for employee performance management and talent development, including evaluation campaigns, employee evaluations, objectives management, skills assessment, competency evaluation, development plans, notifications, and Artificial Intelligence services.

The business services are supported by:

- Keycloak Identity Provider.
- Spring Cloud Gateway.
- Eureka Discovery Server.
- Angular Frontend.
- MySQL Databases.

1.4.2 AI Integration in the Evaluation Service

The intelligent part of the platform is integrated into the **Evaluation Service**. This AI module uses historical evaluation data, employee objectives, skills, and development plan progress to support decision-making.

The AI module focuses on training recommendation functionalities.

- **Training Recommendation:** Suggesting personalized training actions based on detected skill gaps, required competencies, employee evaluation results, and development objectives.

This integration transforms the Evaluation Service from a simple performance tracking module into an intelligent decision-support component that helps HR administrators and managers identify competency gaps and recommend appropriate training actions.

1.4.3 Security Architecture

Security is implemented using **Keycloak**, an open-source Identity and Access Management solution. Authentication follows the **OpenID Connect (OIDC)** protocol with secure token-based access. Authorization is enforced through **Role-Based Access Control (RBAC)**. [W18]

Three roles are defined in the system:

- **HR_ADMIN:** Full administrative access to manage employees, departments, services, contracts, documents, evaluation campaigns, skills, development plans, and AI-assisted analysis.
- **MANAGER:** Access to team-related information, objectives, evaluations, skills, development plans, and evaluation insights.
- **EMPLOYEE:** Limited access to personal profile, personal HR data, self-evaluation, objectives, and development plans.

When a new employee is created, the system automatically provisions a Keycloak account with the appropriate role and sends a welcome email containing login credentials, ensuring a seamless onboarding experience.

1.4.4 Infrastructure and Service Discovery

The microservices communicate through an **API Gateway** based on Spring Cloud Gateway. This gateway handles request routing, prefix stripping, CORS configuration, and JWT validation.

Services register themselves with a **Eureka Service Registry**, enabling dynamic service discovery and load-balanced routing between service instances. This infrastructure improves maintainability, flexibility, and scalability in a microservices environment.

1.4.5 DevSecOps Approach

The project integrates DevSecOps principles to ensure secure, automated, and reliable development and delivery:

- Continuous Integration and Continuous Deployment pipelines.
- Containerization using Docker for environment consistency.
- Automated testing and code quality analysis.
- Security scanning and vulnerability detection throughout the build pipeline.

1.4.6 High-Level Solution Workflow

Figure 1.2 illustrates the high-level architecture of the proposed SIRH platform, from user interaction to secured access, business processing, AI-assisted training recommendation, and data persistence.

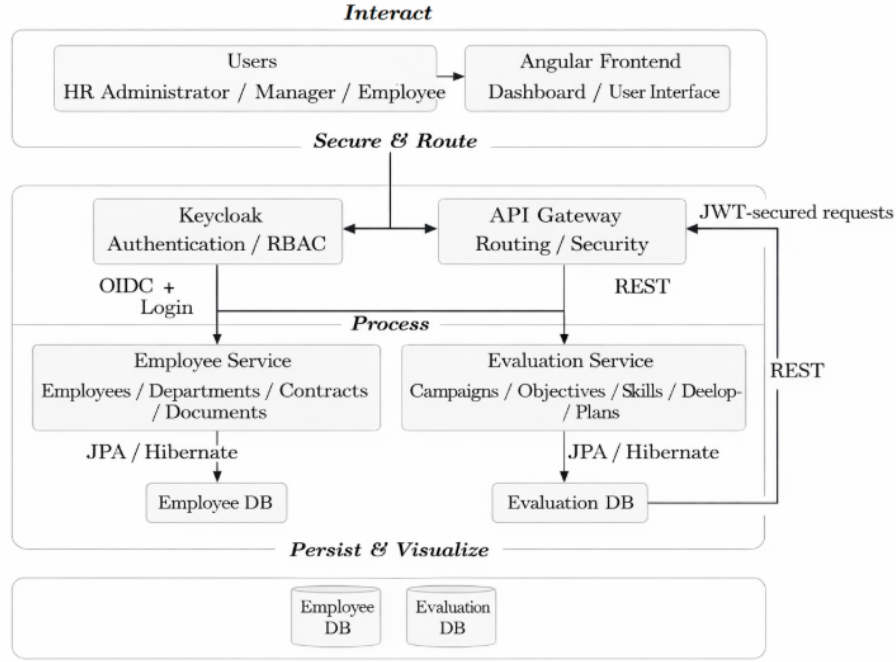


Figure 1.2: High-level architecture of the proposed SIRH platform.

This architecture enforces a clear separation of concerns between presentation, authentication, routing, business logic, AI processing, and data persistence. It enables independent deployment and scaling of each service, improves maintainability, and prepares the system for future DevSecOps automation.

1.5 Project Objectives

The main objective of this project is to design and implement a modern, secure, scalable, and intelligent SIRH.

This objective is divided into the following sub-objectives:

- Develop a responsive frontend using Angular.
- Implement backend microservices using Spring Boot.
- Integrate authentication and authorization with Keycloak.
- Configure API Gateway routing and Eureka service discovery.
- Design a robust and scalable database structure.
- Implement employee and evaluation management features.
- Integrate AI-assisted features inside the Evaluation Service.
- Ensure secure deployment using DevSecOps practices.

1.6 Project Management Methodology

1.6.1 Scrum Methodology

The development of the proposed SIRH platform followed the Agile Scrum methodology in order to organize the project using an iterative and incremental approach. Scrum is an Agile framework based on short development cycles, called *Sprints*, which enable the delivery of functional software increments while allowing continuous feedback and adaptation to changing requirements.

Considering the complexity of the proposed microservices-based architecture, Scrum proved to be particularly suitable for coordinating the implementation of the different functional modules. These modules include employee management, authentication and authorization using Keycloak, performance evaluation management, competency and objective management, AI-assisted training recommendations, development plans, analytical dashboards, and reporting functionalities.

The Product Backlog was established at the beginning of the project and progressively refined throughout the development process. It was then organized into multiple releases, each containing two sprints. Every sprint focused on implementing a coherent set of functionalities and concluded with a functional increment of the application.[W1]

The Scrum workflow adopted during the project is summarized as follows:

Product Backlog → Sprint Planning → Sprint Development → Sprint Review → Sprint Retrospective

This organization made it possible to prioritize high-value functionalities, monitor the progress of the project continuously, and efficiently integrate feedback throughout the development lifecycle.

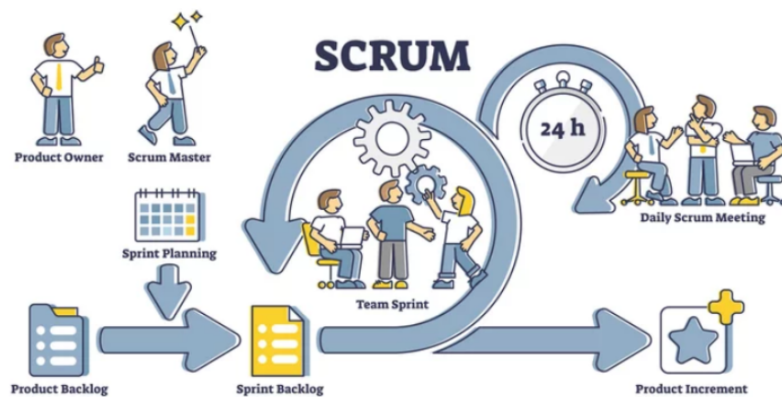


Figure 1.3: Scrum process adopted for the development of the SIRH platform

1.6.2 Scrum Roles

The Scrum roles were adapted to the context of this final-year project as follows:

- **Product Owner:** the academic supervisor and the professional supervisor, responsible for defining the project requirements, prioritizing the Product Backlog, validating the developed functionalities, and ensuring that the delivered solution satisfies the project objectives.
- **Scrum Master:** the student, responsible for organizing the development activities, planning the sprints, monitoring project progress, resolving technical impediments, and ensuring the proper application of Scrum principles throughout the project.
- **Development Team:** the student, responsible for all technical aspects of the project, including requirements analysis, system design, backend development using Spring Boot, frontend development using Angular, AI module implementation using FastAPI, testing, integration, documentation, and deployment preparation.[W1]

1.6.3 Scrum Ceremonies

The Scrum ceremonies were adapted and applied throughout the development process as follows:

- **Sprint Planning:** conducted at the beginning of each sprint to select the user stories from the Product Backlog, define the sprint objectives, estimate the workload, and prepare the implementation plan.
- **Daily Scrum:** performed individually on a daily basis to monitor development progress, identify technical challenges, and reorganize priorities whenever necessary.
- **Sprint Review:** organized at the end of each sprint to demonstrate the completed functionalities, validate the implemented features with the supervisors, and collect feedback for future improvements.
- **Sprint Retrospective:** conducted after each sprint to analyze encountered difficulties, identify lessons learned, and improve the organization and efficiency of subsequent sprints.[W1]

Applying the Scrum methodology significantly improved the organization of the development process by promoting incremental delivery, continuous validation of functionalities, effective project monitoring, and progressive implementation of the proposed SIRH platform.

Conclusion

This chapter presented the host organization ARABSOFTE and the context of the internship. It analyzed the limitations of existing HR systems and introduced the proposed solution based on a secure and scalable microservices architecture composed mainly of the Employee Service and the Evaluation Service. It also introduced the AI component integrated into the Evaluation Service and described the Scrum methodology adopted to organize the project work iteratively and efficiently.

The next chapter will focus on the analysis and specification of system requirements.

ANALYSIS AND SPECIFICATION OF REQUIREMENTS

Plan

Introduction	14
1 Requirements Specification	14
2 System Modeling	17
3 Project Planning	20
4 System Architecture	28
5 Development Environment	30
Conclusion	36

Introduction

This chapter presents the requirements analysis and specification of the proposed Human Resources Information System (SIRH). It begins by identifying the main actors involved in the system and defining both the functional and non-functional requirements.

The chapter then introduces the system modeling through use-case and class diagrams, followed by the project planning artifacts adopted during development. Finally, it presents the target architecture and the technical environment used to implement the solution.

2.1 Requirements Specification

2.1.1 Identification of Actors

The SIRH platform is designed around three principal business actors. Each actor interacts with the system according to a specific scope of responsibility and a defined set of permissions.

- **HR Administrator**

Responsible for managing employees, departments, services, contracts, documents, evaluation campaigns, competencies, development plans, dashboards, and AI-assisted evaluation insights.

- **Manager**

Responsible for supervising teams, evaluating employees, validating objectives, monitoring competencies, and following employee performance.

- **Employee**

Can access personal information, participate in self-evaluations, consult objectives, view assigned development plans, and follow recommended training actions.

The platform also relies on **Keycloak** as an external Identity and Access Management solution responsible for authentication, token generation, and role management.

2.1.2 Functional Requirements

The SIRH platform must satisfy a set of functional requirements covering employee administration, organizational management, performance evaluation, AI assistance, and reporting.

2.1.2.1 Authentication and Access Control

The system must:

- Authenticate users through Keycloak using OpenID Connect.
- Support role-based access control.
- Protect backend resources using JWT authentication.
- Restrict access according to assigned roles.
- Automatically create Keycloak accounts for newly created employees.

2.1.2.2 Employee Management

The Employee Service must allow:

- Employee creation, update, consultation, and deletion.
- Employee search and filtering.
- Profile picture management.
- Department and service assignment.
- Contract and document association.

2.1.2.3 Department and Service Management

The Employee Service must support:

- Department CRUD operations.
- Service CRUD operations.
- Assignment of managers.
- Organizational structure visualization.

2.1.2.4 Contract and Document Management

The Employee Service must provide:

- Contract lifecycle management.
- Contract type management.
- HR document upload and download.
- Employee-document association.

2.1.2.5 Evaluation Management

The Evaluation Service must support:

- Evaluation campaign management.
- Employee self-evaluation.
- Manager evaluation.
- Evaluation validation and archival.
- Evaluation notifications.

2.1.2.6 Objectives, Skills and Development Plans

The Evaluation Service must allow:

- Objective creation and monitoring.
- Skill and competency management.
- Competency evaluation.
- Development plan creation and follow-up.

2.1.2.7 Artificial Intelligence Features

The platform provides intelligent decision-support functionalities integrated into the Evaluation Service.

- Detect competency gaps using evaluation and competency assessment data.
- Recommend personalized training programs.
- Assist HR administrators and managers in talent development decisions.

2.1.2.8 Reporting and Analytics

The platform must provide:

- HR statistics dashboards.
- Evaluation statistics.
- Reporting tools.
- AI-assisted indicators for decision support.

2.1.3 Non-Functional Requirements

In addition to functional requirements, the platform must satisfy the following quality attributes:

- **Security**

Sensitive HR and evaluation data must be protected using authentication, authorization, JWT validation, and role-based access control.

- **Performance**

The platform must provide acceptable response times for common operations such as login, employee search, evaluation consultation, and dashboard display.

- **Scalability**

The architecture must support independent scaling of the Employee Service and the Evaluation Service.

- **Maintainability**

The system must be modular, documented, and easy to evolve.

- **Reliability**

Services must remain available and resilient to failures.

- **Usability**

The frontend must be responsive, intuitive, and accessible.

2.2 System Modeling

System modeling provides a visual representation of the requirements identified previously.

2.2.1 Global Use-Case Diagram

The global use-case diagram shown in Figure 2.1 summarizes the interactions between the different actors and the SIRH platform.

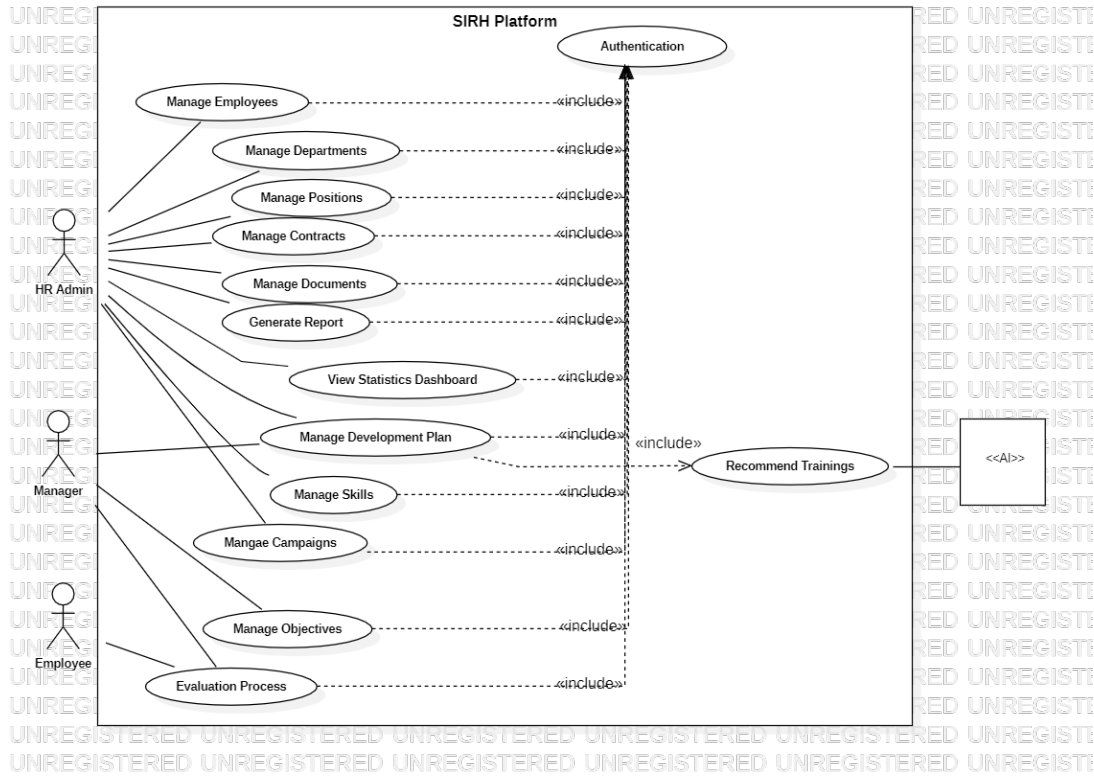


Figure 2.1: Global use-case diagram of the SIRH platform.

The HR Administrator has the widest scope of responsibilities, while Managers and Employees interact with a subset of functionalities according to their business roles.

2.2.2 Employee Management Domain Model

Figure 2.2 illustrates the class diagram of the Employee Management domain, presenting the principal entities, their attributes, associated enumerations, and the relationships implemented within the Employee Service microservice.

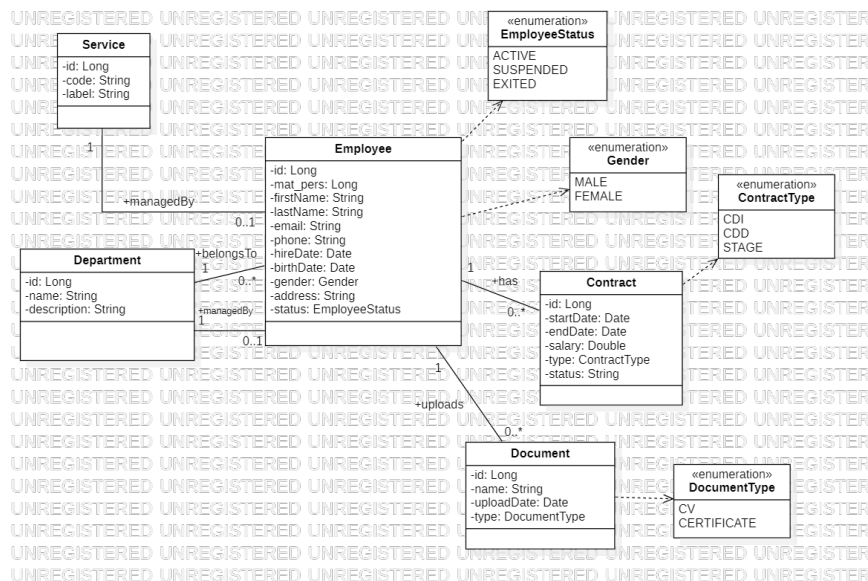


Figure 2.2: Class diagram of the Employee Service domain.

The domain revolves around the Employee entity and its relationships with Departments, Services, Contracts, and Documents.

2.2.3 Evaluation Management Domain Model

Figure 2.3 presents the class diagram of the Evaluation Management domain. It illustrates the core entities, their attributes, associated enumerations, and the relationships implemented within the Evaluation Service microservice.

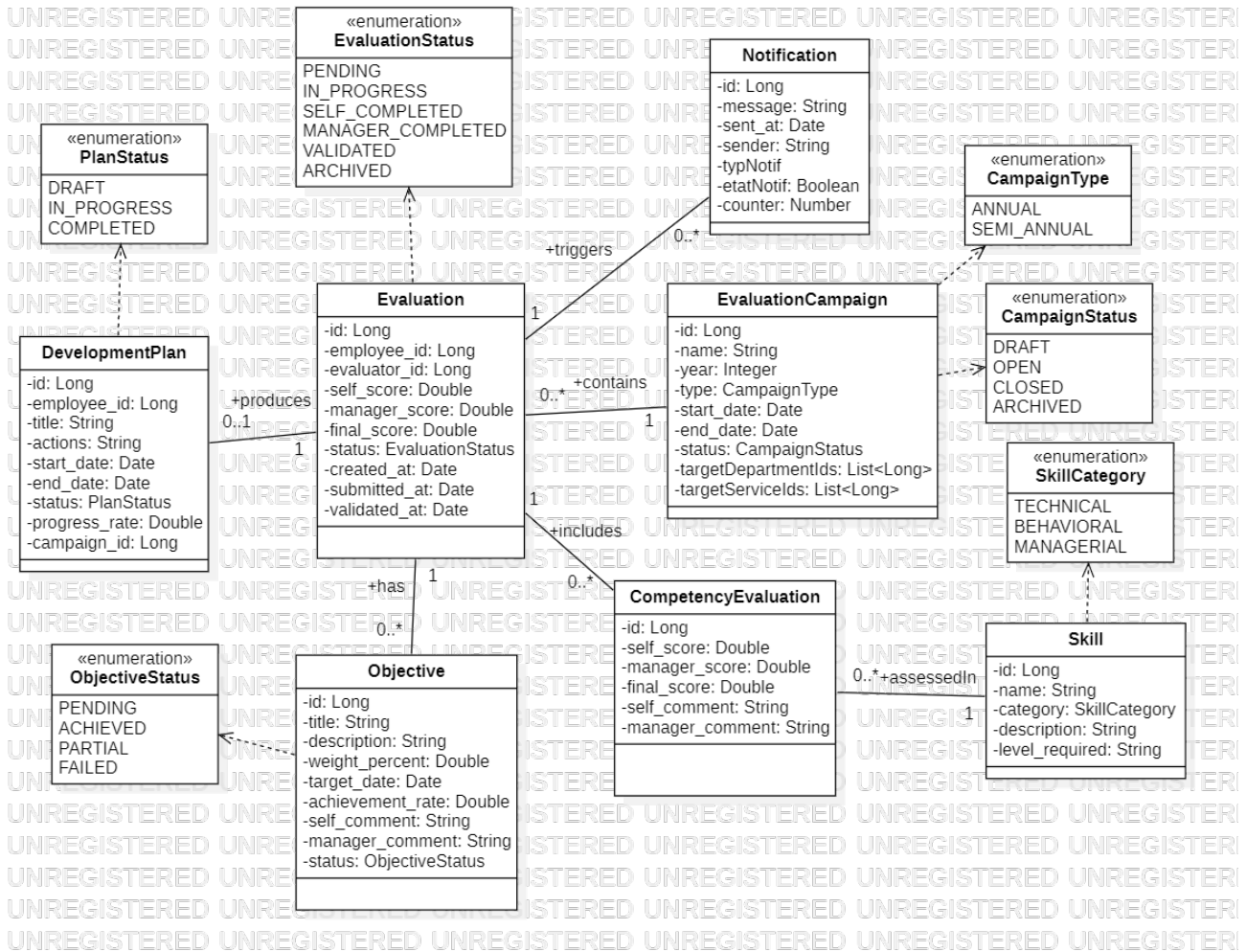


Figure 2.3: Class diagram of the Evaluation Service domain.

The model includes Evaluation Campaigns, Objectives, Competency Evaluations, Skills, Development Plans, and Notifications.

2.2.4 Artificial Intelligence Module

The Evaluation Service integrates an Artificial Intelligence module intended to assist managers and HR administrators during decision-making processes.

The AI module is designed to process:

- Historical evaluations.
- Objectives achievement rates.
- Competency assessments.
- Development plan progress.

The module provides a training recommendation service:

- **Training Recommendation**

Identifies competency gaps and recommends suitable training programs based on evaluation results, competency assessments, and development objectives.

2.3 Project Planning

To ensure incremental delivery and continuous validation of functionalities, the Scrum methodology was adopted.

2.3.1 Project Team and Roles

The project was managed according to the Scrum methodology presented in the previous section. Since this is a final-year project, the Scrum organization was adapted to a supervised single-developer context. Table ?? presents the assignment of the Scrum roles.

Table 2.1: Scrum Team Roles and Responsibilities

Scrum Role	Assignment
Product Owner	The professional supervisor, Mr. DhiaEddin Blel , responsible for expressing the business needs, validating the requirements, prioritizing the Product Backlog, and ensuring that the developed solution meets the project objectives.
Scrum Master	The student, Mariem Jabberi , responsible for organizing the development process, planning the sprints, coordinating the project activities, resolving potential impediments, and ensuring the proper application of the Scrum methodology.

Scrum Role	Assignment
Development Team	The student, Mariem Jabberi , responsible for the complete technical realization of the project, including requirements analysis, system design, backend development using Spring Boot, frontend development using Angular, AI module development using FastAPI, integration, testing, documentation, and DevOps preparation.

2.3.2 Global Product Backlog

The Product Backlog contains the main user stories identified for the SIRH platform. These user stories express the needs of the different actors and describe the expected behavior of the system.

Each user story is described using the following columns:

- **ID:** a unique identifier used to reference and track the user story.
- **User Story:** a functional need written from the point of view of an actor using the format “As a ..., I want to ..., so that ...”.
- **Priority:** the importance level of the user story in the implementation order.
- **Estimate:** the estimated workload required to implement the user story.

Table 2.2 presents the global Product Backlog of the SIRH platform.

Table 2.2: Global Product Backlog

ID	User Story	Priority	Estimate
US01	As a user, I want to authenticate through Keycloak so that I can securely access the platform.	High	2 days
US02	As a user, I want to log out securely so that my session is closed properly.	High	1 day
US03	As a system administrator, I want an API Gateway so that all requests are centralized and secured.	High	2 days
US04	As a system administrator, I want service discovery using Eureka so that microservices can communicate dynamically.	High	2 days

ID	User Story	Priority	Estimate
US05	As a user, I want access to be restricted according to my role so that the platform remains secure.	High	2 days
US06	As an HR Administrator, I want to create employees so that new employees can be registered in the system.	High	2 days
US07	As an HR Administrator, I want to update employee information so that employee records remain accurate.	High	1 day
US08	As an HR Administrator, I want to consult employee information so that I can view employee details.	High	1 day
US09	As an HR Administrator, I want to delete employees so that obsolete employee records can be removed.	High	1 day
US10	As an HR Administrator, I want to upload an employee photo so that the employee profile is complete.	Medium	1 day
US11	As an HR Administrator, I want to assign an employee to a service so that the organizational structure remains accurate.	High	1 day
US12	As an HR Administrator, I want to automatically create a Keycloak account for an employee so that the employee can access the platform.	High	2 days
US13	As an HR Administrator, I want to send a welcome email with a temporary password so that the employee can log in for the first time.	Medium	1 day
US14	As an HR Administrator, I want to create departments so that the company structure can be defined.	High	1 day
US15	As an HR Administrator, I want to update departments so that department information remains accurate.	High	1 day
US16	As an HR Administrator, I want to consult departments so that I can view the organizational structure.	Medium	1 day
US17	As an HR Administrator, I want to delete departments so that obsolete structures can be removed.	Medium	1 day

ID	User Story	Priority	Estimate
US18	As an HR Administrator, I want to assign managers to departments so that responsibilities are clearly defined.	High	1 day
US19	As an HR Administrator, I want to create services so that departments can be divided into operational units.	High	1 day
US20	As an HR Administrator, I want to update services so that service information remains accurate.	High	1 day
US21	As an HR Administrator, I want to consult services so that I can view company services.	Medium	1 day
US22	As an HR Administrator, I want to delete services so that obsolete services can be removed.	Medium	1 day
US23	As an HR Administrator, I want to assign managers to services so that service supervision is defined.	High	1 day
US24	As an HR Administrator, I want to visualize the organization chart so that I can understand the company hierarchy.	Medium	1 day
US25	As an HR Administrator, I want to create employee contracts so that contractual information can be recorded.	High	1 day
US26	As an HR Administrator, I want to update employee contracts so that contract data remains accurate.	High	1 day
US27	As an HR Administrator, I want to consult employee contracts so that I can track employment agreements.	Medium	1 day
US28	As an HR Administrator, I want to delete employee contracts so that invalid or obsolete contracts can be removed.	Medium	1 day
US29	As an HR Administrator, I want to upload employee documents so that HR files are centralized.	High	1 day
US30	As an HR Administrator, I want to consult employee documents so that I can access HR records.	Medium	1 day

ID	User Story	Priority	Estimate
US31	As an HR Administrator, I want to update employee documents so that document information remains accurate.	Medium	1 day
US32	As an HR Administrator, I want to download employee documents so that I can use them outside the platform.	Medium	1 day
US33	As an HR Administrator, I want to delete employee documents so that obsolete files can be removed.	Medium	1 day
US34	As an HR Administrator, I want to create evaluation campaigns so that employee performance can be assessed.	High	2 days
US35	As an HR Administrator, I want to update evaluation campaigns so that campaign information remains accurate.	High	1 day
US36	As an HR Administrator, I want to consult evaluation campaigns so that I can follow evaluation periods.	High	1 day
US37	As an HR Administrator, I want to delete evaluation campaigns so that obsolete campaigns can be removed.	Medium	1 day
US38	As an HR Administrator, I want to define campaign scope by department and service so that the correct employees are evaluated.	High	2 days
US39	As an HR Administrator, I want to attach competency questions to a campaign so that employees can be evaluated by skill.	High	2 days
US40	As an Employee, I want to consult assigned evaluations so that I can know what I need to complete.	High	1 day
US41	As an Employee, I want to perform my self-evaluation so that I can participate in the evaluation process.	High	2 days
US42	As a Manager, I want to consult team evaluations so that I can follow evaluation progress.	High	1 day
US43	As a Manager, I want to evaluate employee competencies so that the final performance score can be calculated.	High	2 days

ID	User Story	Priority	Estimate
US44	As an HR Administrator, I want to validate evaluations so that finalized evaluations become official.	High	1 day
US45	As a user, I want to receive notifications so that I am informed about evaluation actions.	Medium	1 day
US46	As an HR Administrator, I want to create skills so that competencies can be evaluated.	High	1 day
US47	As an HR Administrator, I want to update skills so that competency information remains accurate.	Medium	1 day
US48	As an HR Administrator, I want to consult skills so that I can view available competencies.	Medium	1 day
US49	As an HR Administrator, I want to delete skills so that obsolete competencies can be removed.	Medium	1 day
US50	As a Manager, I want to create objectives for employees so that their work goals can be defined.	Medium	1 day
US51	As a Manager, I want to update objectives so that goal information remains accurate.	Medium	1 day
US52	As a Manager, I want to consult objectives so that I can monitor employee progress.	Medium	1 day
US53	As a Manager, I want to delete objectives so that obsolete goals can be removed.	Medium	1 day
US54	As an Employee, I want to update objective progress so that my manager can follow my achievements.	Medium	1 day
US55	As the system, I want to import open training courses weekly so that the training catalogue remains updated.	Medium	2 days
US56	As an HR Administrator, I want to consult the training catalogue so that I can view available courses.	Medium	1 day
US57	As an HR Administrator, I want to generate AI recommendations so that suitable trainings are proposed based on evaluation results.	High	3 days

ID	User Story	Priority	Estimate
US58	As an HR Administrator, I want to review AI recommendations so that I can verify the proposed trainings.	High	1 day
US59	As an HR Administrator, I want to approve AI recommendations so that they become available for assignment.	High	1 day
US60	As an HR Administrator, I want to reject AI recommendations so that unsuitable trainings are excluded.	High	1 day
US61	As a Manager, I want to assign approved trainings so that an employee receives a development plan.	High	2 days
US62	As an Employee, I want to consult my development plan so that I can follow assigned training actions.	High	1 day
US63	As an Employee, I want to update development plan progress so that my advancement is tracked.	Medium	1 day
US64	As an HR Administrator, I want to consult assigned development plans so that I can monitor employee progression.	Medium	1 day
US65	As a Manager, I want to consult assigned development plans so that I can monitor my team progression.	Medium	1 day
US66	As an HR Administrator, I want to view an HR dashboard so that I can monitor global HR indicators.	High	2 days
US67	As a Manager, I want to view a manager dashboard so that I can monitor my team indicators.	Medium	2 days
US68	As an Employee, I want to view an employee dashboard so that I can track my evaluations, objectives, and development plans.	Medium	1 day
US69	As an HR Administrator, I want to generate reports so that I can analyze HR and evaluation data.	Medium	2 days
US70	As an HR Administrator, I want to export reports so that they can be shared outside the platform.	Medium	1 day

ID	User Story	Priority	Estimate
US71	As a DevOps engineer, I want to containerize the services using Docker so that the platform can be deployed consistently.	Medium	1 day
US72	As a DevOps engineer, I want to configure a CI/CD pipeline so that testing and deployment can be automated.	Medium	2 days
US73	As a DevOps engineer, I want to add static code analysis so that code quality can be monitored.	Medium	2 days
US74	As a DevOps engineer, I want to add security scanning so that vulnerabilities can be detected early.	Medium	2 days
Total			95 days

2.3.3 Release Planning

The project is organized into three releases. Each release contains two sprints and delivers a coherent functional increment. This organization follows the Scrum approach and allows progressive implementation, testing, and validation of the platform.

Table 2.3 presents the Release planning of the SIRH project.

Table 2.3: Release planning of the SIRH project

Release	Sprint Functionalities	Estimate
Release 1: Foundation and Employee Management	Sprint 1: Project setup, Angular base structure, Keycloak configuration, JWT-secured API Gateway, Eureka Service Registry, and role-based access control. Sprint 2: Employee CRUD, photo upload, department management, service management, contract management, document management, organization chart, Keycloak account provisioning, and welcome email.	31 days

Release	Sprint Functionalities	Estimate
Release 2: Evaluation and Competency Management	Sprint 3: Evaluation campaigns, campaign scope, competency questions, self-evaluation workflow, manager evaluation workflow, HR validation, skills management, objectives management, and notifications. Sprint 4: Development plan management, competency gap analysis, AI-based training recommendation, personalized learning paths, HR dashboards, evaluation dashboards, KPI monitoring, reporting generation, statistics visualization, and performance analytics.	31 days
Release 3: DevSecOps and Deployment	Sprint 5: Docker containerization of frontend, backend services, databases, Keycloak, Eureka Server, API Gateway, and AI Engine. Sprint 6: CI/CD pipeline setup, static code analysis, security scanning, monitoring preparation, deployment automation, and final deployment validation.	28 days
Total		90 days

2.4 System Architecture

The proposed solution follows a microservices architecture.

2.4.1 Logical Architecture

Figure 2.4 illustrates the logical organization of the SIRH platform and the main interactions between its frontend, security layer, microservices, AI module, and databases.

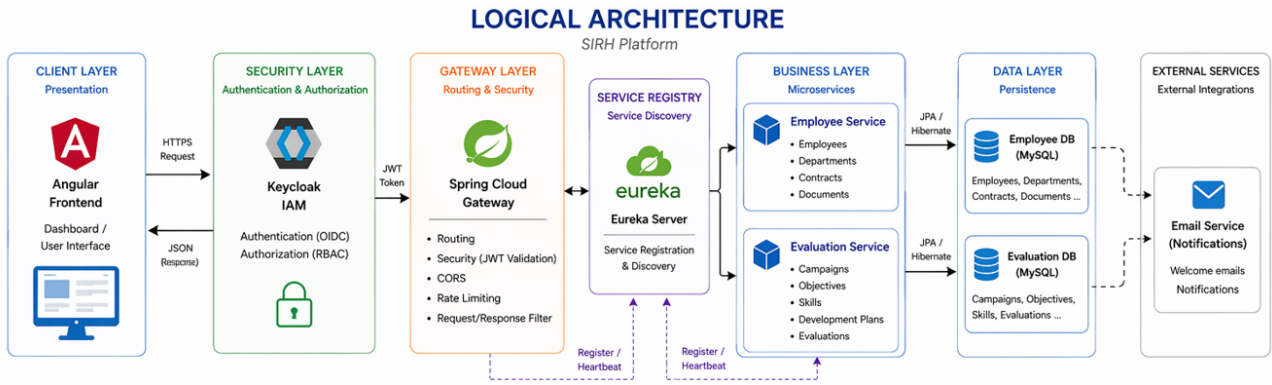


Figure 2.4: Logical architecture of the proposed SIRH platform.

The architecture is composed of:

- Angular Frontend.
- Keycloak Authentication Server.
- Spring Cloud Gateway.
- Eureka Service Registry.
- Employee Service.
- Evaluation Service.
- Artificial Intelligence Module.
- Employee Database.
- Evaluation Database.

2.4.2 Physical Architecture

Figure 2.5 describes the physical deployment view of the platform, where each component is prepared to run independently in a containerized environment.

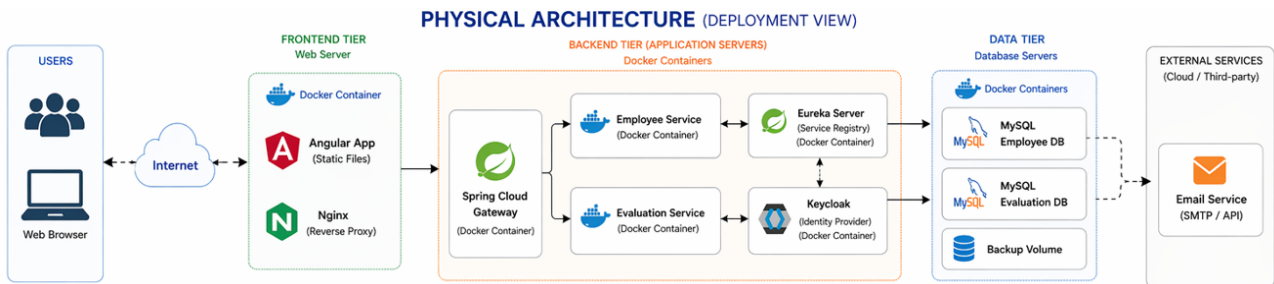


Figure 2.5: Physical deployment architecture.

All components are containerized using Docker and can be deployed independently.

2.5 Development Environment

This section presents the working environment used to design, implement, test, and prepare the deployment of the SIRH platform.

2.5.1 Hardware Environment

The development work was carried out on a personal computer running Windows 11 Professional. This machine was used to execute the Angular frontend, the Spring Boot microservices, the FastAPI AI engine, the databases, Keycloak, and the containerized services during local testing.

The hardware environment used during the implementation is summarized in Table 2.4.

Table 2.4: Hardware environment used during development

Component	Specification
Device name	DESKTOP-813MLHU
Operating system	Windows 11 Professional, 64-bit operating system, x64-based processor
Processor	Intel(R) Core(TM) i7-6500U CPU @ 2.50GHz, 2.60GHz
Memory	12.0 GB RAM, 11.9 GB usable, DDR3
Graphics card	AMD Radeon(TM) R7 M360, 4 GB; Intel(R) HD Graphics 520, 128 MB
Storage	477 GB total storage, 327 GB used
Windows version	Windows 11 Professional, Version 24H2

2.5.2 Software Environment

This subsection presents the main software tools and technologies used during the development of the SIRH platform. Each tool was selected according to its role in the implementation, testing, integration, deployment, or documentation of the system.

- **Spring Boot**

Spring Boot is an open-source Java framework used to create standalone and production-ready applications. In this project, it was used to implement the backend microservices, including the Employee Service, the Evaluation Service, the API Gateway, and the Eureka Discovery Server.

Figure 2.6 shows the Spring Boot logo. [W5]



Figure 2.6: Spring Boot – Logo

- **Angular**

Angular is a TypeScript-based frontend framework used to build dynamic single-page web applications. In this project, it was used to develop the role-based interfaces for HR Administrators, Managers, and Employees, including dashboards, forms, evaluation screens, development plans, reports, and AI recommendation views. [W13]

Figure 2.7 shows the Angular logo.



Figure 2.7: Angular – Logo

- **Keycloak**

Keycloak is an open-source identity and access management solution. It was used in this project to manage authentication, JWT token generation, users, roles, and role-based access control for the HR Admin, Manager, and Employee spaces. [W18]

Figure 2.8 shows the Keycloak logo.



Figure 2.8: Keycloak – Logo

- **Spring Cloud Gateway**

Spring Cloud Gateway was used as the unique entry point of the microservices architecture. It handles request routing, JWT validation, CORS configuration, and access control before forwarding requests to the appropriate backend service. [W8]

Figure 2.9 shows the Spring Cloud logo.



Figure 2.9: Spring Cloud Gateway – Logo

- **Netflix Eureka**

Netflix Eureka was used as a service discovery server. It allows the API Gateway and backend microservices to dynamically register themselves and discover each other without hardcoding service locations. [W9]

- **FastAPI**

FastAPI is a modern Python web framework used to create REST APIs. In this project, it was used to implement the AI engine responsible for importing open courses and generating personalized training recommendations based on evaluation results, skill gaps, objectives, and employee context. [W26]

Figure 2.10 shows the FastAPI logo.



Figure 2.10: FastAPI – Logo

- **Python**

Python was used to develop the AI recommendation module. It supported content-based filtering, TF-IDF vector similarity, collaborative filtering, clustering score calculation, and the import of open training catalogues from public sources. [W25]

Figure 2.11 shows the Python logo.



Figure 2.11: Python – Logo

- **scikit-learn**

scikit-learn is a Python machine learning library. In this project, it was used for TF-IDF vectorization and cosine similarity calculation in order to compare employee development needs with available training catalogue items. [W27]

- **MySQL**

MySQL is a relational database management system. It was used to store the data of the Employee Service, the Evaluation Service, and Keycloak, including employees, departments, services, contracts, documents, evaluation campaigns, objectives, skills, recommendations, and development plans. [W24]

Figure 2.12 shows the MySQL logo.



Figure 2.12: MySQL – Logo

- **Docker**

Docker was used to containerize the different components of the platform, including the Angular frontend, backend microservices, Keycloak, MySQL databases, Eureka Server, API Gateway, and the FastAPI AI engine. [W29]

Figure 2.13 shows the Docker logo.



Figure 2.13: Docker – Logo

- **Docker Compose**

Docker Compose was used to orchestrate the local execution of the complete platform. It allows the different containers to be started together and connected through the same Docker network. [W31]

Figure 2.14 shows the Docker Compose logo.



Figure 2.14: Docker Compose – Logo

- **IntelliJ IDEA**

IntelliJ IDEA was used as the main development environment for the Spring Boot backend services. It provided tools for Java development, Maven dependency management, debugging, and project organization.

Figure 2.15 shows the IntelliJ IDEA logo.



Figure 2.15: IntelliJ IDEA – Logo

- **Visual Studio Code**

Visual Studio Code was used for frontend development, configuration files, Docker files, YAML files, and some Python scripts. It provided a lightweight environment for editing and managing different parts of the project.

Figure 2.16 shows the Visual Studio Code logo.



Figure 2.16: Visual Studio Code – Logo

- **Swagger / OpenAPI**

Swagger UI was used to document and test backend REST APIs. It provided an interactive interface for exploring endpoints, request bodies, responses, and secured API operations. [W22]

Figure 2.17 shows the Swagger logo.



Figure 2.17: Swagger – Logo

- **Git and GitHub**

Git was used for source code version control, while GitHub was used to host the project repository and prepare CI/CD automation. These tools helped manage project versions, branches, commits, and DevSecOps workflows. [W35]

Figure 2.18 shows the GitHub logo.



Figure 2.18: GitHub – Logo

- **SonarQube**

SonarQube was used in the DevSecOps part of the project to analyze code quality. It helps detect bugs, code smells, vulnerabilities, and maintainability issues in the source code. [W42]

Figure 2.19 shows the SonarQube logo.



Figure 2.19: SonarQube – Logo

- **Trivy**

Trivy is a security scanner used to detect vulnerabilities in Docker images and project dependencies. In this project, it was integrated into the DevSecOps workflow to improve container security before deployment. [W48]

Figure 2.20 shows the Trivy logo.



Figure 2.20: Trivy – Logo

Conclusion

This chapter presented the analysis and specification of the proposed SIRH platform. It identified the system actors, formalized the functional and non-functional requirements, and introduced the use-case and class diagrams associated with the Employee Service and Evaluation Service domains.

The chapter also presented the Artificial Intelligence training recommendation module, the product backlog, the target architecture, and the technical environment used throughout the project.

The next chapter presents the realization of Sprint 1, including the implementation of Keycloak authentication, Eureka Server, API Gateway, and JWT-based security.

RELEASE 1: FOUNDATION AND EMPLOYEE MANAGEMENT

Plan

Introduction	38
1 Sprint 1: Authentication and System Foundation	38
2 Sprint 2: Employee Management	50
Conclusion	65

Introduction

This chapter presents the first release of the SIRH platform. This release groups the first two sprints of the project and establishes both the technical foundation and the core employee management functionalities.

Sprint 1 focuses on authentication, authorization, service discovery, API Gateway routing, and the integration of Keycloak with the Angular frontend and backend services. Sprint 2 builds on this foundation by implementing the Employee Service, including employee management, departments, services, contracts, documents, organizational structure visualization, automatic Keycloak account provisioning, and welcome email notification.

3.1 Sprint 1: Authentication and System Foundation

3.1.1 Sprint 1 Backlog

The Sprint 1 backlog presented in Table 3.1 groups the user stories related to the technical foundation, authentication, authorization, and secure communication between the frontend and backend services.

Table 3.1: Sprint 1 backlog

ID	User Story	Priority	Estimate
US-01	As a development team, I want to initialize the backend and frontend projects so that the platform has a stable technical foundation.	High	2 days
US-02	As a system, I want to register backend services dynamically using Eureka Server so that microservices can be discovered without hard-coded addresses.	High	2 days
US-03	As a system, I want to route all requests through an API Gateway so that access to backend services is centralized and controlled.	High	2 days
US-04	As a user, I want to authenticate securely using Keycloak so that I can access the platform according to my identity.	High	3 days

ID	User Story	Priority	Estimate
US-05	As a system, I want to protect backend APIs using JWT tokens so that only authenticated users can access secured resources.	High	2 days
US-06	As an HR Administrator, Manager, or Employee, I want to access only the functionalities allowed by my role so that the system enforces proper authorization.	High	2 days
US-07	As a frontend application, I want to integrate with Keycloak so that login, logout, token retrieval, and route protection are handled securely.	High	2 days
Total			15 days

3.1.2 Sprint 1 Analysis and Design

This section presents the analysis and design decisions adopted during Sprint 1. The objective is to describe how the authentication flow, microservices foundation, service discovery, and secured routing were designed before implementation.

3.1.2.1 Use-Case Model of Sprint 1

The use-case model of this sprint focuses on authentication and protected access to the platform. Figure 3.1 illustrates the interactions between the user and the authentication features implemented during Sprint 1.

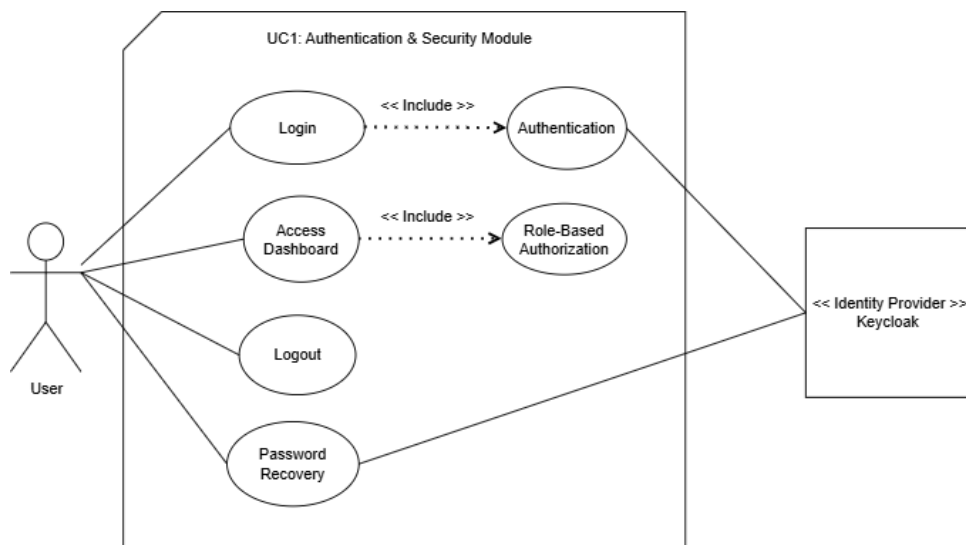


Figure 3.1: Use-case diagram of Sprint 1

The main actor of this sprint is the **User**, which may correspond to an HR Administrator, a Manager, or an Employee. Regardless of the business role, all users must authenticate through Keycloak before accessing the application. The access rights are then determined according to the roles assigned in the authentication server.

The main use cases covered during this sprint are:

- **Login:** Allows the user to authenticate through Keycloak.
- **Access Dashboard:** Allows an authenticated user to access the secured platform interface.
- **Role-Based Authorization:** Verifies the user role before allowing access to protected resources.
- **Logout:** Ends the user session securely.
- **Password Recovery:** Allows the user to recover access through Keycloak.

3.1.2.2 Authentication Workflow

The authentication workflow starts when the user attempts to access the Angular frontend. If the user is not authenticated, the frontend redirects the request to Keycloak. After verifying the user credentials, Keycloak issues a JWT access token. This token is then attached to subsequent HTTP requests sent from Angular to the backend.

The API Gateway receives the request, validates the token, discovers the required service through Eureka Server, and routes the request to the appropriate microservice. Depending on the requested functionality, the request may be forwarded either to the Employee Service or to the Evaluation Service.

Figure 3.2 illustrates the authentication workflow between the Angular frontend, Keycloak, the API Gateway, Eureka Server, and the backend microservices.

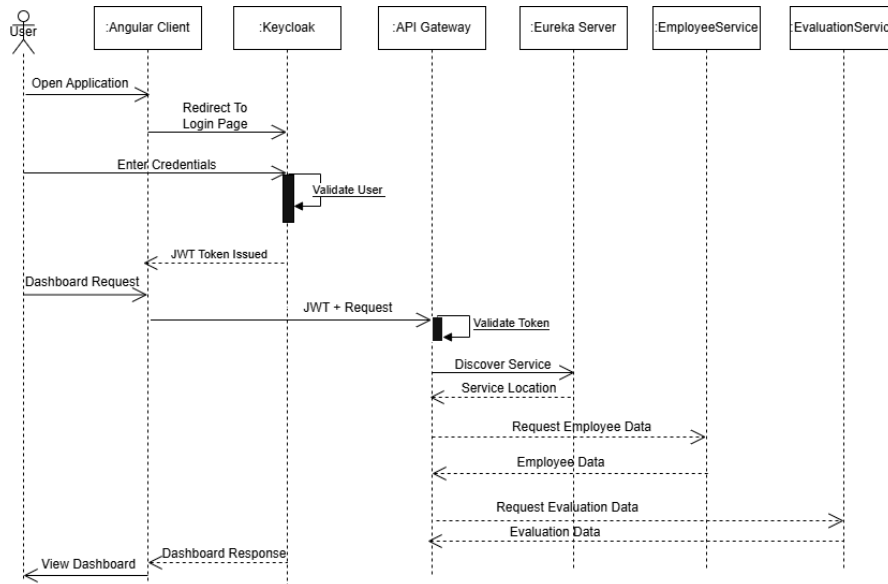


Figure 3.2: Sequence diagram of the authentication workflow

This process ensures that access to the platform is centralized, secure, and consistent across all services.

3.1.2.3 Sprint 1 Architecture

The authentication architecture is based on the separation of responsibilities between the frontend, the identity provider, the gateway, the service registry, and the backend microservices. This architecture ensures secure access to all platform resources while maintaining loose coupling between components. [W1, W18]

- **Angular Frontend:** Provides the user interface and initiates authentication requests through Keycloak.
- **Keycloak:** Acts as the Identity and Access Management server responsible for user authentication, role assignment, and JWT token generation. [W18]
- **API Gateway:** Serves as the single entry point for all client requests. It validates JWT tokens and routes requests to the appropriate microservice. [W8]
- **Eureka Server:** Provides service discovery and allows dynamic registration and lookup of microservices. [W9]
- **Employee Service:** Manages employee-related operations and accepts requests only from authenticated users through the API Gateway.
- **Evaluation Service:** Manages evaluation campaigns, objectives, skills, development plans, and evaluation processes while enforcing the same JWT-based security mechanism.

Figure 3.3 presents the authentication and security architecture implemented during Sprint 1.

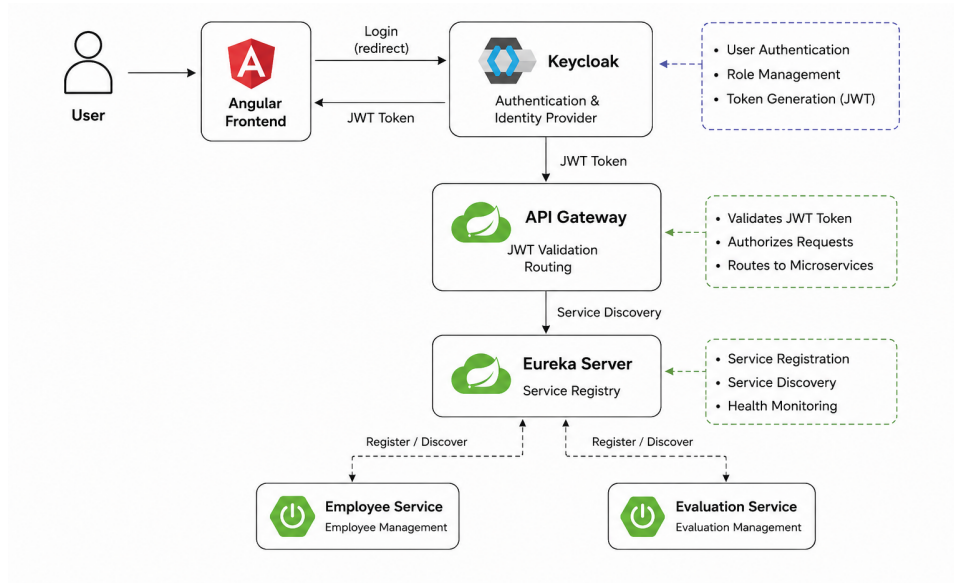


Figure 3.3: Sprint 1 authentication and security architecture

This architecture centralizes authentication and authorization through Keycloak, eliminating the need to duplicate security logic inside each microservice. The API Gateway acts as a security layer by validating JWT tokens before forwarding requests.

3.1.2.4 Role-Based Access Control Design

Role-Based Access Control was adopted to restrict platform functionalities according to the responsibilities of each user. [W20] Table 3.2 presents the access scope associated with each role.

Table 3.2: Role-based access control model

Role	Authorized Scope
HR_ADMIN	Full access to administration features, employee management, organizational structure, contracts, documents, evaluation management, dashboards, reports, AI recommendations, and development plans.
MANAGER	Access to team-related information, employee evaluations, approved AI recommendations, objective tracking, development plan assignment, dashboards, and progress monitoring.

Role	Authorized Scope
EMPLOYEE	Limited access to personal evaluations, competency results, objectives, assigned development plans, progress update, notifications, and personal dashboard.

This authorization model is enforced at two levels. First, Angular uses route guards and role checks to adapt the visible interface. Second, backend APIs validate JWT claims and reject unauthorized requests.

3.1.3 Realization and Configuration of Sprint 1

This section describes the practical implementation and configuration work completed during Sprint 1.

3.1.3.1 Project Initialization

The first step consisted of preparing the frontend and backend project structure. The Angular application was initialized as the presentation layer, while Spring Boot projects were created for the backend services and infrastructure components.

The project structure was organized to separate responsibilities between:

- Frontend application.
- API Gateway.
- Eureka Server.
- Employee Service.
- Evaluation Service.
- Keycloak configuration.

This separation provides a clean foundation for maintenance and future extension.

3.1.3.2 Eureka Server Configuration

Eureka Server was configured as the service registry of the platform. It allows microservices to register themselves automatically at startup and enables the gateway to discover available services dynamically.

Figure 3.4 shows the Eureka dashboard with the registered microservices.

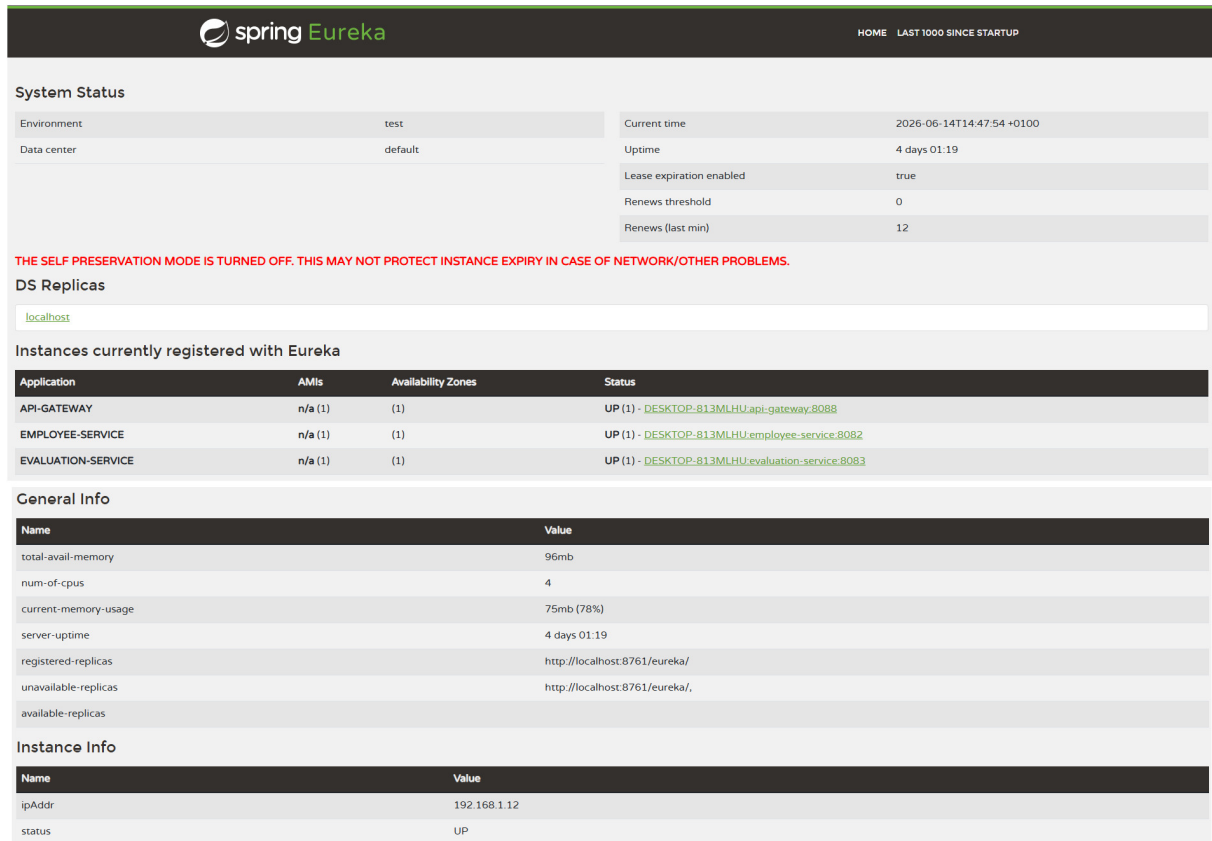


Figure 3.4: Eureka Server dashboard showing registered microservices

This configuration avoids hard-coded service addresses and improves the flexibility of the microservices architecture.

3.1.3.3 API Gateway Configuration

Spring Cloud Gateway was implemented as the unique entry point of the backend ecosystem. It receives requests from the Angular frontend and forwards them to the appropriate microservice according to predefined routes.

The gateway was configured to support:

- Dynamic routing through service names registered in Eureka.
- Prefix-based routing for the Employee Service and Evaluation Service.
- CORS configuration for frontend communication.
- JWT validation through the Keycloak issuer URI.
- Centralized request filtering.

Figure 3.5 illustrates the Spring Cloud Gateway route and security configuration.

```
server:
  port: 8088

spring:
  application:
    name: api-gateway

  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8085/realms/sirh-realm

cloud:
  gateway:
    globalcors:
      add-to-simple-url-handler-mapping: true
      corsConfigurations:
        "[/**]":
          allowedOrigins:
            - "http://localhost:4200"
          allowedMethods:
            - GET
            - POST
            - PUT
            - DELETE
            - OPTIONS
          allowedHeaders:
            - "*"
          allowCredentials: false

    routes:
      - id: employee-service
        uri: lb://EMPLOYEE-SERVICE
        predicates:
          - Path=/employee/**
        filters:
          - StripPrefix=1

      - id: evaluation-service
        uri: lb://EVALUATION-SERVICE
        predicates:
          - Path=/evaluation/**
        filters:
          - StripPrefix=1

logging:
  level:
    org.springframework.security: INFO
    org.springframework.cloud.gateway: INFO

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

Figure 3.5: Spring Cloud Gateway route and security configuration

This gateway simplifies communication between the frontend and microservices by exposing a single access point.

3.1.3.4 Keycloak Installation and Configuration

Keycloak was adopted as the Identity and Access Management (IAM) solution of the SIRH platform to provide centralized authentication and authorization. During this sprint, the authentication infrastructure was configured by creating a dedicated realm, registering the Angular application as a client, defining the application roles, and creating users for authentication and authorization testing.

The main configuration steps included:

- Creation of the SIRH realm.
- Configuration of the Angular client.
- Configuration of redirect URIs and web origins.
- Definition of application roles.

- Creation of test users.
- Assignment of roles to users.

Realm Configuration The first step consisted of configuring the Keycloak realm that hosts all authentication resources of the SIRH platform. The configuration includes the creation of the realm, customization of the login options, application theme configuration, and verification of the final configured environment.

Figures 3.6–3.9 illustrate the different stages of the realm configuration.

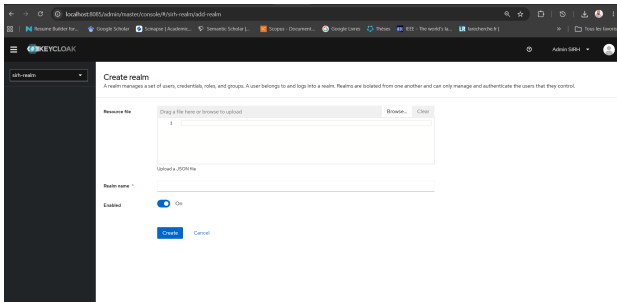


Figure 3.6: Realm Configuration (A)

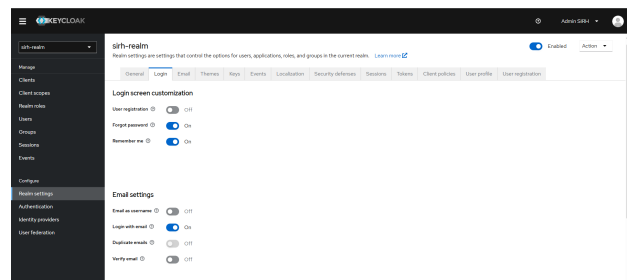


Figure 3.7: Realm Configuration (B)

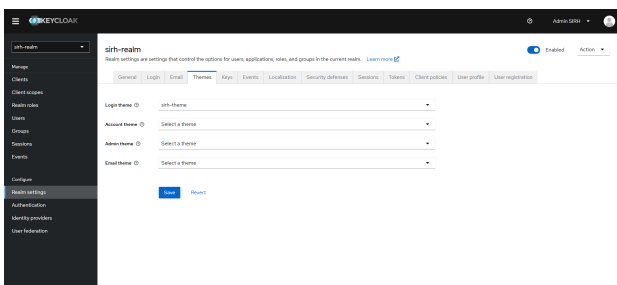


Figure 3.8: Realm Configuration (C)

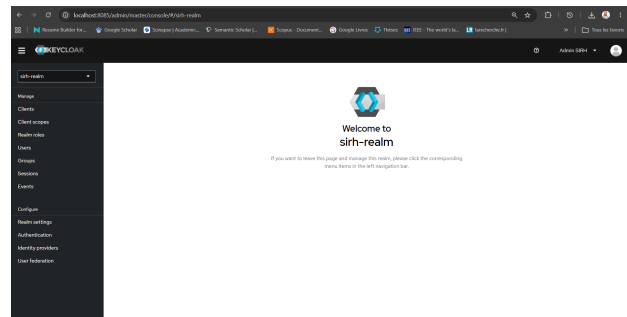


Figure 3.9: Realm Configuration (D)

The created realm provides the authentication environment in which users, roles, clients, and security policies are managed throughout the application.

Keycloak Client Configuration Once the realm was created, the Angular frontend was registered as an OpenID Connect client. The client configuration defines the authentication protocol, the supported authentication flows, the redirect URIs, and the authorized web origins required for secure communication between the frontend application and Keycloak.

Figures 3.10–3.12 present the three successive configuration steps of the Angular client.

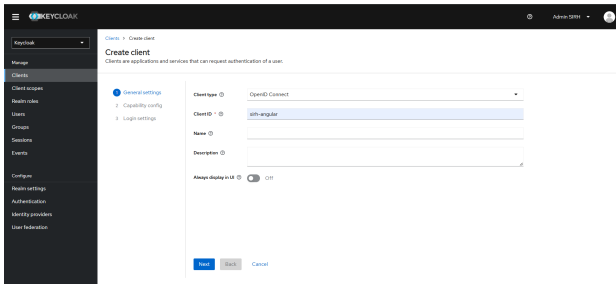


Figure 3.10: Keycloak Client (A): General settings

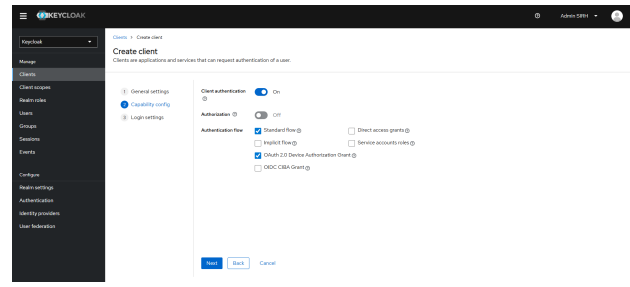


Figure 3.11: Keycloak Client (B): Capability configuration

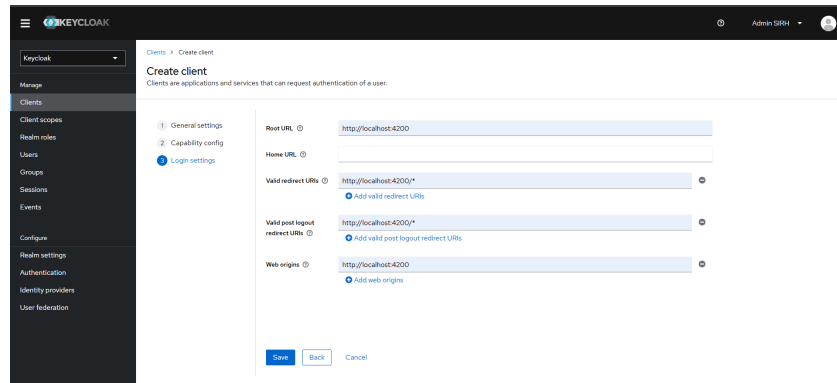


Figure 3.12: Keycloak Client (C): Login settings

The resulting client configuration enables the Angular application to authenticate users securely and obtain JWT access tokens from the Keycloak authorization server.

Role Configuration After configuring the client, the application roles required for role-based access control were defined within the realm.

Figures 3.13 and 3.14 present the creation of a new realm role and the complete list of roles configured for the SIRH platform.

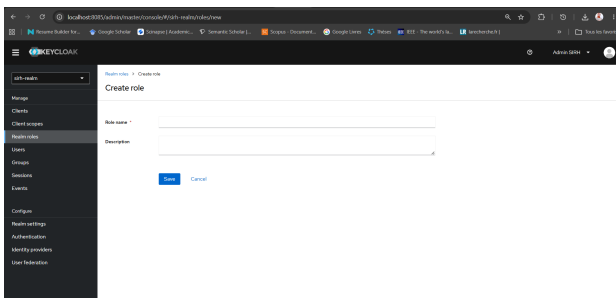


Figure 3.13: Realm Role (A): Creating a new role

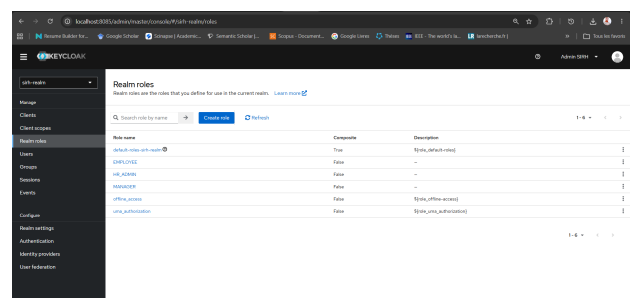


Figure 3.14: Realm Role (B): Configured SIRH roles

Three business roles were created:

- HR_ADMIN
- MANAGER
- EMPLOYEE

These roles are later used by Spring Security to enforce role-based authorization throughout the platform.

User Configuration To validate the authentication and authorization mechanisms, several users were created and assigned to the previously defined roles.

Figures 3.15 and 3.16 illustrate the creation of a new user and the complete list of users available in the SIRH realm.

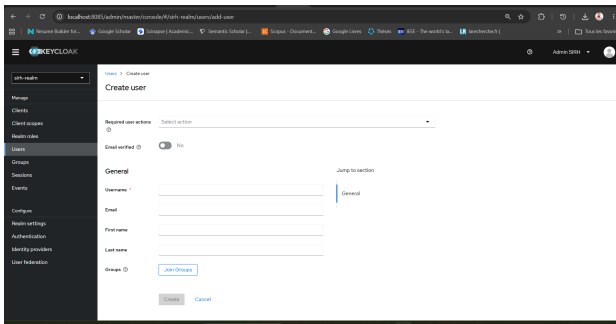


Figure 3.15: Keycloak Users (A)

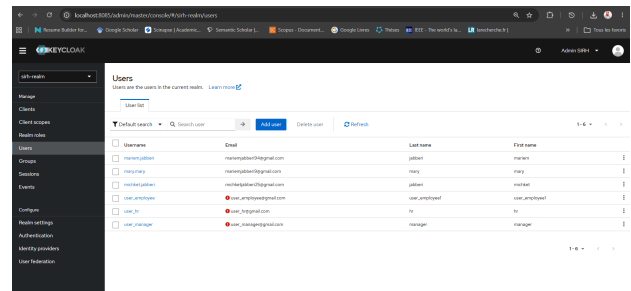


Figure 3.16: Keycloak Users (B)

Each user was assigned the appropriate role to verify authentication, authorization, and access control during the integration tests performed in this sprint.

3.1.3.5 Spring Security and JWT Integration

After configuring Keycloak, Spring Security was integrated into the API Gateway and backend microservices to secure all REST endpoints. Each service was configured as an OAuth2 Resource Server capable of validating JWT access tokens issued by Keycloak.

The security configuration provides the following functionalities:

- JWT token decoding.
- Verification of the token issuer.
- Extraction of user roles from JWT claims.
- Protection of REST endpoints.

- Rejection of unauthorized requests.

The API Gateway validates every incoming access token before forwarding the request to the appropriate microservice, ensuring that only authenticated and authorized users can access protected resources.

3.1.3.6 Authentication Workflow Validation

Following the complete Keycloak and Spring Security configuration, the authentication workflow was validated through end-to-end integration tests using Postman.

The validation scenario consists of the following steps:

1. The user submits authentication credentials to the Keycloak token endpoint.
2. Keycloak authenticates the user.
3. A JWT access token is generated.
4. The access token is attached to subsequent REST requests.
5. The API Gateway validates the JWT token.
6. The request is forwarded to the appropriate microservice.

To verify the authentication mechanism, a token request was executed against the Keycloak OpenID Connect endpoint using the Resource Owner Password Credentials flow. A successful authentication returns both an access token and a refresh token, confirming the correct integration between Keycloak, Spring Security, and the secured backend services.

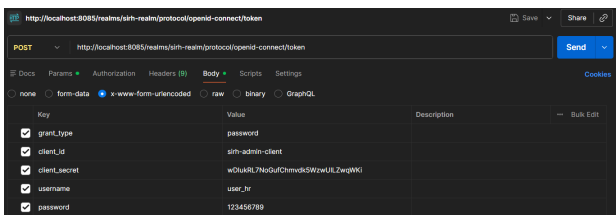


Figure 3.17: JWT Authentication Validation
(A)

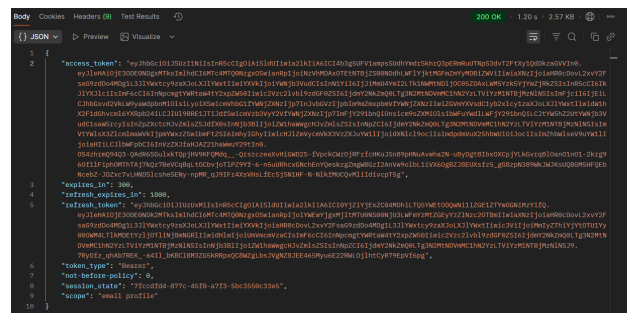


Figure 3.18: JWT Authentication Validation
(B)

Figure 3.17 illustrates the authentication request sent to the Keycloak token endpoint, including the client identifier, client secret, user credentials, and the **password** grant type.

Figure 3.18 presents the successful response returned by Keycloak, containing the generated JWT access token, refresh token, expiration information, and session metadata. These results confirm

that the authentication infrastructure is correctly configured and that authenticated users can securely access the protected REST services through the API Gateway.

3.2 Sprint 2: Employee Management

3.2.1 Sprint 2 Backlog

The Sprint 2 backlog presented in Table 3.3 groups the user stories related to employee management, departments, services, contracts, documents, organizational structure visualization, Keycloak account provisioning, and welcome email notification.

Table 3.3: Sprint 2 backlog

ID	User Story	Priority	Estimate
US-01	As an HR Administrator, I want to create employees so that new employees can be registered in the platform.	High	2 days
US-02	As an HR Administrator, I want to update employees so that employee information remains accurate.	High	1 day
US-03	As an HR Administrator, I want to consult employees so that I can view employee information when needed.	High	1 day
US-04	As an HR Administrator, I want to delete employees so that obsolete employee records can be removed.	High	1 day
US-05	As an HR Administrator, I want to manage departments so that the organizational structure can be maintained.	High	2 days
US-06	As an HR Administrator, I want to manage services so that departments can be organized into functional units.	High	2 days
US-07	As an HR Administrator, I want to manage employee contracts so that contractual information can be stored and tracked.	High	2 days
US-08	As an HR Administrator, I want to manage employee documents so that administrative files can be uploaded, consulted, updated, downloaded, and deleted.	High	1 day
US-09	As a system, I want to automatically create a Keycloak account when a new employee is added so that the employee can access the platform.	High	2 days

ID	User Story	Priority	Estimate
US-10	As a system, I want to send a welcome email containing temporary login credentials so that the new employee can authenticate.	Medium	1 day
US-11	As an HR Administrator, I want to visualize the organizational structure so that departments, services, managers, and employees can be clearly represented.	Medium	1 day
Total			16 days

3.2.2 Sprint 2 Analysis and Design

This section presents the analysis and design decisions adopted during Sprint 2. The objective of this sprint is to implement the Employee Service as an independent microservice responsible for managing employee-related data and organizational information.

The Employee Service communicates with the Angular frontend through the API Gateway and stores its business data in a dedicated database. It also communicates with Keycloak for automatic account provisioning and with the email service for welcome notifications.

3.2.2.1 Use-Case Model of Sprint 2

The use-case model of Sprint 2 focuses mainly on the HR Administrator, who is responsible for managing employees, departments, services, contracts, documents, and the organizational structure.

Figure 3.19 illustrates the use-case diagram of Sprint 2 and summarizes the main interactions between the HR Administrator and the Employee Management module.

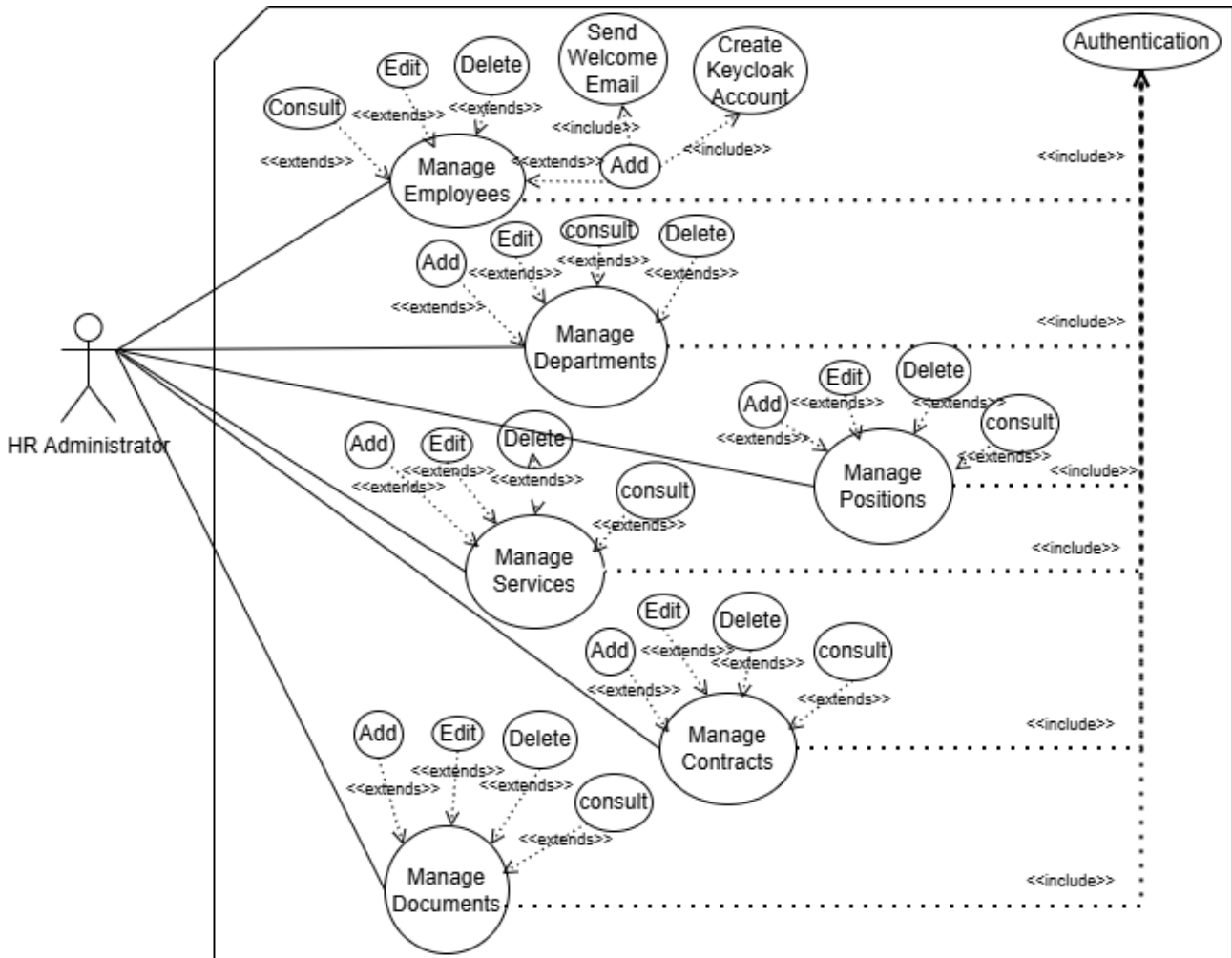


Figure 3.19: Use-case diagram of Sprint 2

The main use cases covered during this sprint are:

- **Manage Employees:** create, update, consult, delete, search, and filter employees.
- **Manage Departments:** create, update, delete, consult, and assign managers to departments.
- **Manage Services:** create, update, delete, consult, assign managers, and link services to departments.
- **Manage Contracts:** create, update, consult, and delete employee contracts.
- **Manage Documents:** upload, consult, update, download, and delete employee documents.
- **Visualize Organization Structure:** display departments, services, managers, and employees.
- **Create Keycloak Account:** automatically provision a user account when an employee is created.
- **Send Welcome Email:** send temporary login credentials to the newly created employee.

3.2.2.2 Employee Creation Workflow

The employee creation workflow starts when the HR Administrator fills in the employee creation form from the Angular frontend. The request is sent to the API Gateway with a JWT token. After token validation, the request is forwarded to the Employee Service.

The Employee Service validates the submitted information, stores the employee data in the database, creates the corresponding user account in Keycloak, generates a temporary password, and sends a welcome email to the employee.

Figure 3.20 presents the sequence diagram of employee creation with automatic Keycloak account provisioning.

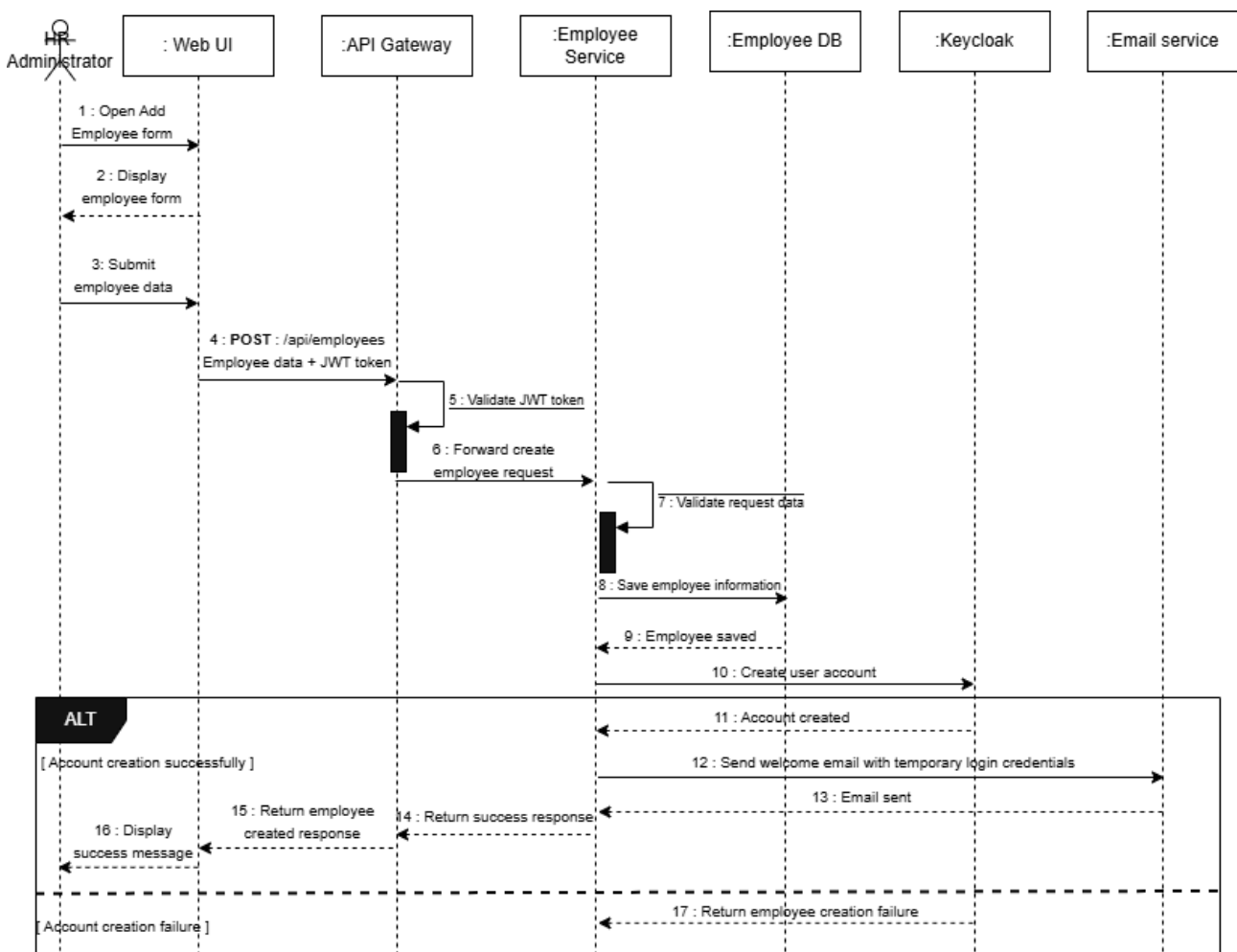


Figure 3.20: Sequence diagram of employee creation with Keycloak account provisioning

This workflow ensures that the administrative creation of an employee is directly linked to platform access provisioning. In the successful scenario, the employee is created, a Keycloak account is provisioned, and a welcome email containing temporary credentials is sent.

3.2.2.3 Document Upload Workflow

Document management is another important functionality of Sprint 2. The HR Administrator can upload administrative documents and associate them with a specific employee. The uploaded file is stored in the document storage space, while its metadata is persisted in the Employee database.

Figure 3.21 presents the sequence diagram of employee document upload.

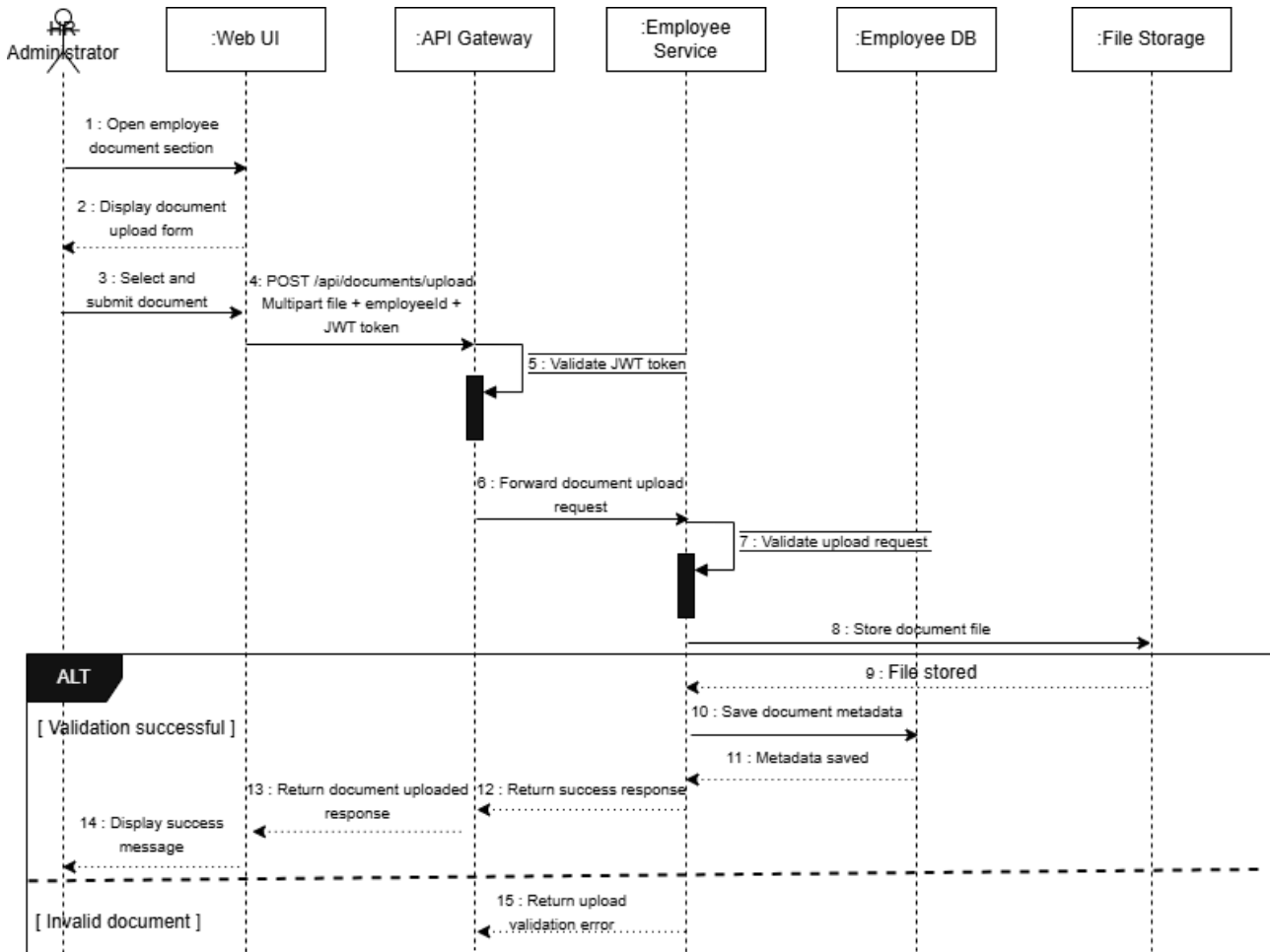


Figure 3.21: Sequence diagram of employee document upload

The upload workflow includes JWT validation, upload request validation, file storage, metadata persistence, and response propagation to the frontend.

3.2.2.4 Sprint 2 Architecture

The architecture of Sprint 2 is based on the integration of the Angular frontend, the API Gateway, the Employee Service, the Employee database, Keycloak, the file storage system, and the email notification service.

The main components involved in this sprint are:

- **Angular Frontend:** provides HR administration interfaces for employees, departments, services, contracts, documents, and organization chart visualization.
- **API Gateway:** validates JWT tokens and routes requests to the Employee Service.
- **Employee Service:** manages employee-related business logic and exposes REST APIs.
- **Employee Database:** stores employees, departments, services, contracts, documents, and related metadata.
- **Keycloak:** creates and manages user accounts associated with employees.
- **File Storage:** stores uploaded employee documents.
- **Email Service:** sends welcome emails containing temporary credentials.

Figure 3.22 illustrates the architecture implemented during Sprint 2.

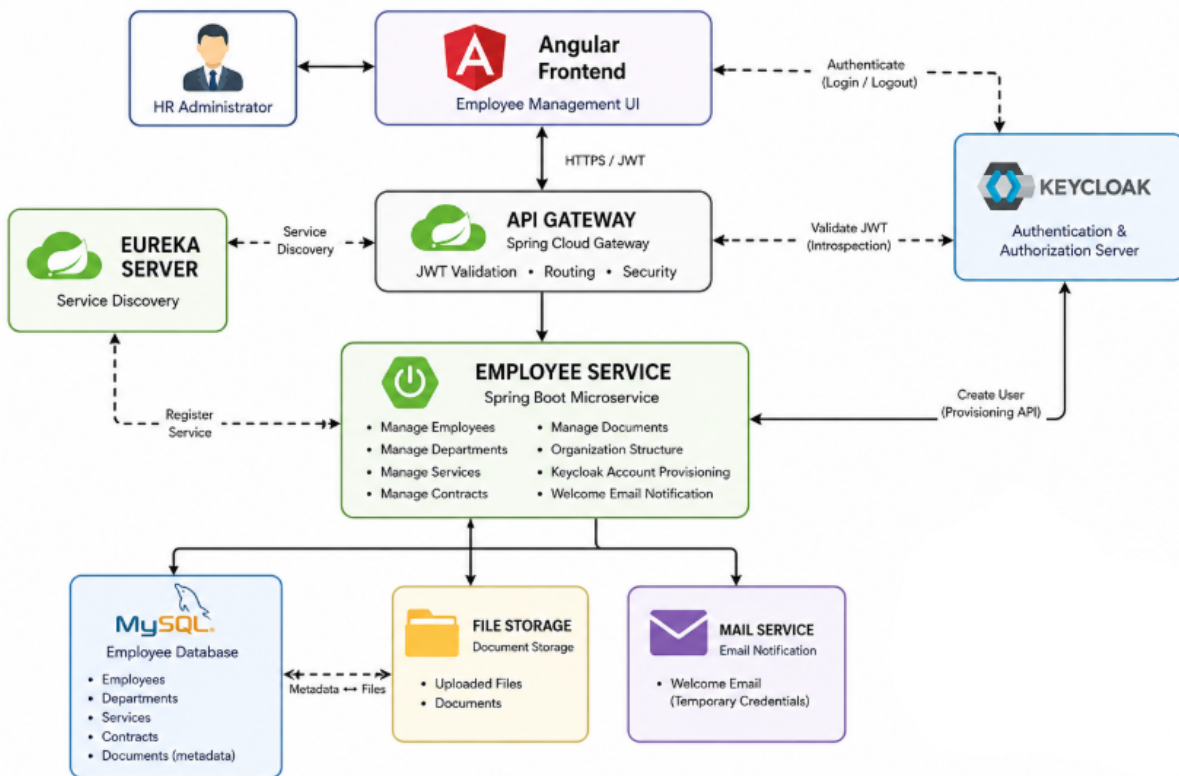


Figure 3.22: Sprint 2 Employee Management architecture

This architecture keeps employee administration functionalities isolated inside a dedicated microservice while ensuring secure communication through the API Gateway and Keycloak.

3.2.3 Realization of Sprint 2

This section presents the practical implementation of Sprint 2 functionalities using the Angular frontend and the Employee Service backend.

3.2.3.1 Employee Management Module

The employee management module provides the HR Administrator with a complete interface to manage employee records. The implemented features include employee creation, consultation, update, deletion, search, filtering, service assignment, contract management, document management, and profile picture upload.

Figure 3.23 presents the employee list interface developed during Sprint 2.

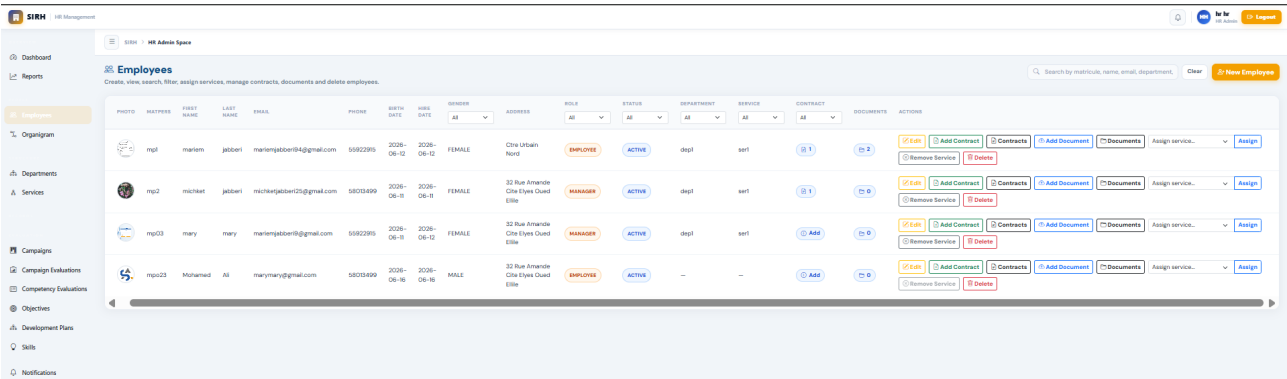


Figure 3.23: Employee list interface

The employee list interface allows the HR Administrator to consult existing employees and access actions such as editing, adding contracts, managing contracts, uploading documents, consulting documents, assigning services, and deleting employees.

Figure 3.24 illustrates the employee creation interface.

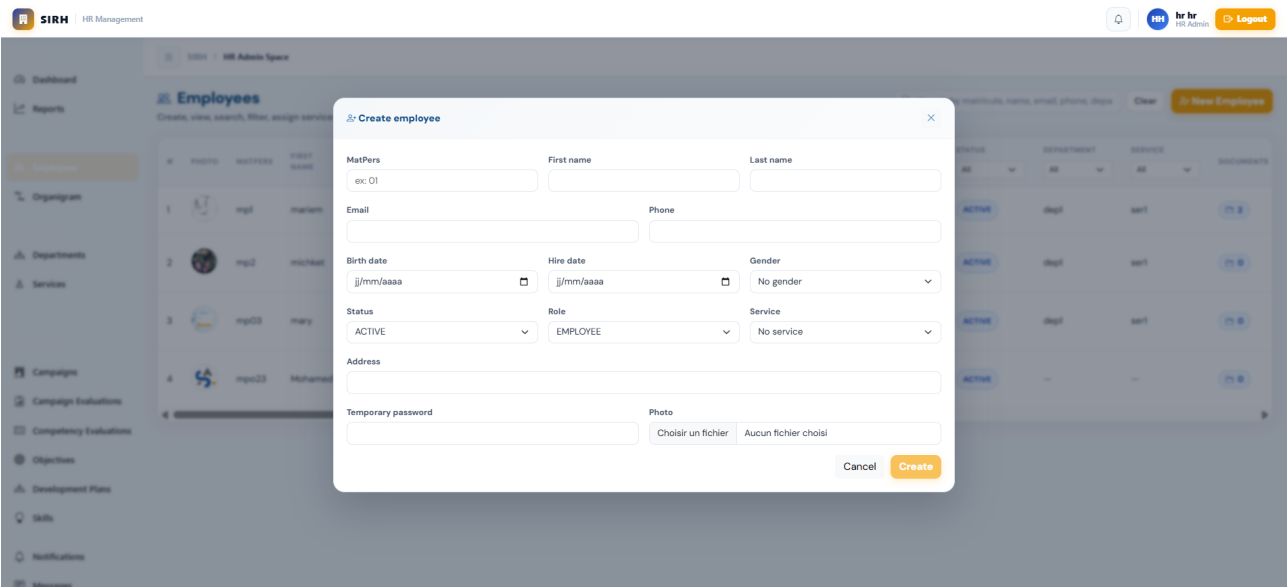


Figure 3.24: Employee creation interface

The employee creation interface collects the required personal and professional information, including identity, contact information, birth date, hire date, gender, status, role, service, address,

temporary password, and photo.

Figure 3.25 presents the employee update interface.

Figure 3.25: Employee update interface

The employee update interface allows the HR Administrator to modify existing employee information and optionally replace the employee photo or temporary password.

3.2.3.2 Automatic Keycloak Account Provisioning

One of the main improvements implemented during Sprint 2 is the automatic creation of a Keycloak account when a new employee is added.

After saving the employee information in the database, the Employee Service sends a request to Keycloak to create a new user account. A temporary password is generated and the account is assigned the appropriate role according to the employee profile.

This mechanism reduces manual administrative work and ensures that each employee created in the SIRH platform can access the application securely.

3.2.3.3 Welcome Email Notification

After successful Keycloak account creation, the system sends a welcome email to the newly created employee. This email contains the username, temporary password, and login URL.

Figure 3.26 illustrates the welcome email generated automatically after employee account creation.

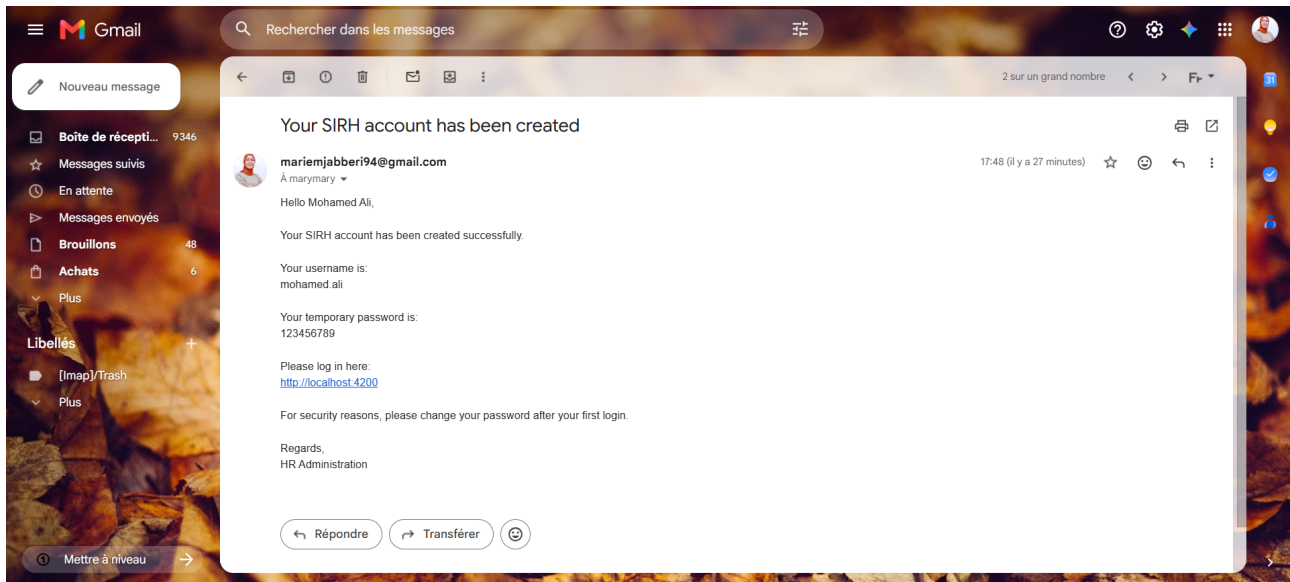


Figure 3.26: Welcome email containing temporary credentials

This feature improves the onboarding process by informing the employee that their account has been created and by providing the initial authentication information.

3.2.3.4 Department Management Module

The department management module allows the HR Administrator to create, update, consult, delete, and assign managers to departments. Departments represent the main organizational units of the company and are used to structure employees and services.

Figure 3.27 presents the department list interface.

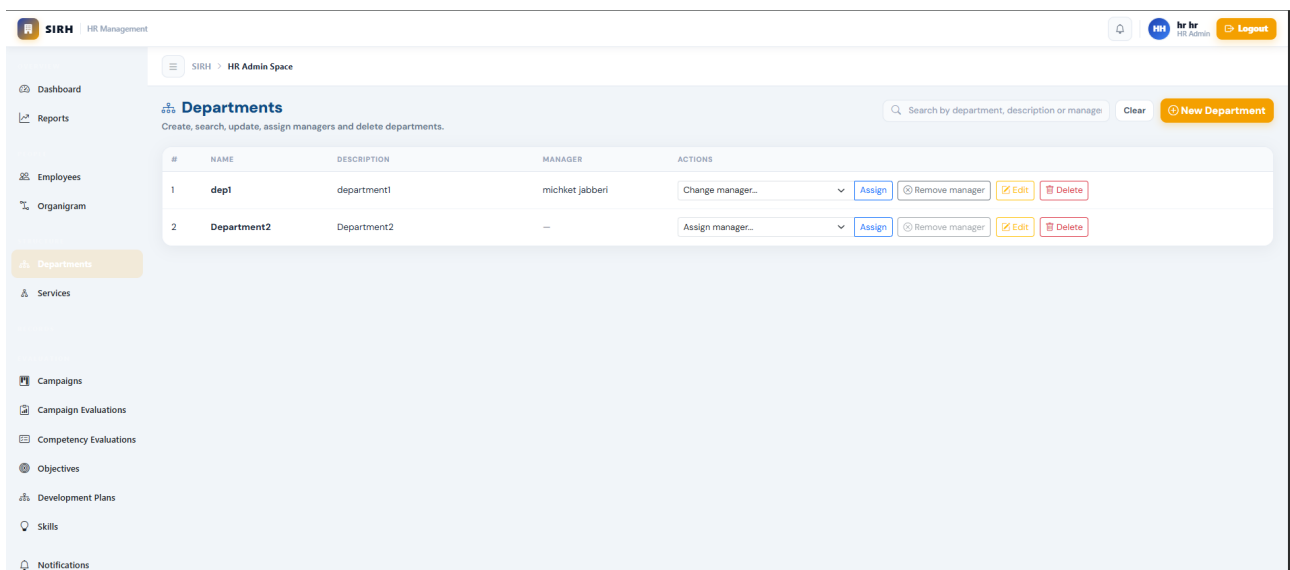


Figure 3.27: Department list interface

The department list interface displays existing departments, their descriptions, assigned managers,

and available actions.

Figure 3.28 illustrates the department creation interface.

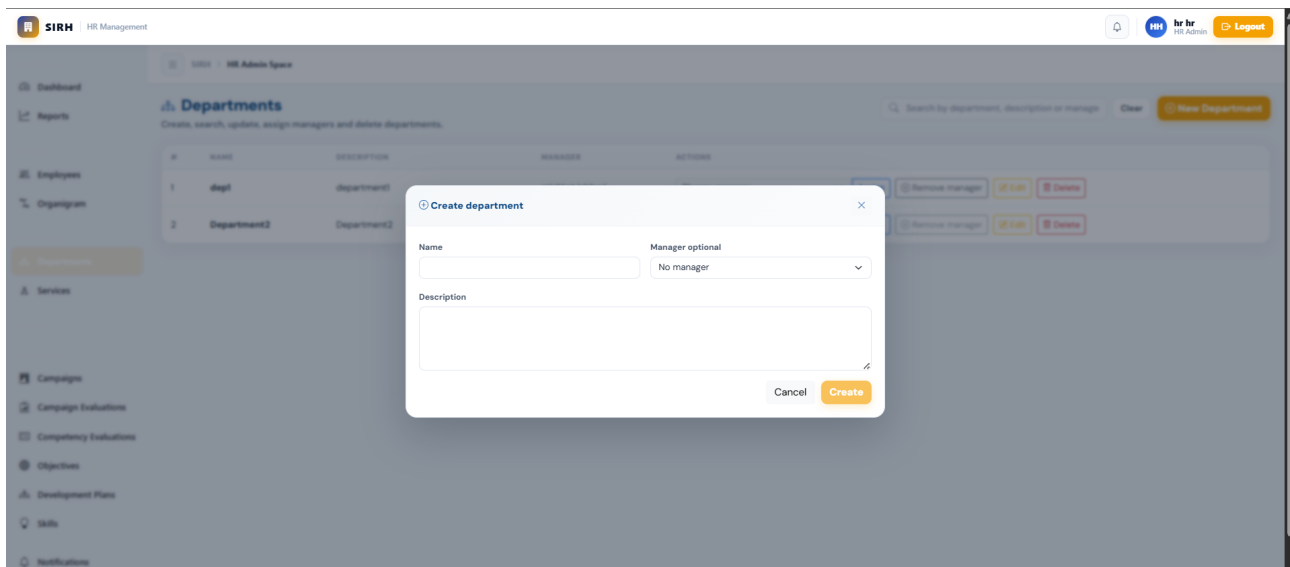


Figure 3.28: Department creation interface

The department creation form allows the HR Administrator to enter the department name, select an optional manager, and provide a description.

Figure 3.29 presents the department update interface.

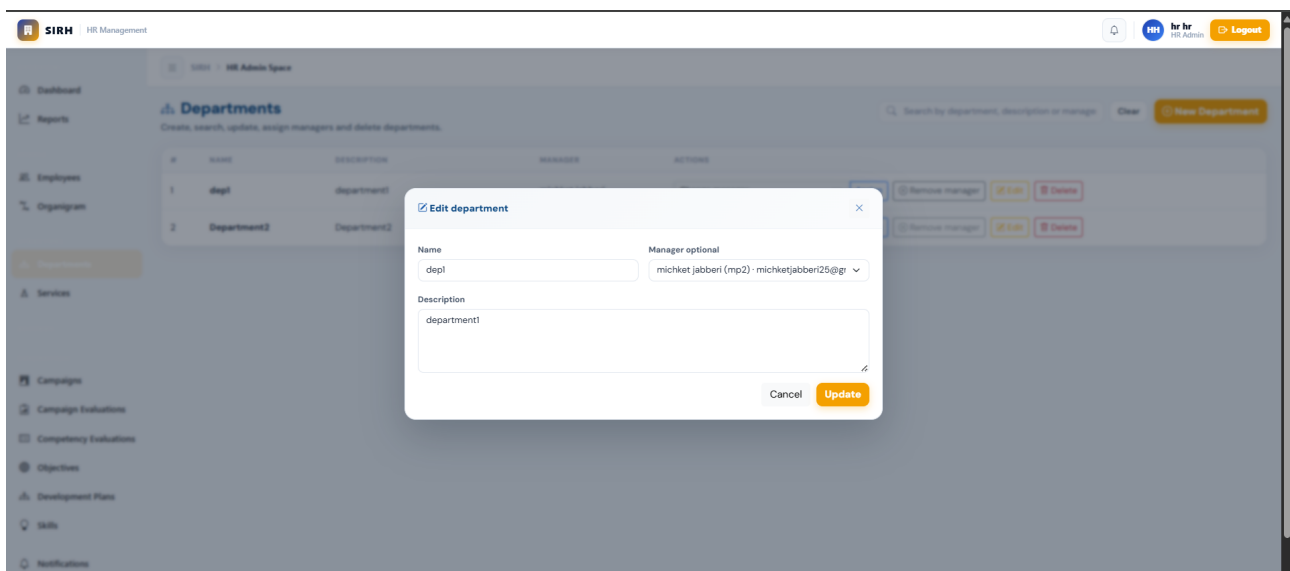


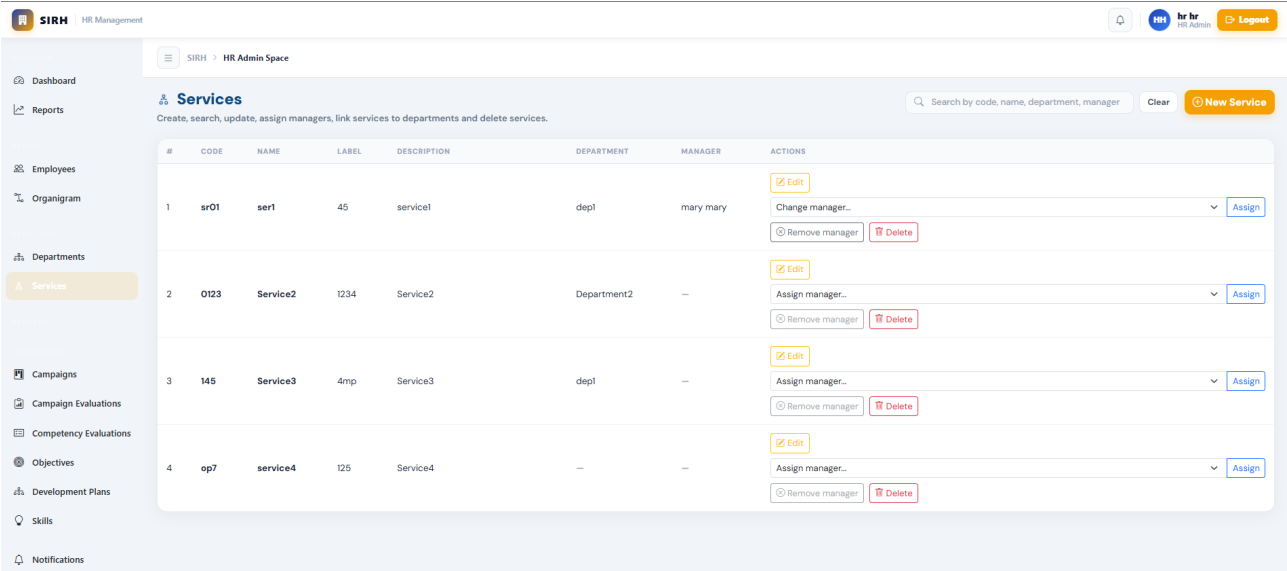
Figure 3.29: Department update interface

The department update interface allows the modification of department information and manager assignment.

3.2.3.5 Service Management Module

The service management module allows the HR Administrator to manage services inside departments. Each service can be associated with a department and can contain multiple employees.

Figure 3.30 presents the service list interface.

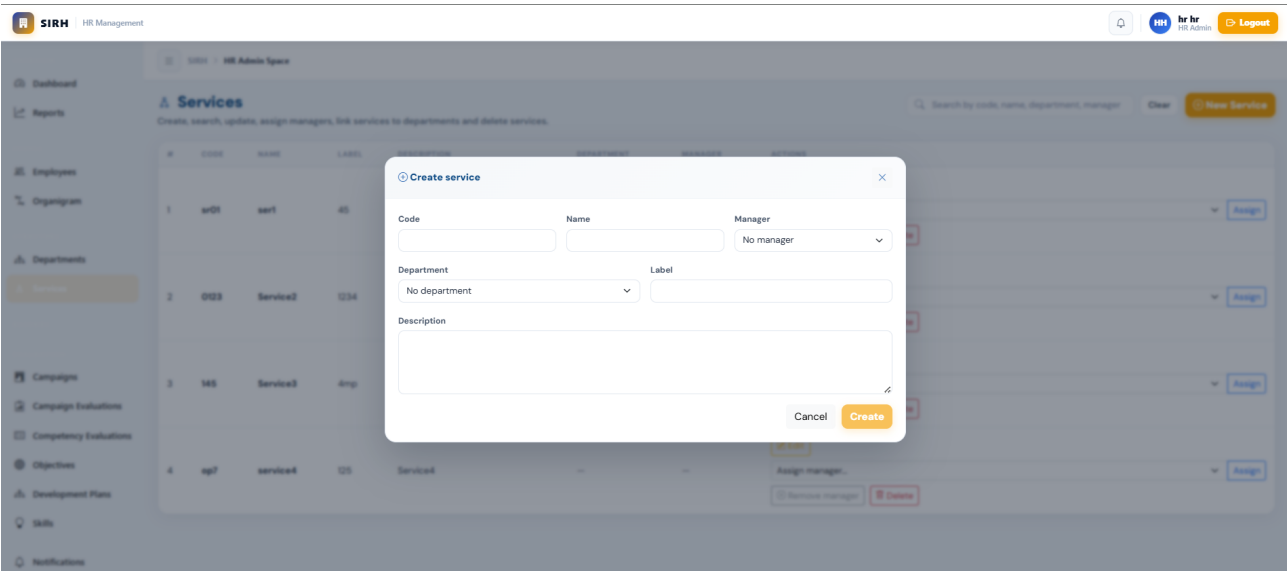


#	CODE	NAME	LABEL	DESCRIPTION	DEPARTMENT	MANAGER	ACTIONS
1	sr01	ser1	45	service1	depl	mary mary	Edit Change manager... Assign Remove manager Delete
2	0123	Service2	1234	Service2	Department2	—	Edit Assign manager... Assign Remove manager Delete
3	145	Service3	4mp	Service3	depl	—	Edit Assign manager... Assign Remove manager Delete
4	op7	service4	125	Service4	—	—	Edit Assign manager... Assign Remove manager Delete

Figure 3.30: Service list interface

The service list interface displays service code, name, label, description, department, manager, and available actions.

Figure 3.31 illustrates the service creation interface.



Create service

Code: Name: Manager:

Department: Label:

Description:

[Cancel](#) [Create](#)

Figure 3.31: Service creation interface

The service creation form allows the HR Administrator to enter the service code, name, label, description, department, and optional manager.

Figure 3.32 presents the service update interface.

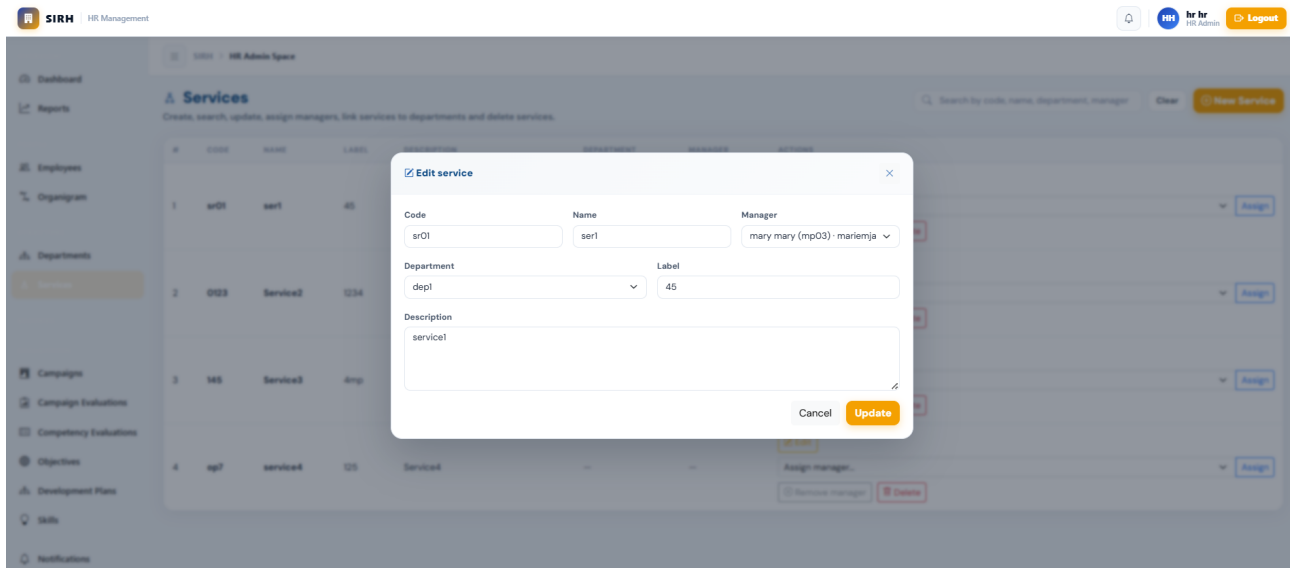


Figure 3.32: Service update interface

The service update interface allows the HR Administrator to modify service information and its organizational assignment.

3.2.3.6 Organizational Structure Visualization

The organization chart module provides a visual representation of the company structure. It displays departments, services, assigned managers, employees inside each service, services without departments, and employees without services.

Figure 3.33 illustrates the organizational structure visualization implemented during Sprint 2.

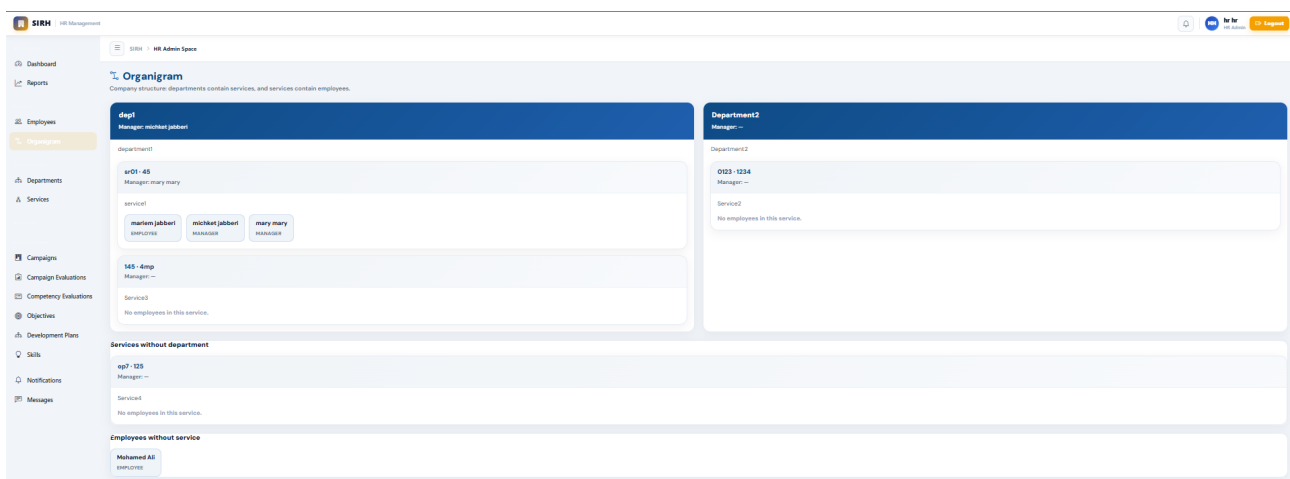


Figure 3.33: Organizational structure visualization

This interface helps the HR Administrator understand and maintain the internal structure of the organization.

3.2.3.7 Contract Management Module

The contract management module allows the HR Administrator to manage employee contracts. The implemented functionalities include contract creation, consultation, update, and deletion.

Each contract is associated with an employee and contains important administrative information such as contract type, start date, and end date.

Figure 3.34 illustrates the contract creation interface.

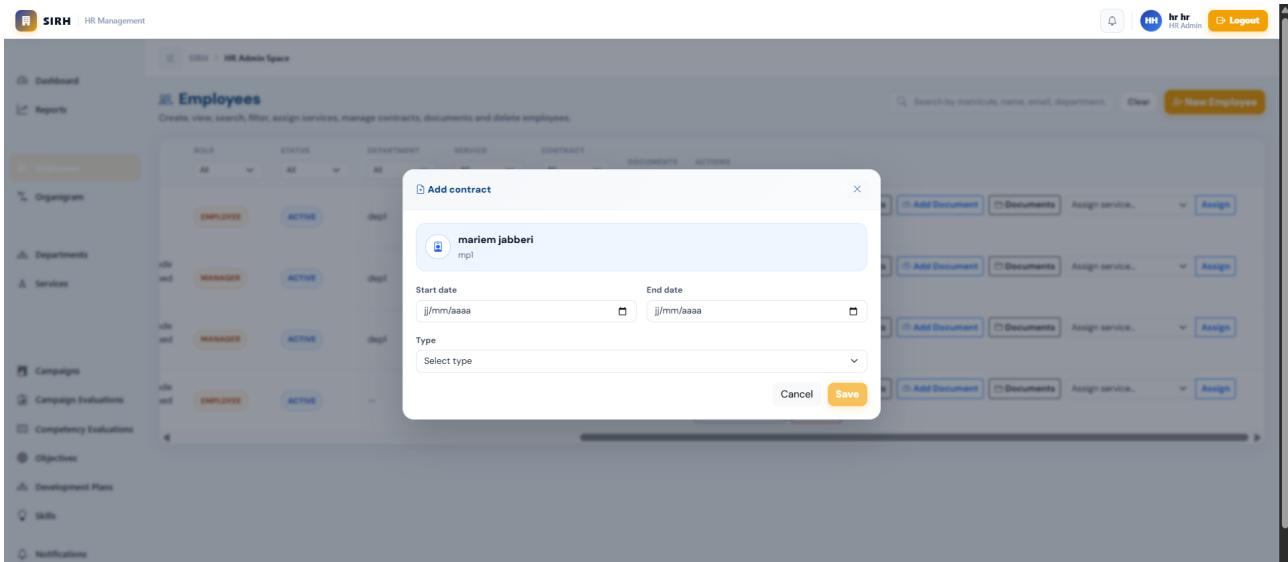


Figure 3.34: Contract creation interface

The contract creation form allows the HR Administrator to create a new contract for a selected employee.

Figure 3.35 presents the employee contracts list.

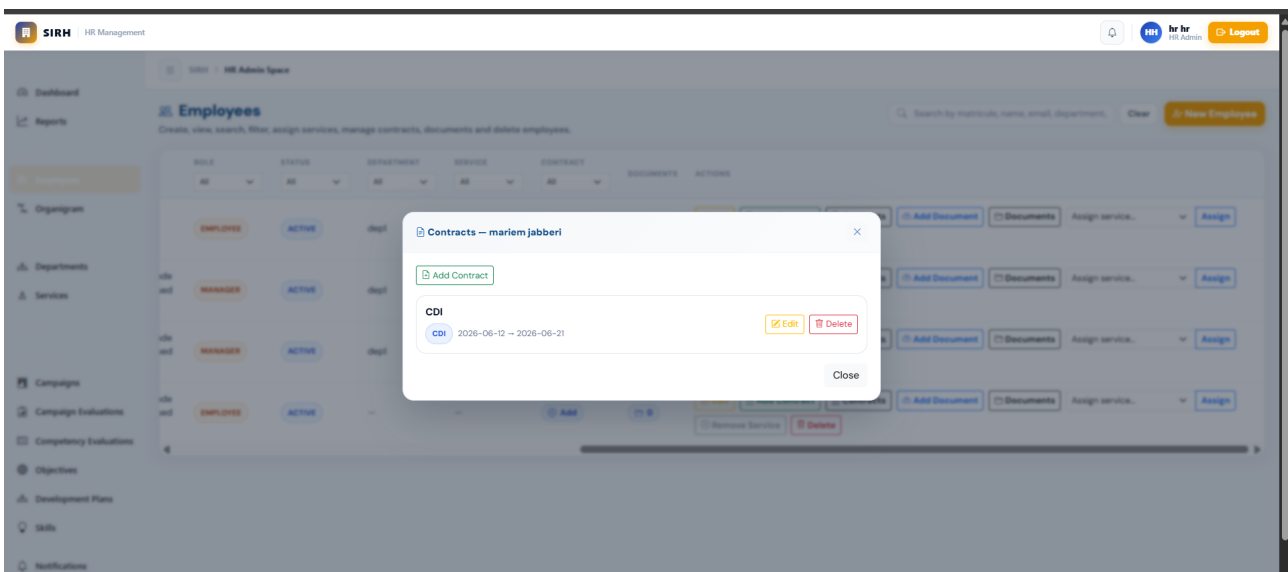


Figure 3.35: Employee contracts list

The contracts list interface displays existing contracts for an employee and provides access to edit and delete actions.

Figure 3.36 illustrates the contract update interface.

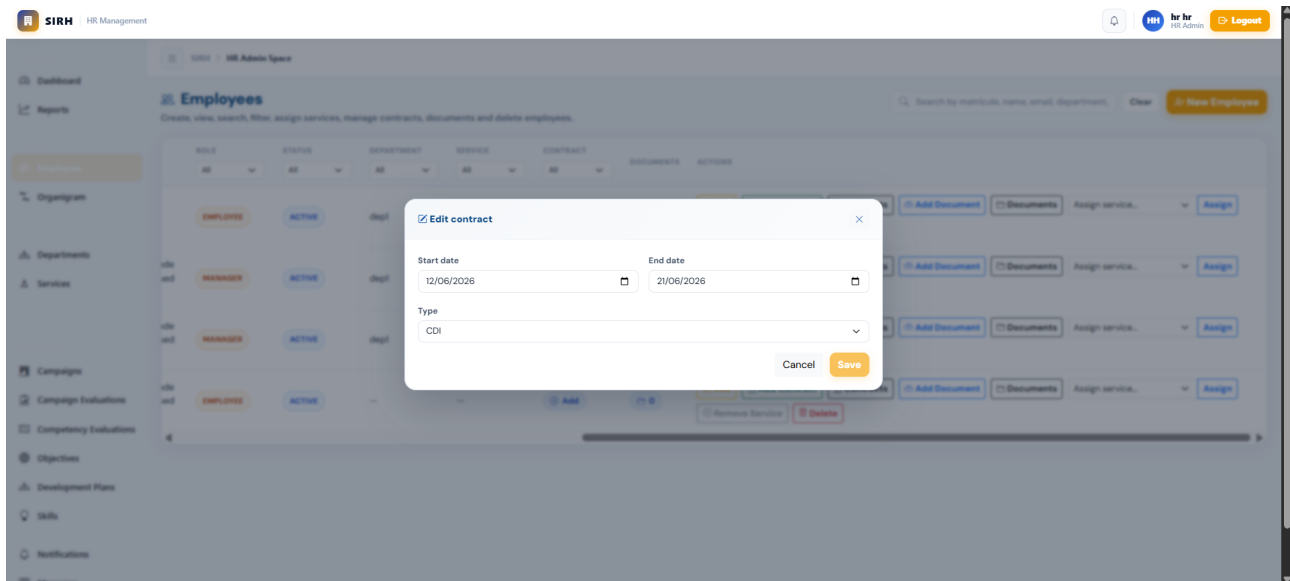


Figure 3.36: Contract update interface

The contract update interface allows the HR Administrator to modify contract dates and type.

3.2.3.8 Document Management Module

The document management module allows the HR Administrator to upload, consult, update, download, and delete employee documents. Documents may include administrative files such as CVs, certificates, contracts, or other HR-related attachments.

When a document is uploaded, the file is stored in the document storage system, while its metadata is saved in the Employee database.

Figure 3.37 illustrates the document upload interface.

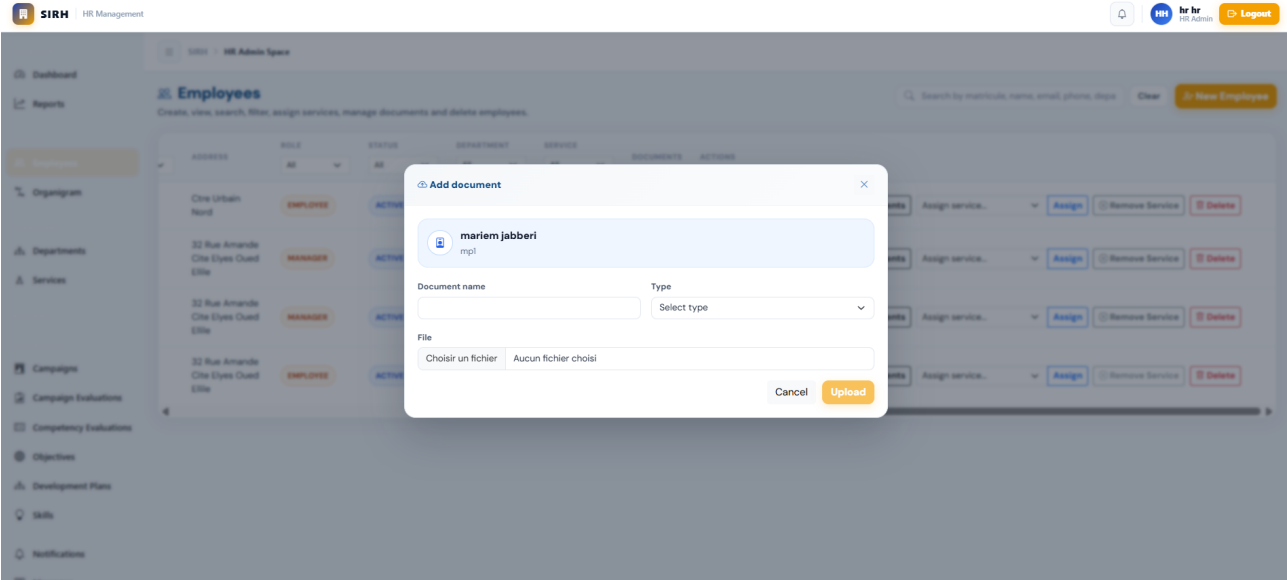


Figure 3.37: Document upload interface

The document upload interface allows the HR Administrator to select the employee, enter the document name, choose the document type, and upload the file.

Figure 3.38 presents the employee documents list.

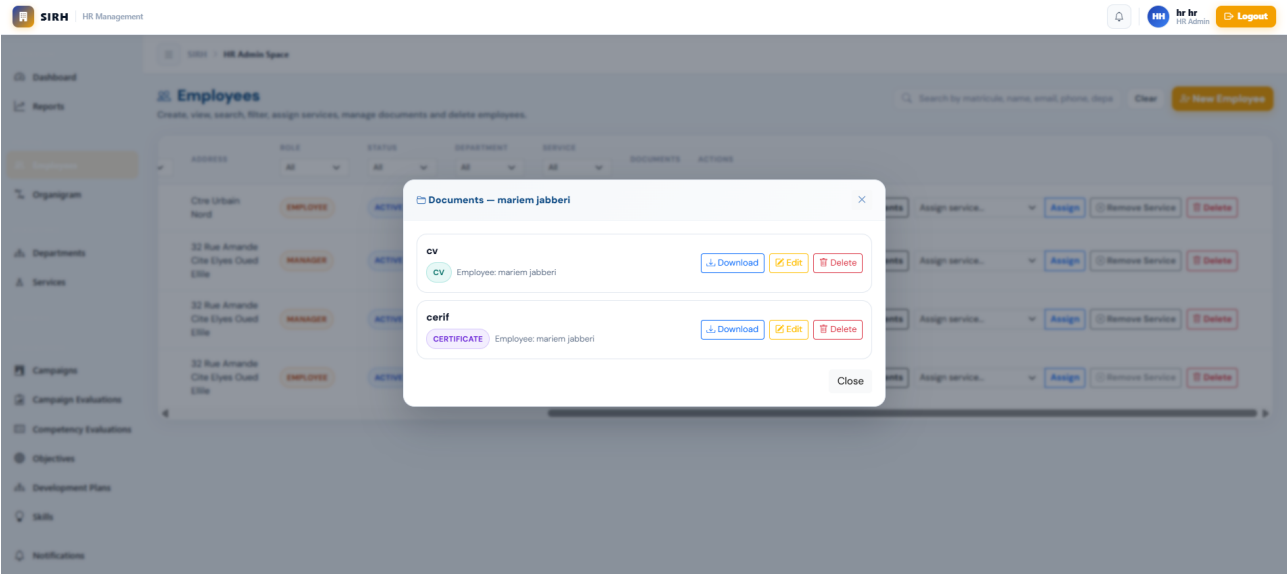


Figure 3.38: Employee documents list

The document list interface displays all documents associated with a selected employee and provides download, edit, and delete actions.

Figure 3.39 illustrates the document update interface.

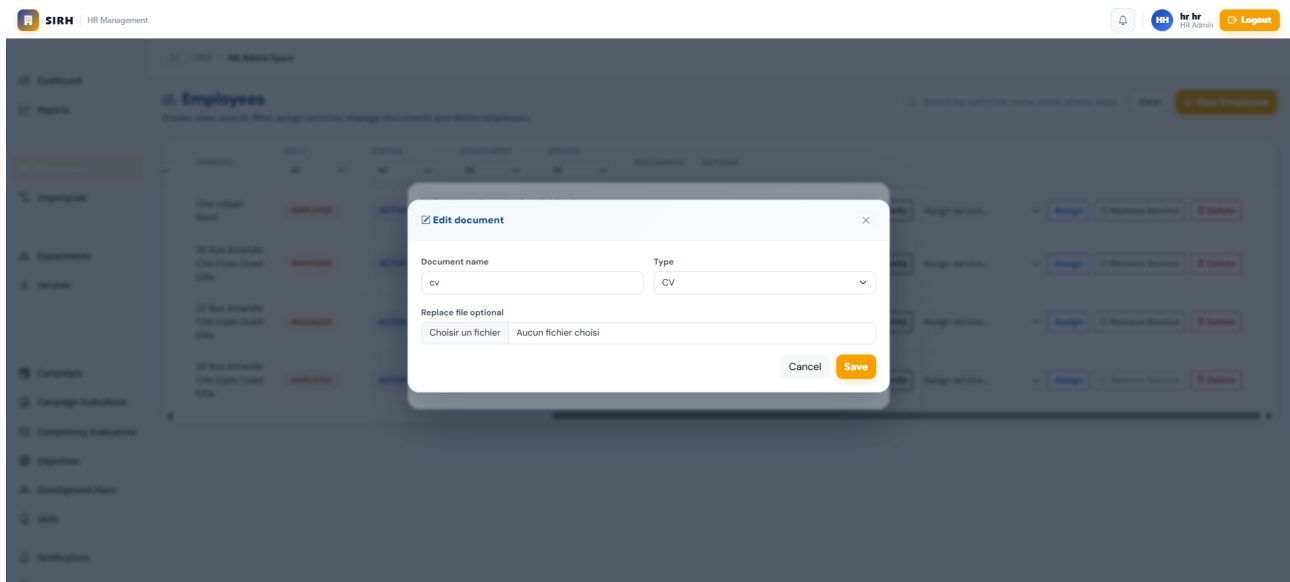


Figure 3.39: Document update interface

The document update interface allows the HR Administrator to modify document metadata and optionally replace the uploaded file.

Conclusion

This release delivered the technical and administrative foundation of the SIRH platform. Sprint 1 established secure authentication, JWT validation, Eureka service discovery, and API Gateway routing. Sprint 2 implemented the Employee Service and the main HR administration functionalities.

The next chapter presents Release 2, which focuses on Evaluation Management, AI-assisted training recommendations, development plans, dashboards, and reports.

RELEASE 2: EVALUATION MANAGEMENT AND DECISION SUPPORT

Plan

Introduction	67
1 Sprint 3: Evaluation Management	67
2 Sprint 4: Development and Decision Support	84
Conclusion	99

Introduction

This chapter presents the second release of the SIRH platform. This release groups Sprint 3 and Sprint 4 and focuses on performance evaluation, competency assessment, objectives, development plans, Artificial Intelligence based training recommendations, dashboards, and reports.

Sprint 3 introduces the Evaluation Service and implements the evaluation workflow, including campaign management, self-evaluation, manager evaluation, skills, objectives, validation, and notifications. Sprint 4 extends the Evaluation Service with development plans, AI-assisted recommendations, training catalogue management, dashboards, and reporting functionalities.

4.1 Sprint 3: Evaluation Management

4.1.1 Sprint 3 Backlog

The Sprint 3 backlog presented in Table 4.1 groups the user stories related to evaluation campaigns, campaign scope, skills, objectives, self-evaluation, manager evaluation, evaluation validation, and notifications.

Table 4.1: Sprint 3 backlog

ID	User Story	Priority	Estimate
US-01	As an HR Administrator, I want to manage evaluation campaigns so that evaluations can be organized by period, type, skills, departments, and services.	High	2 days
US-02	As an HR Administrator, I want to define campaign scope so that evaluations are generated only for employees belonging to selected departments and services.	High	2 days
US-03	As an HR Administrator, I want to manage skills so that competency evaluations can be based on structured criteria.	High	2 days
US-04	As a Manager, I want to create and follow employee objectives so that weighted tasks and progress can be tracked.	High	2 days
US-05	As an Employee, I want to complete my self-evaluation so that I can assess my own competencies and objectives.	High	2 days

ID	User Story	Priority	Estimate
US-06	As a Manager, I want to evaluate employee competencies so that I can complete the managerial assessment.	High	3 days
US-07	As an HR Administrator, I want to review and validate evaluations so that the evaluation process can be finalized.	High	2 days
US-08	As a system, I want to notify users about evaluation actions so that employees, managers, and HR administrators stay informed.	Medium	1 day
Total			16 days

4.1.2 Sprint 3 Analysis and Design

This section presents the analysis and design of Sprint 3. The objective of this sprint is to implement the Evaluation Service as an independent microservice responsible for campaign management, skills, objectives, self-evaluation, manager evaluation, evaluation validation, and notifications.

4.1.2.1 Use-Case Model of Sprint 3

The use-case model of Sprint 3 focuses on the Evaluation Service. The main actors involved in this sprint are the HR Administrator, the Manager, and the Employee.

Figure 4.1 illustrates the use-case diagram of Sprint 3 and summarizes the main interactions between the actors and the evaluation management module.

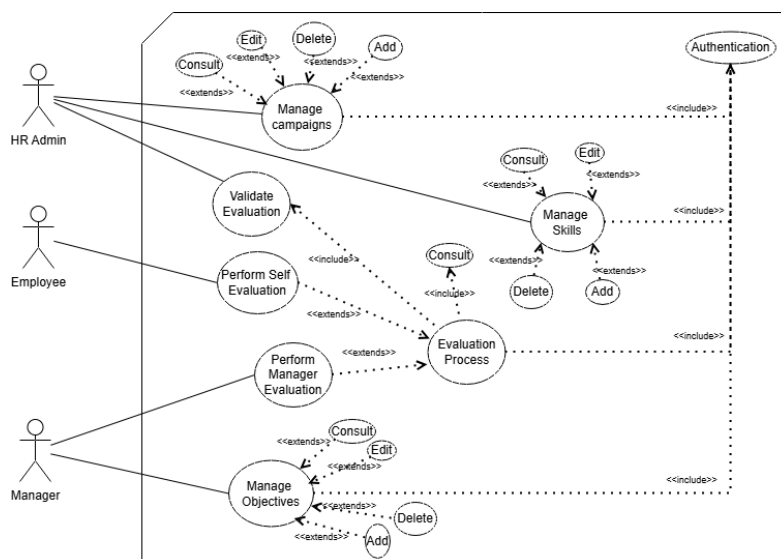


Figure 4.1: Use-case diagram of Sprint 3

The main use cases covered during this sprint are campaign management, skill management, objective management, self-evaluation, manager evaluation, evaluation validation, and authentication.

4.1.2.2 Evaluation Service Design

The Evaluation Service is an independent backend microservice dedicated to performance evaluation. It is integrated into the existing architecture through the API Gateway and Eureka Server.

Its main responsibilities are:

- Manage evaluation campaigns.
- Manage campaign scope through departments and services.
- Manage skills and competency questions.
- Generate campaign evaluations.
- Handle employee self-evaluations.
- Handle manager evaluations.
- Manage objectives and weighted tasks.
- Manage evaluation status transitions.
- Generate evaluation notifications.

This design separates evaluation-related business logic from employee administration, which remains handled by the Employee Service.

4.1.2.3 Evaluation Campaign Workflow

The evaluation campaign workflow starts when the HR Administrator creates a campaign from the web interface. The request is sent through the API Gateway with a JWT token. After validation, the gateway discovers the Evaluation Service through Eureka Server and forwards the request.

The Evaluation Service validates the campaign data, saves the campaign, generates evaluation records for targeted employees, creates competency questions, and sends notifications to the concerned users.

Figure 4.2 presents the sequence diagram of the evaluation campaign workflow.

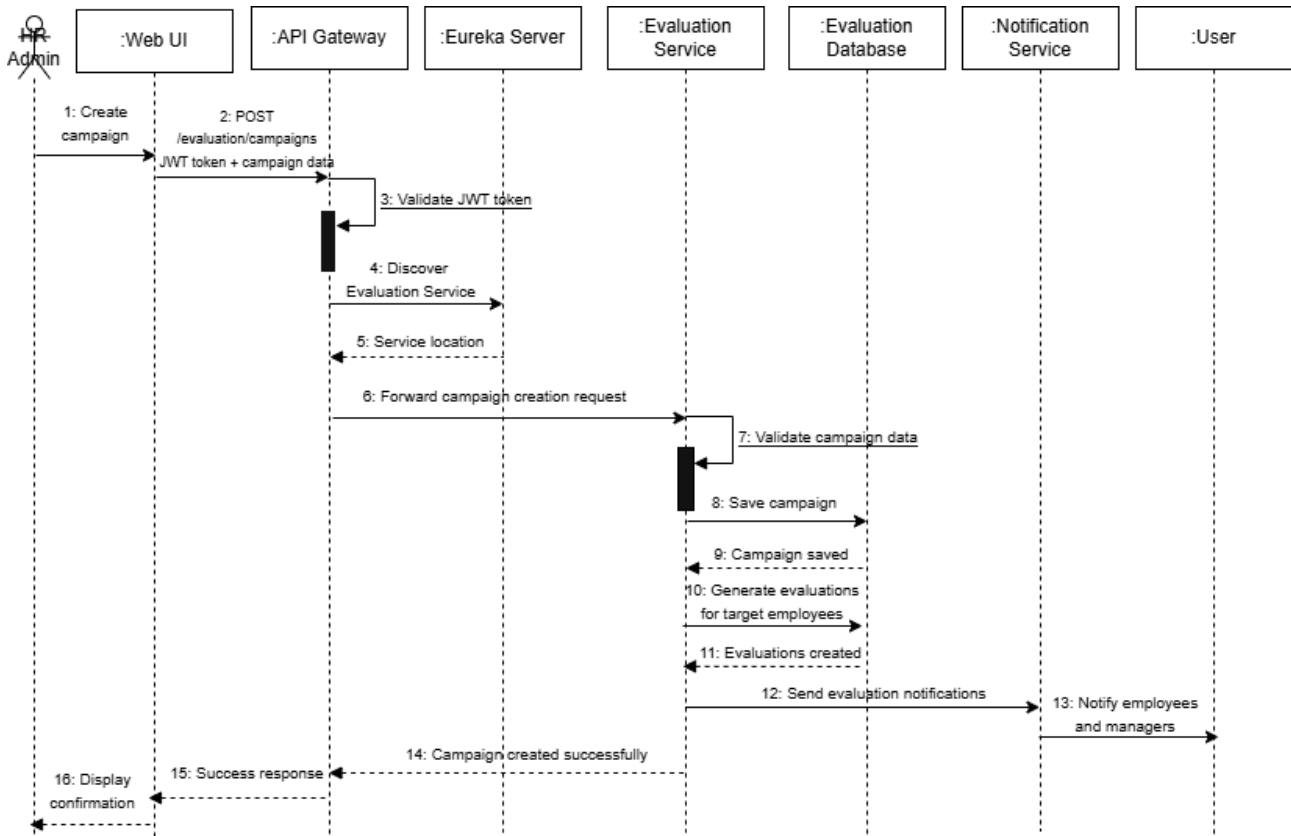


Figure 4.2: Sequence diagram of the evaluation campaign workflow

4.1.2.4 Self-Evaluation Workflow

The self-evaluation workflow starts when an employee opens an assigned evaluation. The system retrieves the evaluation details and displays the competency questions. After the employee submits scores and comments, the Evaluation Service stores the self-evaluation result, updates the evaluation status to `SELF_COMPLETED`, and creates a notification for the manager.

Figure 4.3 presents the sequence diagram of the self-evaluation workflow.

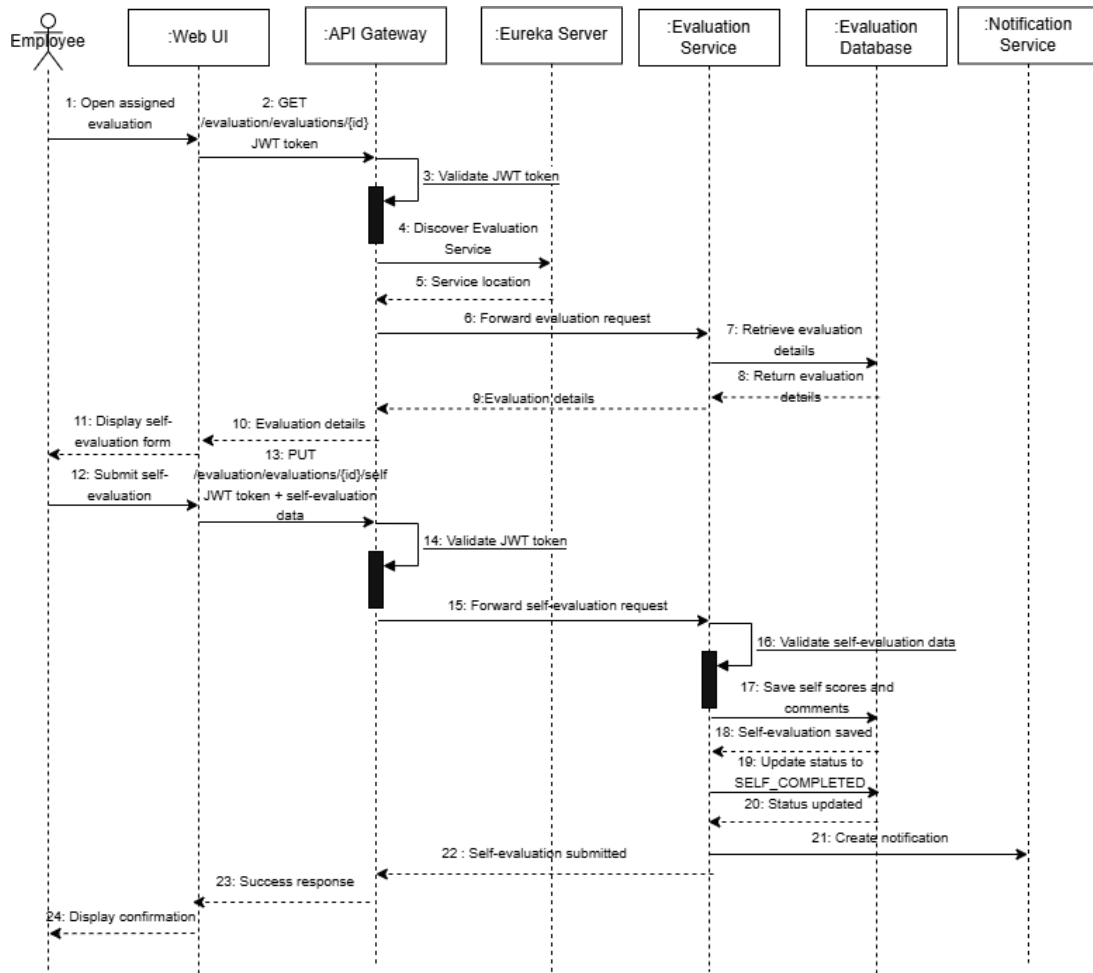


Figure 4.3: Sequence diagram of the self-evaluation workflow

4.1.2.5 Evaluation Status Lifecycle

The evaluation process is controlled using several statuses. Table 4.2 describes the lifecycle followed by an evaluation from its creation to its archive state.

Table 4.2: Evaluation status lifecycle

Status	Description
PENDING	The evaluation is created but not yet started.
IN_PROGRESS	The evaluation process is currently active.
SELF_COMPLETED	The employee has submitted the self-evaluation.
MANAGER_COMPLETED	The manager has submitted the managerial evaluation.
VALIDATED	The HR Administrator has validated the final evaluation.
ARCHIVED	The evaluation has been archived after completion.

4.1.2.6 Sprint 3 Architecture

Sprint 3 reuses the technical foundation implemented in Sprint 1 and extends it with the Evaluation Service. The service is registered in Eureka Server and accessed through the API Gateway. Authentication and authorization are ensured through Keycloak and JWT tokens.

Figure 4.4 illustrates the architecture implemented during Sprint 3.

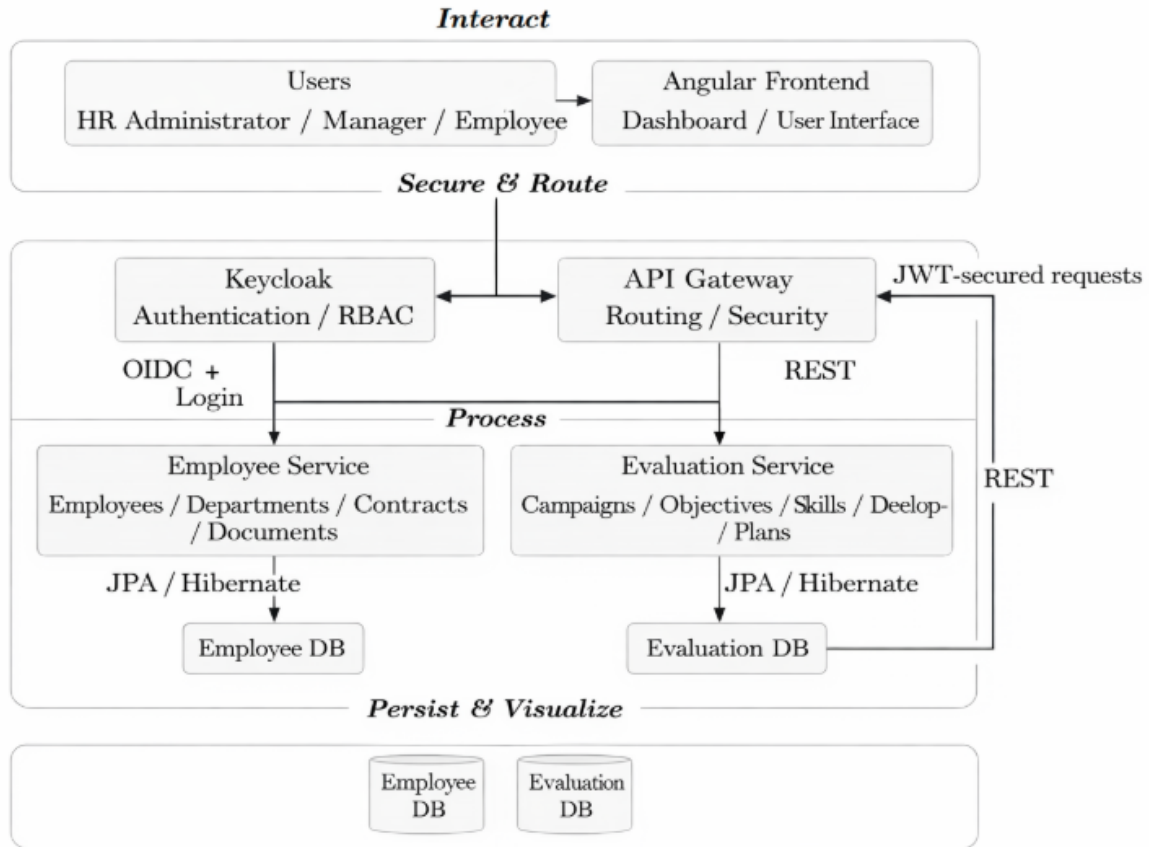


Figure 4.4: Sprint 3 architecture of the Evaluation Service

The architecture includes the Angular Web UI, API Gateway, Eureka Server, Evaluation Service, Evaluation Database, Keycloak authentication server, and notification mechanism.

4.1.3 Realization of Sprint 3

This section presents the practical implementation of Sprint 3 functionalities using the Angular frontend and the Evaluation Service backend.

4.1.3.1 Skills Management Module

The skills management module allows the HR Administrator to create, update, consult, filter, and delete evaluation skills. Each skill is defined by a name, category, description, and required

level.

Figure 4.5 presents the skills management interface.

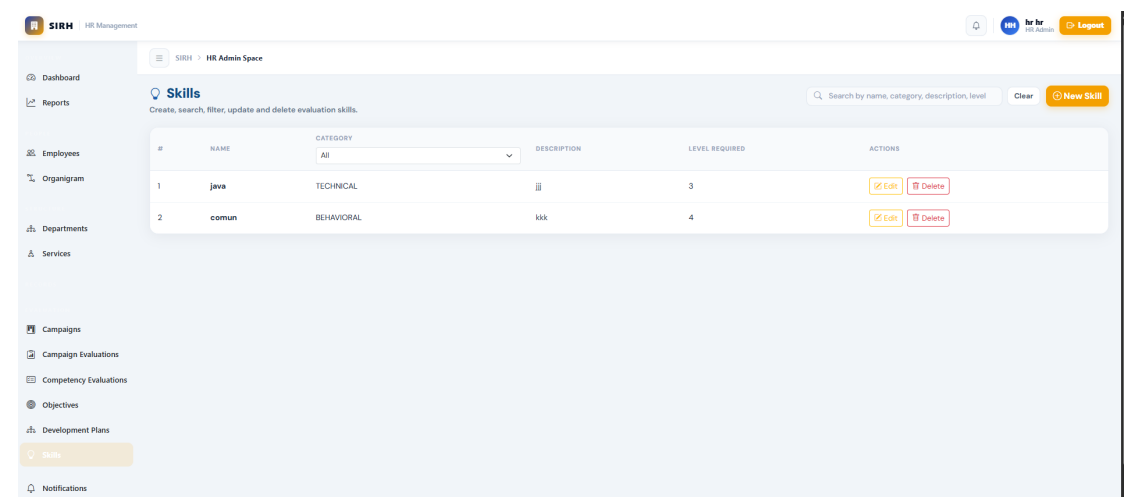


Figure 4.5: Skills management interface

Figure 4.6 illustrates the skill creation interface.

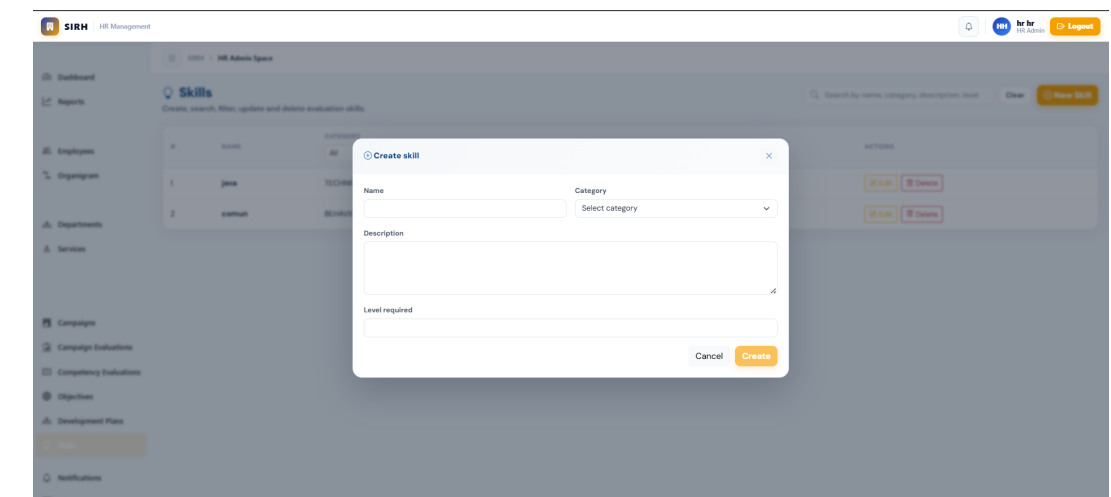


Figure 4.6: Skill creation interface

Figure 4.7 presents the skill update interface.

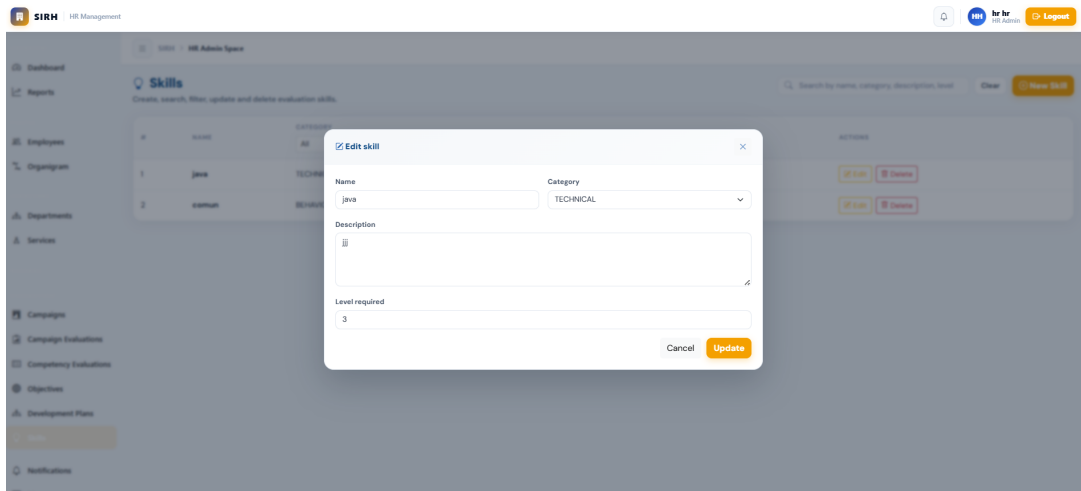


Figure 4.7: Skill update interface

4.1.3.2 Evaluation Campaign Management Module

The campaign management module allows the HR Administrator to create, consult, update, delete, archive, and open evaluation campaigns. A campaign contains its name, year, type, status, period, related skills, questions, departments, and services.

Figure 4.8 presents the evaluation campaigns interface.

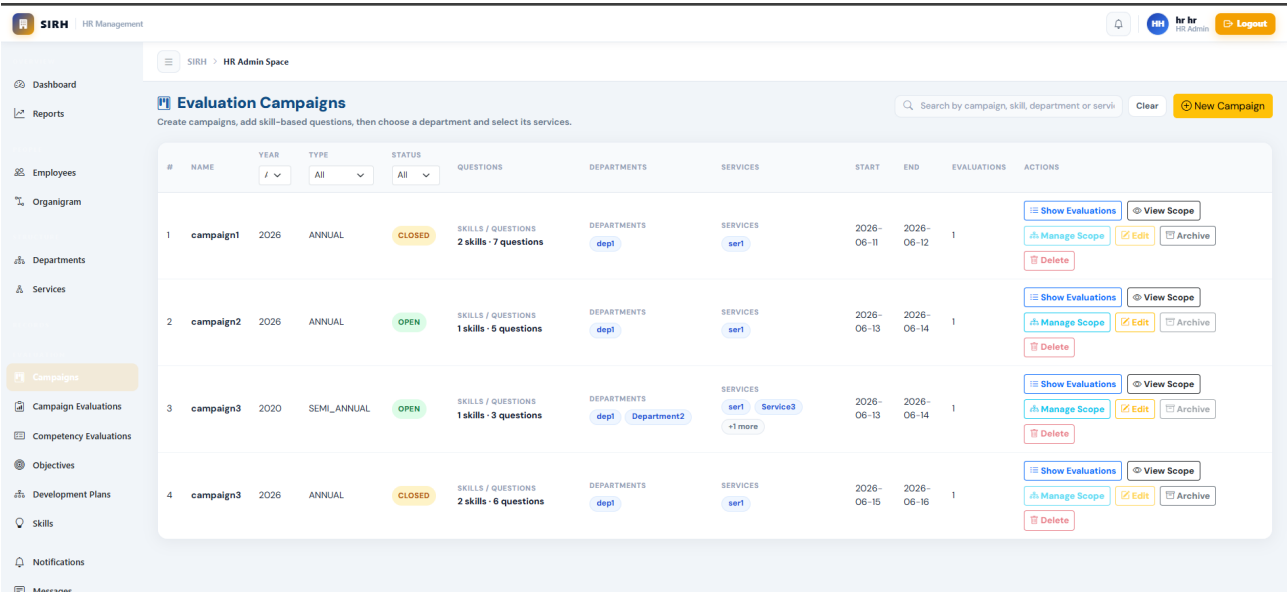


Figure 4.8: Evaluation campaigns interface

Figure 4.9 illustrates the evaluation campaign creation interface.

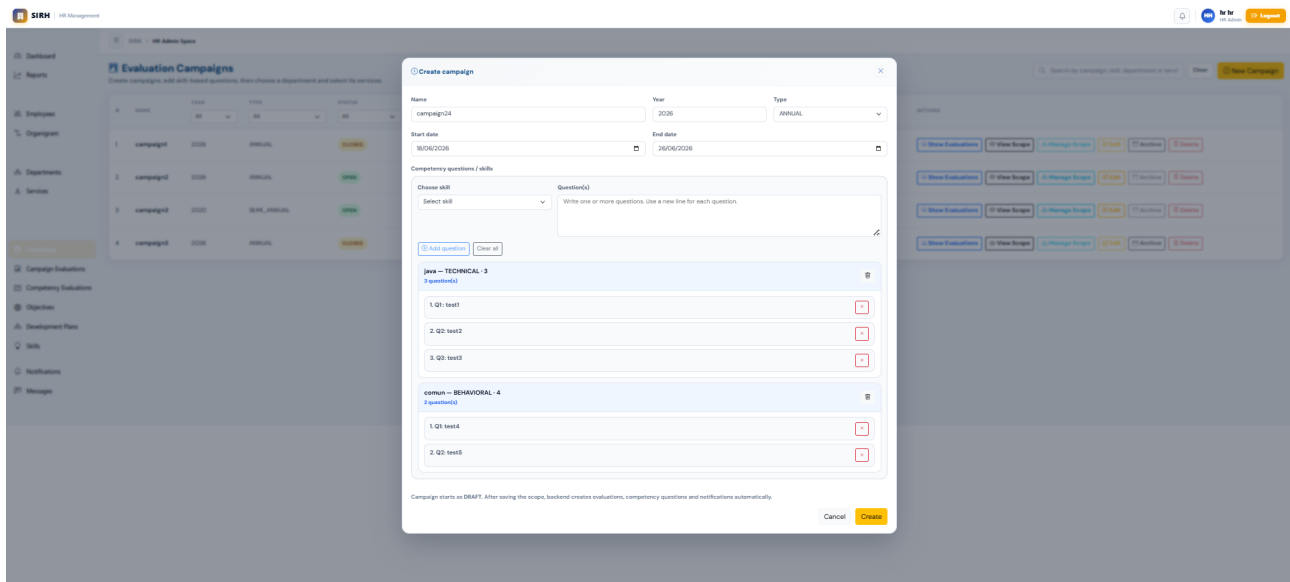


Figure 4.9: Evaluation campaign creation interface

The campaign creation form allows the HR Administrator to define campaign information and add competency questions grouped by skill.

Figure 4.10 presents the evaluation campaign update interface.

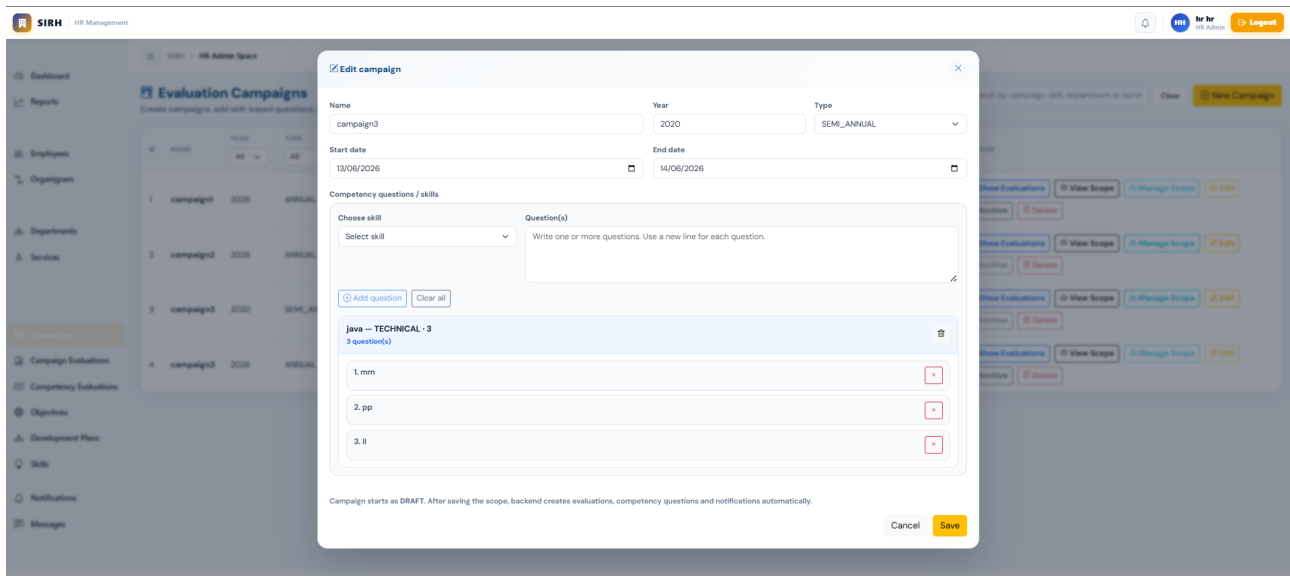


Figure 4.10: Evaluation campaign update interface

4.1.3.3 Campaign Scope Management

After creating a campaign, the HR Administrator defines its scope by selecting departments and services. Evaluations are then generated for employees belonging to the selected services.

Figure 4.11 illustrates the department and service assignment interface.

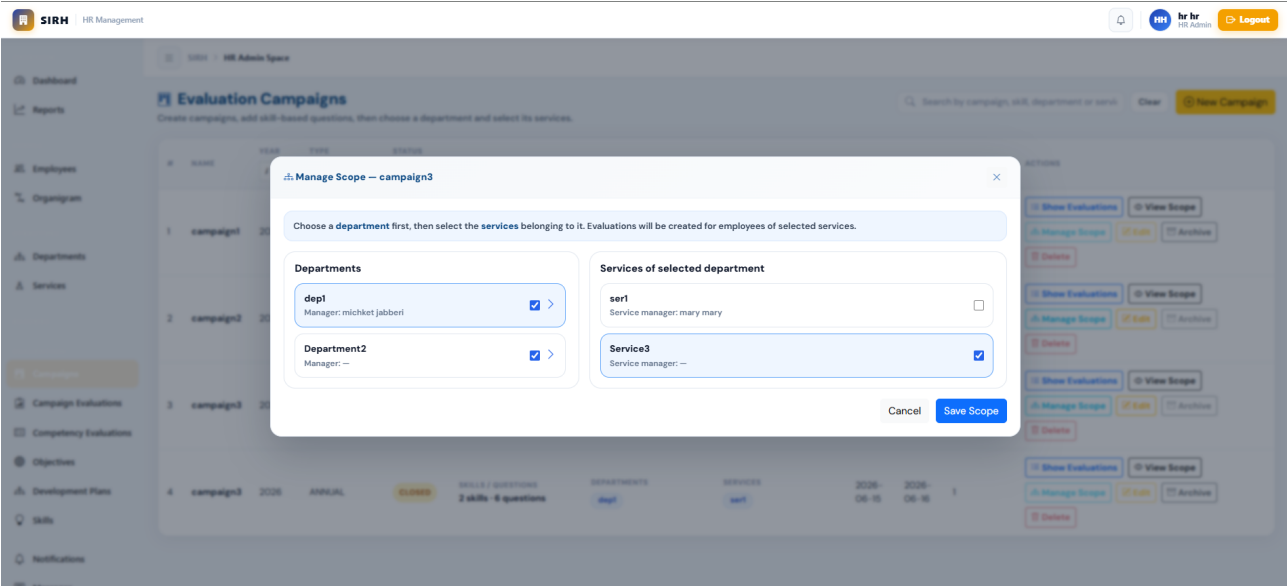


Figure 4.11: Department and service assignment interface

Figure 4.12 presents the campaign scope consultation interface.

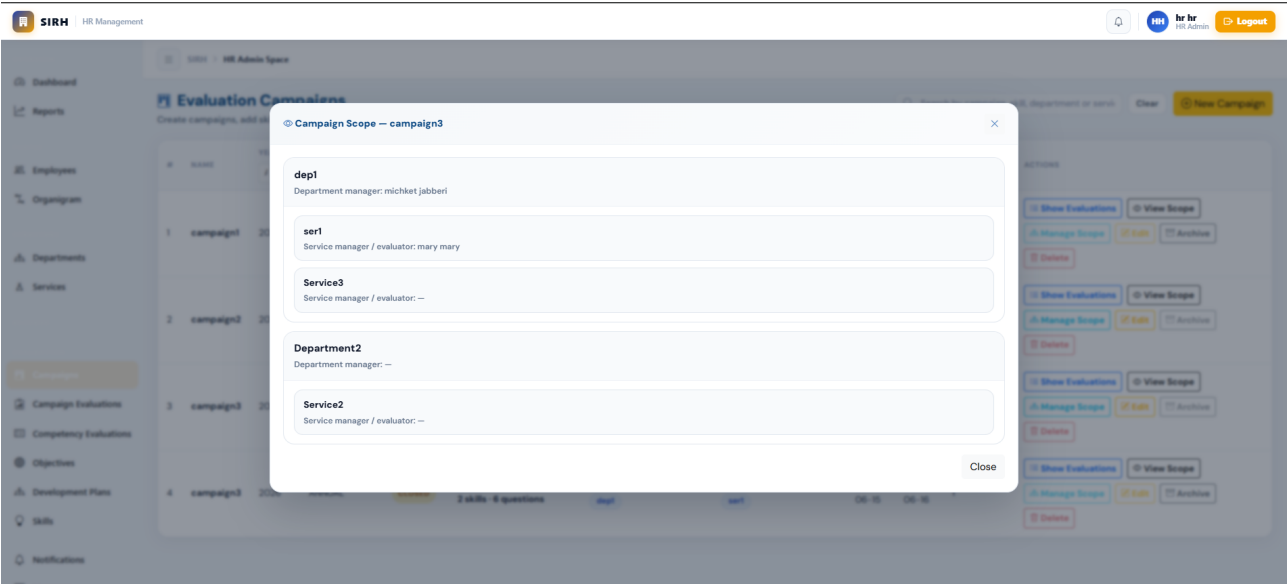


Figure 4.12: Campaign scope consultation interface

4.1.3.4 Campaign Evaluations Module

The campaign evaluations module allows the HR Administrator to review evaluations generated by campaigns, consult competency results, validate completed evaluations, assign development plans, and delete evaluations when necessary.

Figure 4.13 presents the campaign evaluations interface.

#	CAMPAIGN	EMPLOYEE	EVALUATOR	STATUS	SELF SCORE	MANAGER SCORE	FINAL SCORE	CREATED AT	SUBMITTED AT	VALIDATED AT	SUGGESTED DEV PLAN	ACTIONS
1	campaign1	marjem jabberi	mary mary	VALIDATED	2.71	2.82	2.85	2026-06-12	2026-06-12	2026-06-12	—	Show Competencies Validate Assign Dev Plan View Dev Plans Delete
2	campaign2	marjem jabberi	mary mary	SELF_COMPLETED	2.80	—	—	2026-06-13	2026-06-13	—	—	Show Competencies Validate Assign Dev Plan View Dev Plans Delete
3	campaign3	marjem jabberi	mary mary	SELF_COMPLETED	3.33	—	—	2026-06-13	2026-06-13	—	—	Show Competencies Validate Assign Dev Plan View Dev Plans Delete
4	campaign3	marjem jabberi	mary mary	VALIDATED	2.67	2.50	2.56	2026-06-15	2026-06-15	2026-06-15	—	Show Competencies Validate Assign Dev Plan View Dev Plans Delete

Figure 4.13: Campaign evaluations interface

Figure 4.14 illustrates the evaluations of a selected campaign.

#	EMPLOYEE	EVALUATOR	STATUS	SELF SCORE	MANAGER SCORE	FINAL SCORE	CREATED AT	SUBMITTED AT	VALIDATED AT	SUGGESTED DEV PLAN	ACTIONS
1	marjem jabberi	mary mary	VALIDATED	2.67	2.50	2.56	2026-06-15	2026-06-15	2026-06-15	—	Show Competencies Validate Assign Dev Plan View Dev Plans Delete

Figure 4.14: Evaluations of a selected campaign

4.1.3.5 Competency Evaluation Module

The competency evaluation module displays the detailed competency results linked to evaluations. It includes employee scores, manager scores, final scores, employee comments, and manager comments.

Figure 4.15 presents the competency evaluations interface.

Evaluation Competencies
Review competency results and question scores linked to this evaluation.

Search by campaign, employee, skill or score...

#	CAMPAIGN	EMPLOYEE	SKILL	CATEGORY	EMPLOYEE SCORE	MANAGER SCORE	FINAL SCORE	EMPLOYEE COMMENT	MANAGER COMMENT	ACTIONS
1	campaign3	marlem jabberi	java	TECHNICAL	2.33	2.33	2.33	jjk	hju	Show Questions
2	campaign3	marlem jabberi	comun	BEHAVIORAL	3	2.67	2.78	rk	hjh	Show Questions

Figure 4.15: Competency evaluations interface

Figure 4.16 presents the all competency evaluations interface.

All Competency Evaluations
Review all competency evaluations across campaigns.

Search by campaign, employee, skill or score...

#	CAMPAIGN	EMPLOYEE	SKILL	CATEGORY	EMPLOYEE SCORE	MANAGER SCORE	FINAL SCORE	EMPLOYEE COMMENT	MANAGER COMMENT	ACTIONS
1	campaign1	marlem jabberi	java	TECHNICAL	2.75	2.50	2.58	jjk	hju	Show Questions
2	campaign1	marlem jabberi	comun	BEHAVIORAL	2.67	3.33	3.33	rk	hjh	Show Questions
3	campaign2	marlem jabberi	java	TECHNICAL	2.80	2.80	2.80	jjk	hju	Show Questions
4	campaign3	marlem jabberi	comun	BEHAVIORAL	3.33	2.33	2.33	rk	hjh	Show Questions
5	campaign3	marlem jabberi	java	TECHNICAL	2.33	2.33	2.33	jjk	hju	Show Questions
6	campaign3	marlem jabberi	comun	BEHAVIORAL	3	2.67	2.78	rk	hjh	Show Questions

Figure 4.16: All competency evaluations interface

Figure 4.17 illustrates the competency questions and scores interface.

Questions - java

EMPLOYEE SCORE: 2.33, MANAGER SCORE: 2.33, FINAL SCORE: 2.33

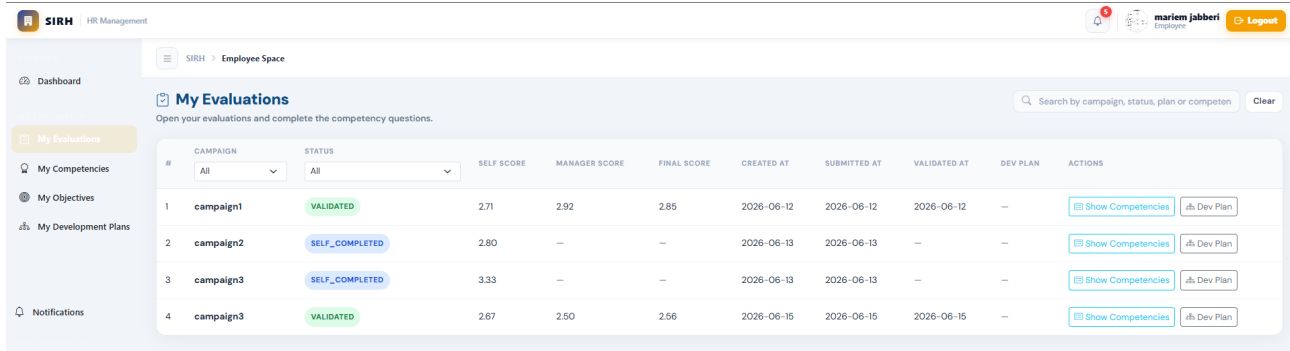
#	QUESTION	EMPLOYEE SCORE	MANAGER SCORE	FINAL SCORE
1	hhg	2	2	2.67
2	khk	2	2	2
3	ksh	3	2	2.33

Figure 4.17: Competency questions and scores interface

4.1.3.6 Employee Self-Evaluation Module

The employee self-evaluation module allows employees to consult their assigned evaluations, open competency questions, submit self-scores, and add comments.

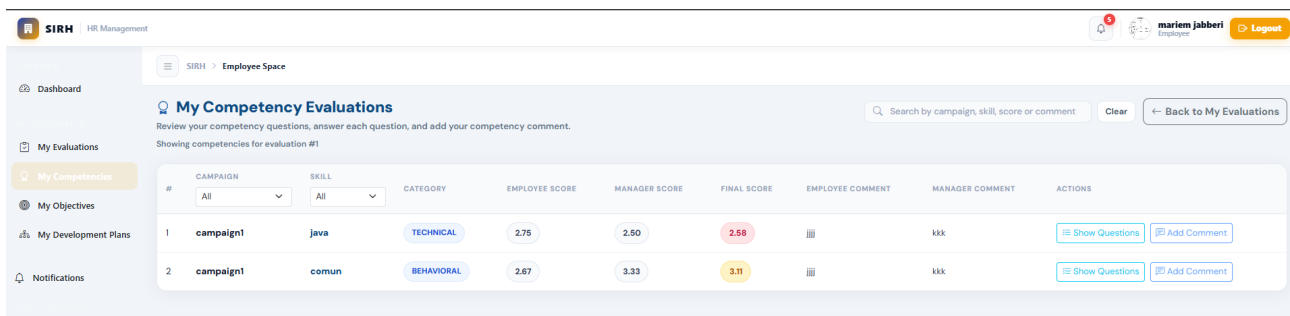
Figure 4.18 presents the employee evaluations interface.



#	CAMPAIGN	STATUS	SELF SCORE	MANAGER SCORE	FINAL SCORE	CREATED AT	SUBMITTED AT	VALIDATED AT	DEV PLAN	ACTIONS
1	campaign1	VALIDATED	2.71	2.92	2.85	2026-06-12	2026-06-12	2026-06-12	—	Show Competencies Dev Plan
2	campaign2	SELF_COMPLETED	2.80	—	—	2026-06-13	2026-06-13	—	—	Show Competencies Dev Plan
3	campaign3	SELF_COMPLETED	3.33	—	—	2026-06-13	2026-06-13	—	—	Show Competencies Dev Plan
4	campaign3	VALIDATED	2.67	2.50	2.58	2026-06-15	2026-06-15	2026-06-15	—	Show Competencies Dev Plan

Figure 4.18: Employee evaluations interface

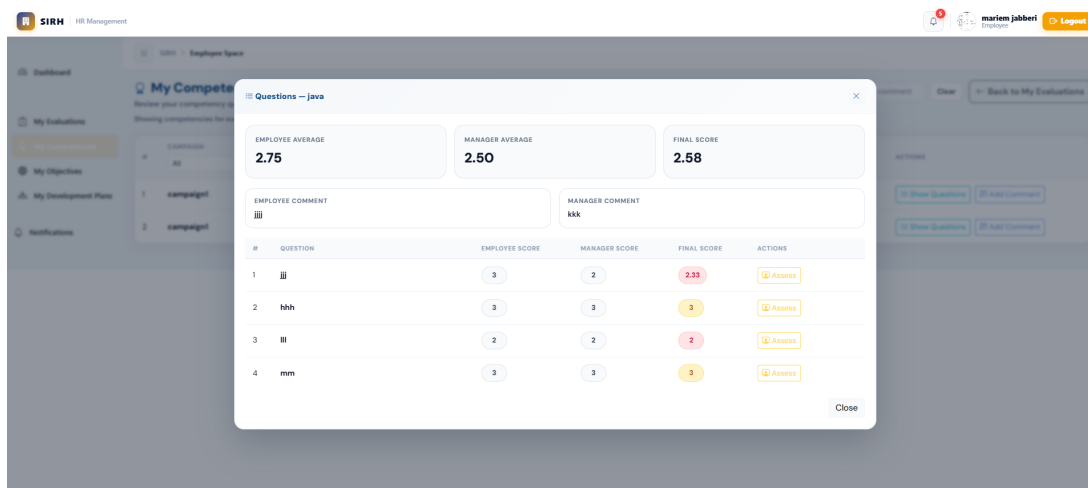
Figure 4.19 presents the employee competency evaluations interface.



#	CAMPAIGN	SKILL	CATEGORY	EMPLOYEE SCORE	MANAGER SCORE	FINAL SCORE	EMPLOYEE COMMENT	MANAGER COMMENT	ACTIONS
1	campaign1	java	TECHNICAL	2.75	2.50	2.58		kkk	Show Questions Add Comment
2	campaign1	comun	BEHAVIORAL	2.67	3.33	3.11		kkk	Show Questions Add Comment

Figure 4.19: Employee competency evaluations interface

Figure 4.20 illustrates the employee competency questions interface.



EMPLOYEE AVERAGE		MANAGER AVERAGE		FINAL SCORE	
2.75		2.50		2.58	
EMPLOYEE COMMENT		MANAGER COMMENT			
		kkk			
#	QUESTION	EMPLOYEE SCORE	MANAGER SCORE	FINAL SCORE	ACTIONS
1	jjj	3	2	2.33	Assess
2	hhh	3	3	3	Assess
3	lll	2	2	2	Assess
4	mmm	3	3	3	Assess

Figure 4.20: Employee competency questions interface

Figure 4.21 presents the employee question assessment interface.

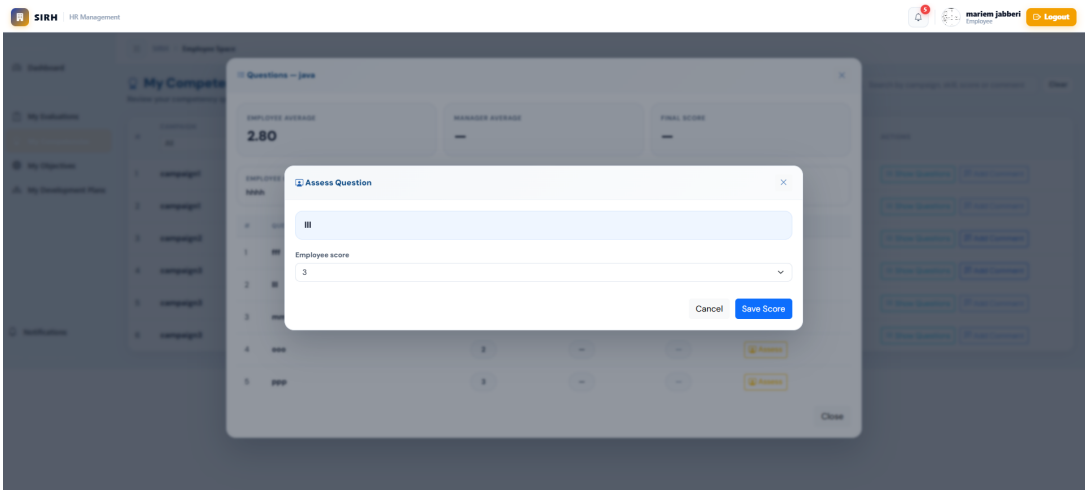


Figure 4.21: Employee question assessment interface

4.1.3.7 Manager Evaluation Module

The manager evaluation module allows managers to review team evaluations, consult competency questions, assess employees, and add managerial comments and scores.

Figure 4.22 presents the manager team evaluations interface.

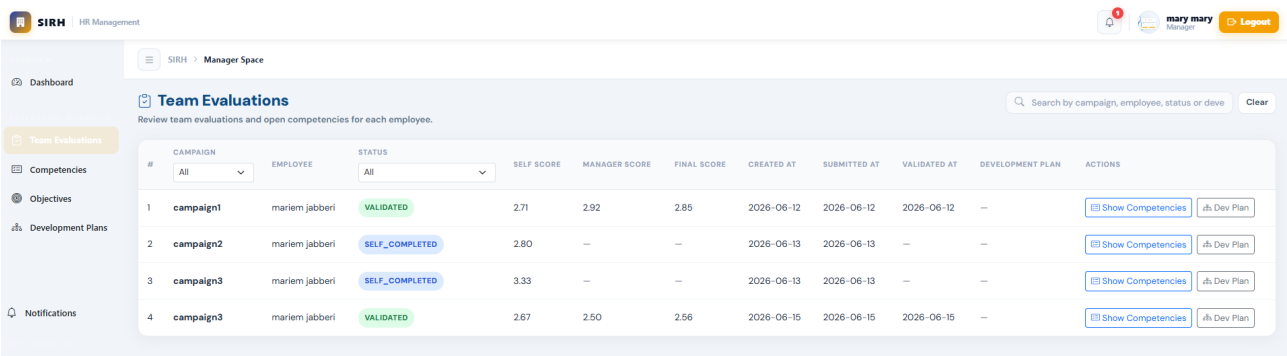


Figure 4.22: Manager team evaluations interface

Figure 4.23 presents the manager competency evaluations interface.

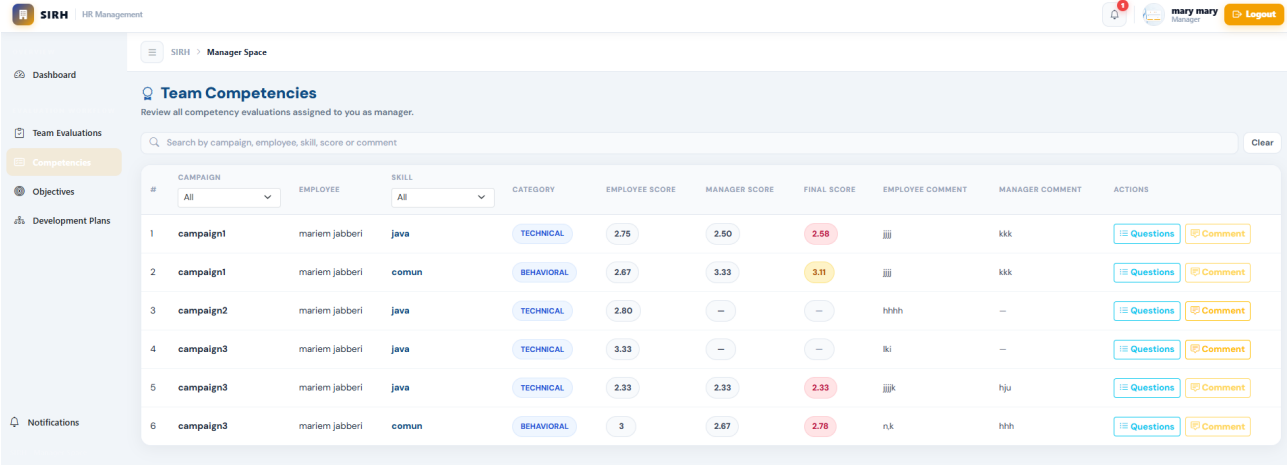


Figure 4.23: Manager competency evaluations interface

Figure 4.24 illustrates the manager evaluation competencies interface.



Figure 4.24: Manager evaluation competencies interface

Figure 4.25 presents the manager competency questions interface.

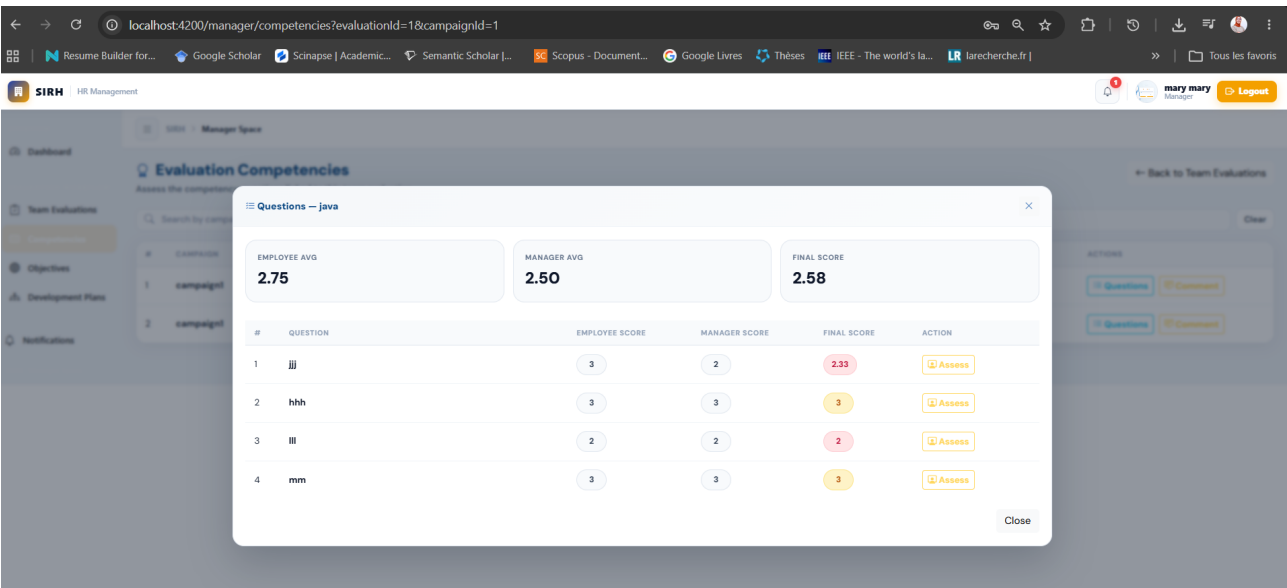


Figure 4.25: Manager competency questions interface

Figure 4.26 illustrates the manager question assessment interface.

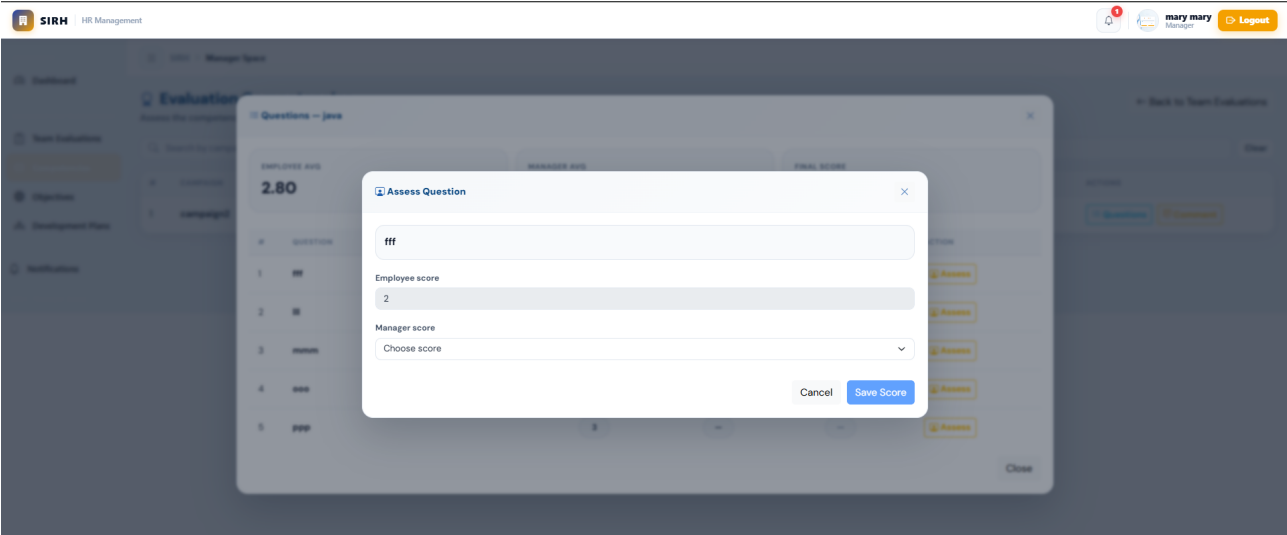


Figure 4.26: Manager question assessment interface

4.1.3.8 Objectives Management Module

The objectives management module allows managers to create weighted objectives for employees. Each objective contains a title, target date, tasks, and task weights. Employees can follow progress through a checklist and submit self-comments.

Figure 4.27 presents the manager objectives interface.

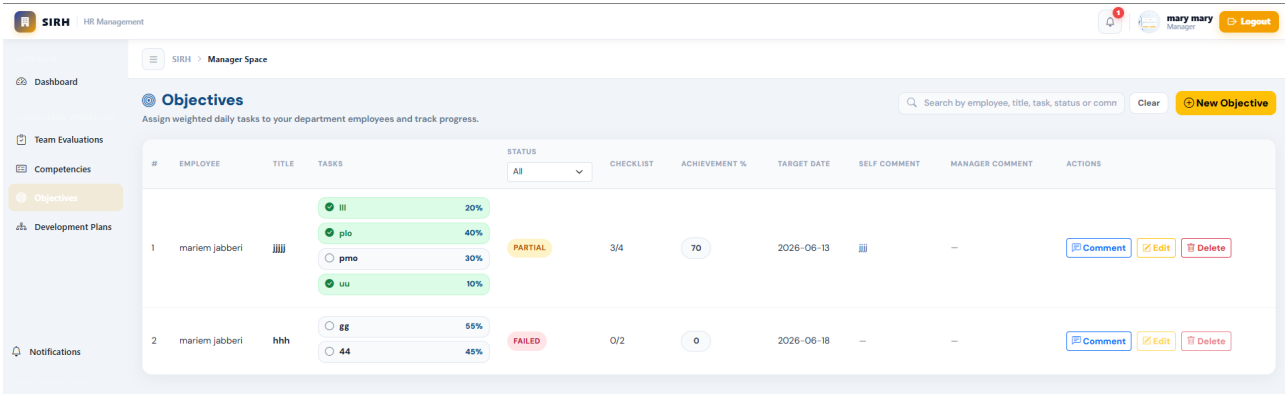


Figure 4.27: Manager objectives interface

Figure 4.28 illustrates the objective creation interface.

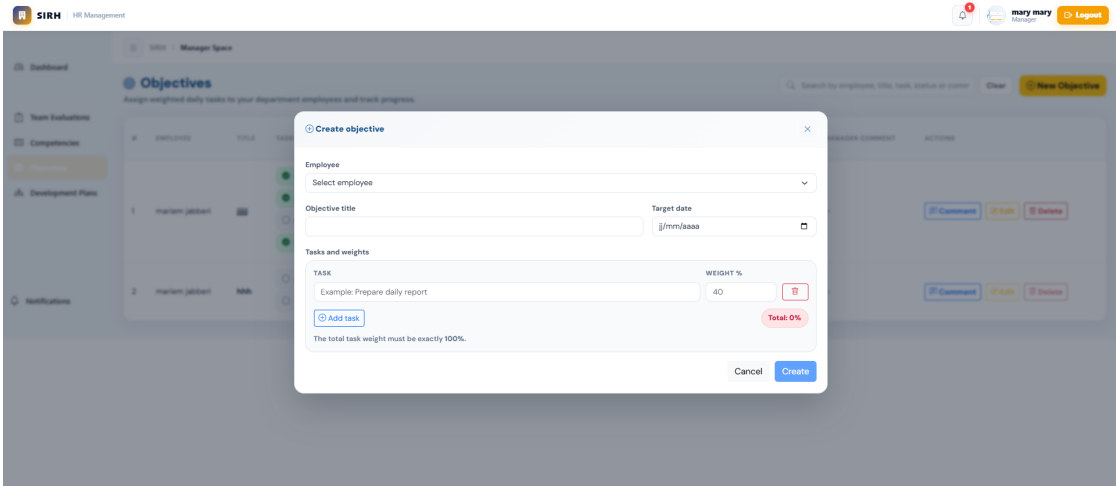


Figure 4.28: Objective creation interface

Figure 4.29 presents the objective update interface.

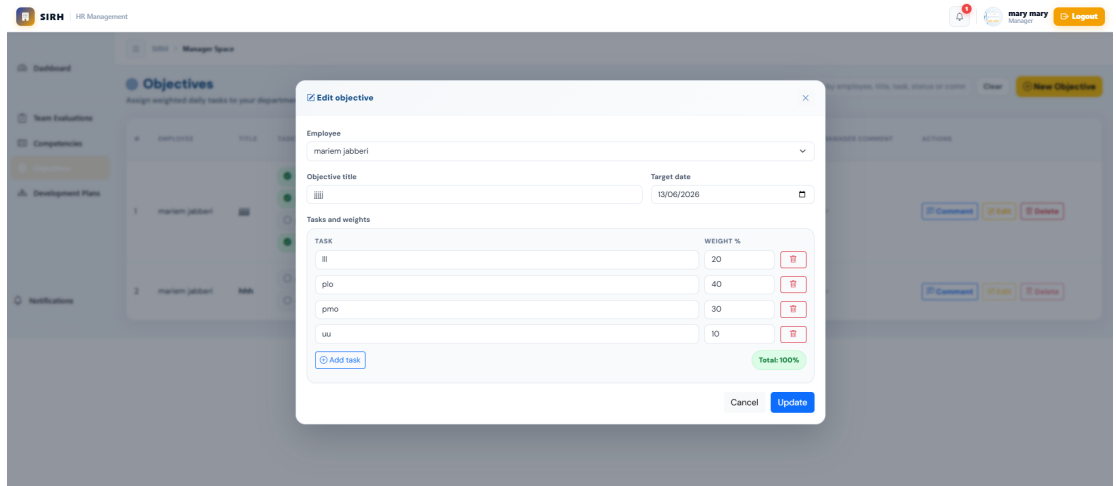


Figure 4.29: Objective update interface

Figure 4.30 presents the HR objectives follow-up interface.

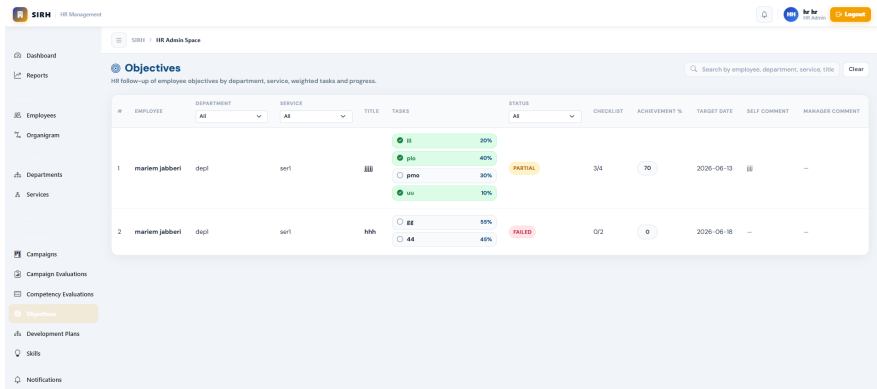


Figure 4.30: HR objectives follow-up interface

4.1.3.9 Employee Objectives Tracking

Employees can view their assigned objectives, update task checklist progress, and add self-comments. The achievement rate is calculated according to the weight of completed tasks.

Figure 4.31 presents the employee objectives interface.

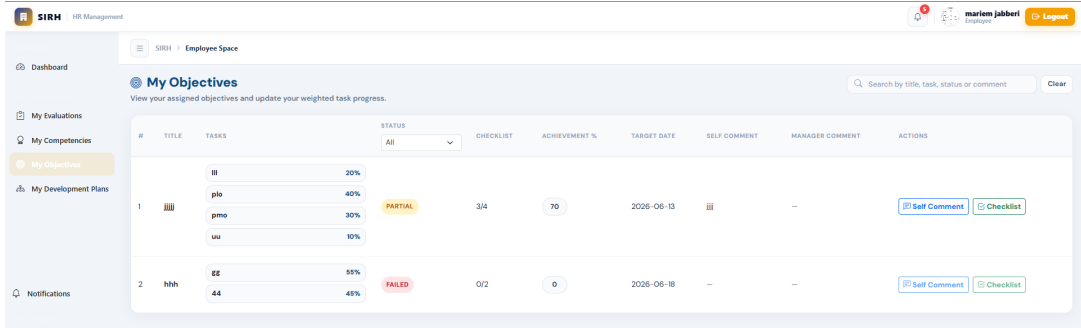


Figure 4.31: Employee objectives interface

Figure 4.32 illustrates the objective checklist update interface.

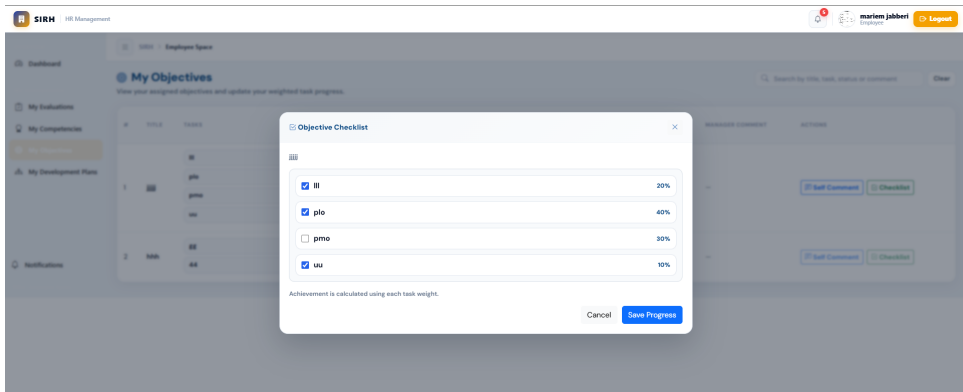


Figure 4.32: Objective checklist update interface

4.2 Sprint 4: Development and Decision Support

4.2.1 Sprint 4 Backlog

The Sprint 4 backlog presented in Table 4.3 groups the user stories related to AI training recommendations, recommendation review, training catalogue management, development plan assignment, progress tracking, dashboards, and reports.

Table 4.3: Sprint 4 backlog

ID	User Story	Priority	Estimate
US-01	As an HR Administrator, I want to generate AI training recommendations from validated evaluations so that competency gaps can be addressed efficiently.	High	2 days
US-02	As an HR Administrator, I want to review, approve, or reject AI recommendations so that only relevant recommendations are proposed to managers.	High	2 days
US-03	As an HR Administrator, I want to consult the training catalogue so that I can verify the learning resources used by the AI module.	Medium	1 day
US-04	As a system, I want to import and update open training courses so that the recommendation catalogue remains enriched.	Medium	2 days
US-05	As a Manager, I want to view HR-approved recommendations so that I can select appropriate trainings for an employee.	High	2 days
US-06	As a Manager, I want to assign selected approved trainings as a development plan so that the employee has a concrete improvement path.	High	1 day
US-07	As an Employee, I want to consult my assigned development plans so that I can follow my improvement actions.	High	1 day
US-08	As an Employee, I want to update my development plan progress so that my manager and HR can follow my advancement.	Medium	1 day
US-09	As an HR Administrator, Manager, or Employee, I want dashboards so that I can monitor relevant HR and evaluation indicators.	High	2 days
US-10	As an HR Administrator, I want reports so that I can analyze evaluation results, competency gaps, and development plan progress.	Medium	1 day
Total			15 days

4.2.2 Sprint 4 Analysis and Design

This section presents the analysis and design of Sprint 4. The objective of this sprint is to transform validated evaluations into decision-support outputs through AI-assisted training recommendations, development plans, dashboards, and reports.

4.2.2.1 Use-Case Model of Sprint 4

The use-case model of Sprint 4 focuses on the development and decision-support workflow. The main actors are the HR Administrator, the Manager, the Employee, and the AI Engine.

Figure 4.33 illustrates the use-case diagram of Sprint 4 and summarizes the interactions between the actors, the development plan module, the training catalogue, and the AI Engine.

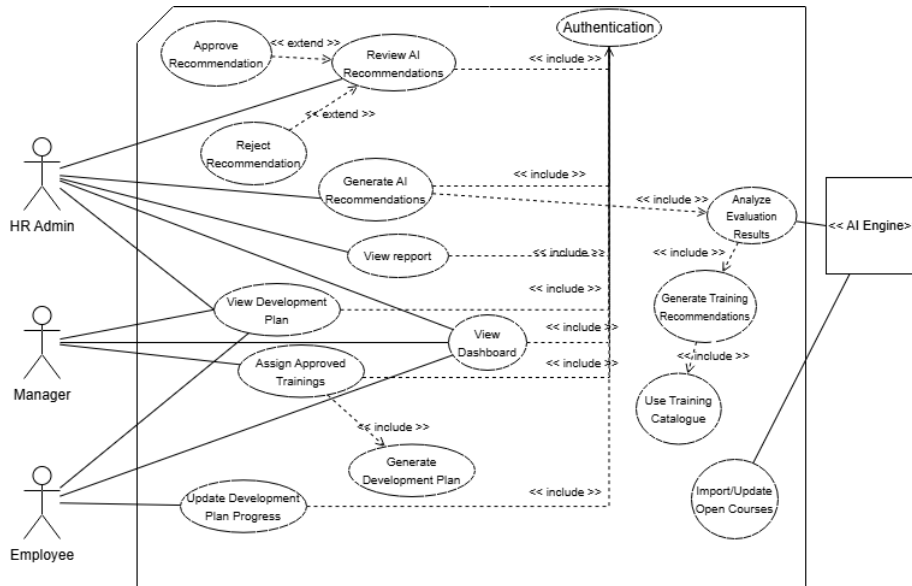


Figure 4.33: Use-case diagram of Sprint 4

The HR Administrator generates AI recommendations, reviews the proposed trainings, approves or rejects them, consults dashboards, reports, training catalogue, and development plans. The Manager consults HR-approved recommendations and assigns selected trainings as a development plan. The Employee consults assigned development plans and updates progress. The AI Engine analyzes evaluation results, detects competency gaps, and generates ranked training recommendations based on the training catalogue.

4.2.2.2 Development Plan and AI Recommendation Design

Sprint 4 extends the Evaluation Service with development plan and AI recommendation functionalities. The development plan represents the final operational result of the evaluation process. It contains

a title, actions, training links, start and end dates, progress percentage, checklist items, and status.

The AI recommendation module supports HR and managers by analyzing validated evaluations and proposing relevant trainings. The recommendation process is based on:

- Competency gaps detected from employee and manager scores.
- Objective gaps based on achievement rates.
- Training catalogue content.
- Completed training history.
- Similar training choices from comparable employee profiles.

The recommendation lifecycle is organized using several statuses. Table 4.4 describes each status used during the recommendation workflow.

Table 4.4: AI recommendation status lifecycle

Status	Description
AI_PROPOSED	The recommendation has been generated by the AI Engine and is waiting for HR review.
HR_APPROVED	The recommendation has been approved by the HR Administrator and can be assigned by the Manager.
HR_REJECTED	The recommendation has been rejected by the HR Administrator.
MANAGER_ASSIGNED	The recommendation has been selected by the Manager and converted into a development plan.

4.2.2.3 AI Recommendation Workflow

The AI recommendation workflow starts when the HR Administrator opens a validated evaluation and clicks on the generation button. The request is sent through the API Gateway with a JWT token. After authentication and service discovery, the Evaluation Service retrieves the evaluation details, skill gaps, objective gaps, employee context, and training catalogue. It then calls the FastAPI AI Engine.

The AI Engine analyzes the evaluation results, computes relevance scores, ranks the training catalogue, and returns the top recommendations. The Evaluation Service saves these recommendations in the database with the status `AI_PROPOSED`.

Figure 4.34 presents the sequence diagram of AI recommendation generation.

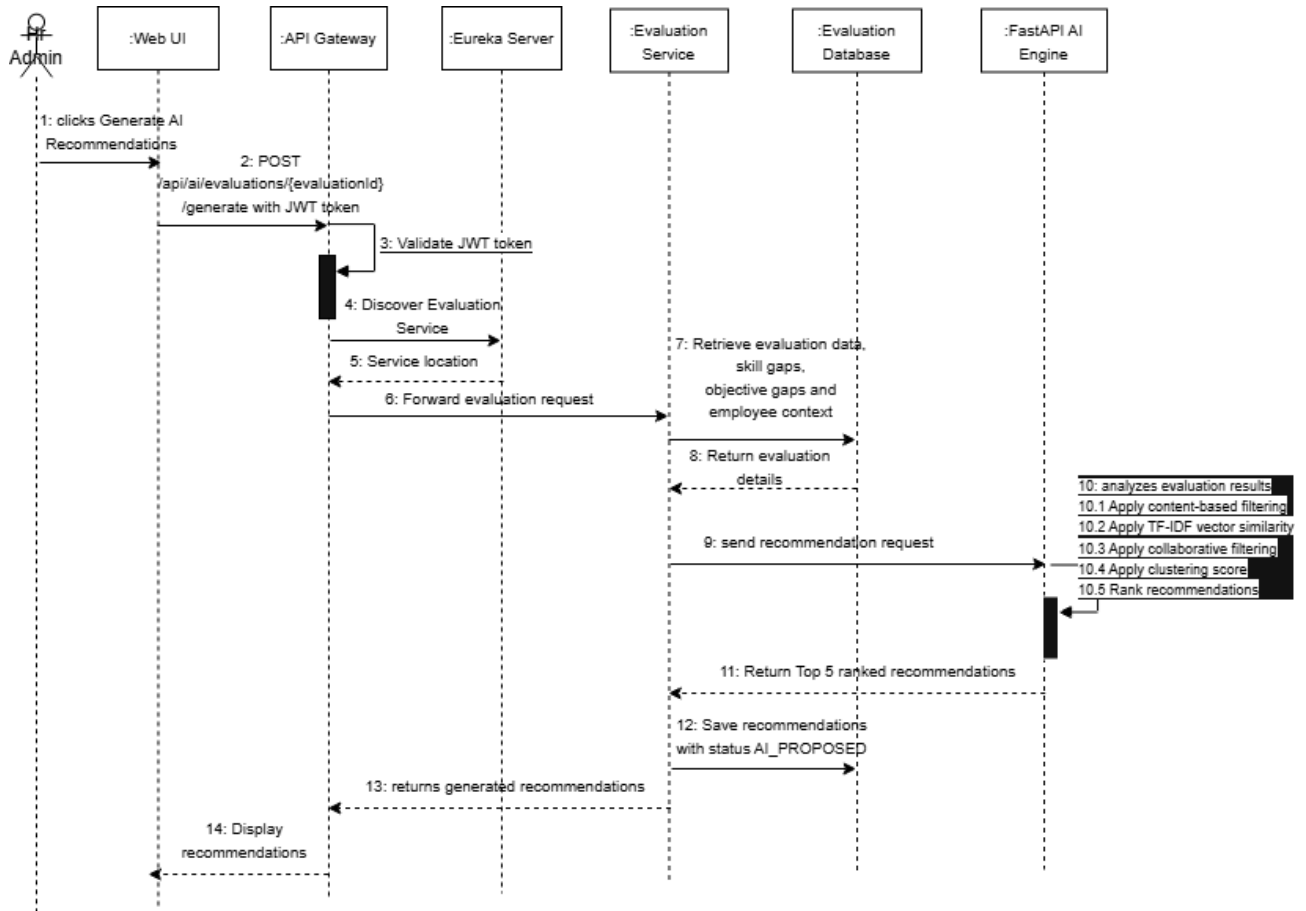


Figure 4.34: Sequence diagram of AI recommendation generation

4.2.2.4 Assign Approved Trainings Workflow

The assignment workflow starts when the Manager opens a validated evaluation and consults HR-approved AI recommendations. The Manager selects one to three approved trainings, optionally defines start and end dates, and submits the assignment.

The Evaluation Service validates the request and the manager authorization, retrieves the selected approved recommendations, generates a development plan, saves it in the database, and updates the selected recommendations to **MANAGER_ASSIGNED**.

Figure 4.35 presents the sequence diagram of assigning approved trainings.

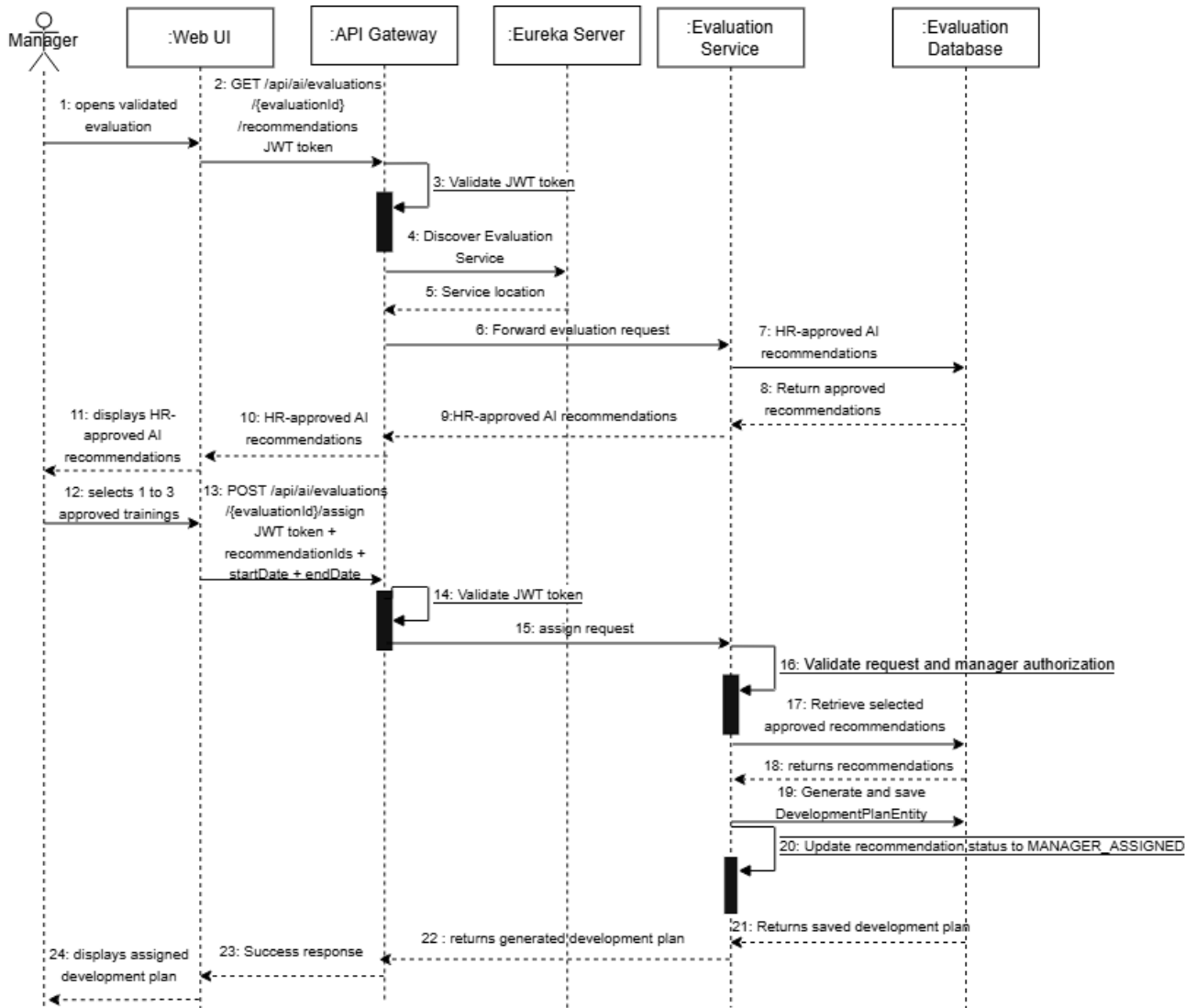


Figure 4.35: Sequence diagram of assigning approved trainings

4.2.2.5 AI Scoring Logic

The AI Engine combines several recommendation techniques to calculate a final relevance score for each training. [W27]

- **Content-based filtering:** compares the training title, description, skill name, category, and level with detected skill gaps and objective gaps.
- **Vector similarity:** uses TF-IDF vectorization and cosine similarity to measure semantic proximity between employee needs and catalogue trainings.
- **Collaborative filtering:** increases the score of trainings already followed by similar employee profiles.
- **Clustering score:** prepares the recommendation engine for future employee profile grouping using K-Means.

This score allows the system to rank the catalogue and return the most relevant training recommendations.

4.2.2.6 Sprint 4 Architecture

Sprint 4 reuses the existing microservices architecture and extends the Evaluation Service with an AI Engine based on FastAPI. The Angular frontend communicates with the backend through the API Gateway. The gateway validates JWT tokens issued by Keycloak and routes requests to the Evaluation Service through Eureka discovery.

The Evaluation Service communicates with the FastAPI AI Engine through a WebClient. The AI Engine processes recommendation requests and also exposes endpoints for importing open courses into the training catalogue.

Figure 4.36 illustrates the architecture implemented during Sprint 4.

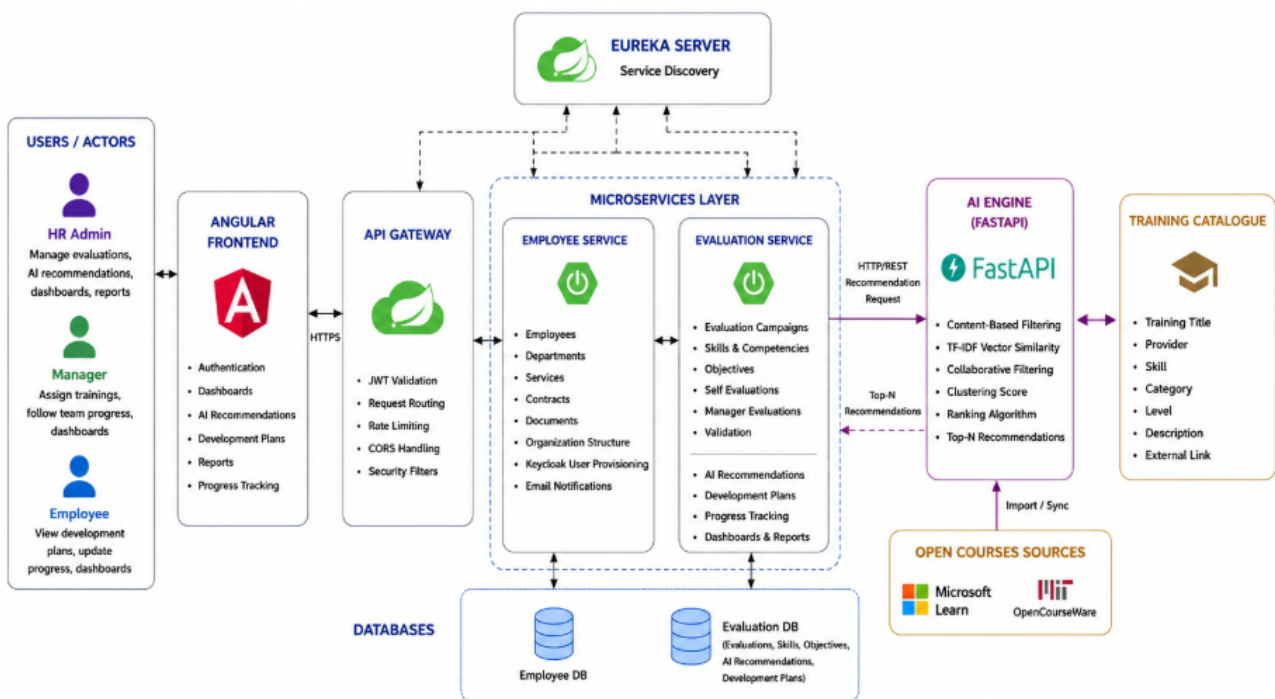


Figure 4.36: Sprint 4 architecture

The main components of Sprint 4 are:

- Angular Web UI.
- API Gateway.
- Eureka Server.
- Keycloak authentication server.
- Evaluation Service.

- Evaluation Database.
- FastAPI AI Engine.
- Training catalogue.
- Dashboards and reports module.

4.2.3 Realization of Sprint 4

This section presents the practical implementation of Sprint 4 functionalities using the Angular frontend, the Evaluation Service, and the FastAPI AI Engine.

4.2.3.1 Training Catalogue Module

The training catalogue contains the learning resources used by the AI recommendation module. Each training contains a title, provider, skill, category, level, description, and external link.

The catalogue is enriched using open learning sources and can be updated periodically. Duplicate trainings are ignored to keep the catalogue clean.

Figure 4.37 presents the training catalogue interface.

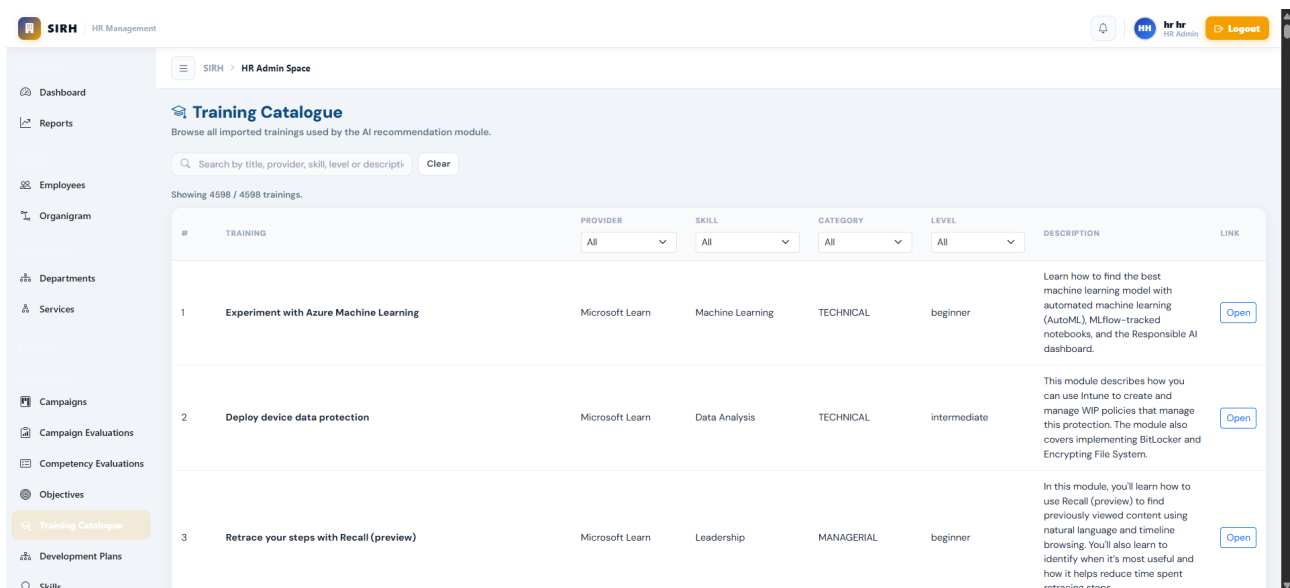


Figure 4.37: Training catalogue interface

4.2.3.2 HR Admin AI Recommendations Module

The HR Administrator can generate AI recommendations from a validated evaluation. The system displays the top five recommended trainings with their relevance score, detected gap, provider, reason, and external training link.

The HR Administrator can approve or reject each recommendation. Only approved recommendations become available to the Manager for assignment.

Figure 4.38 presents the AI recommendations interface.

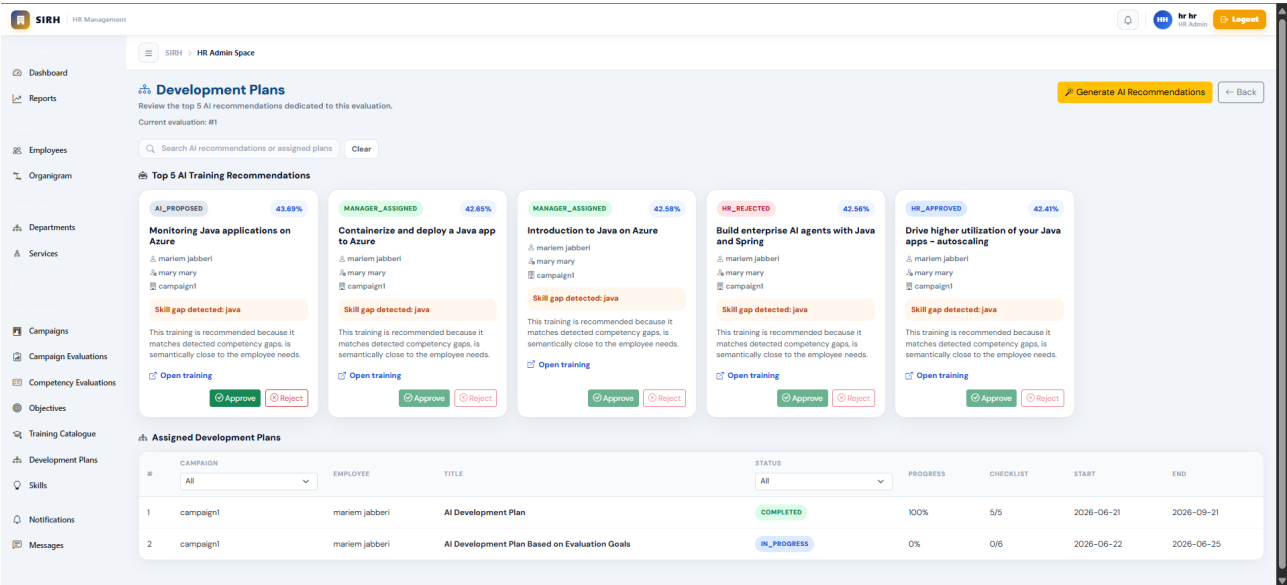


Figure 4.38: Top AI training recommendations for a validated evaluation

4.2.3.3 Manager Development Plan Assignment

The Manager can access the approved recommendations of a selected evaluation. The interface displays HR-approved AI recommendations as cards. The Manager selects one to three trainings and assigns them as a development plan.

Figure 4.39 illustrates the manager assignment interface.

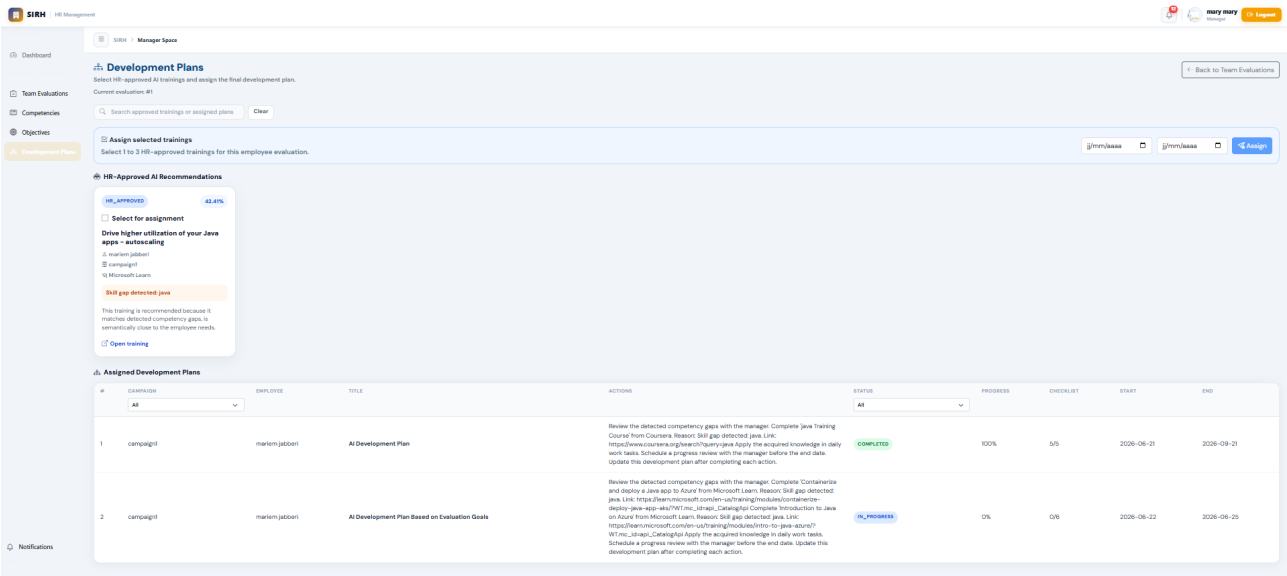


Figure 4.39: Manager assignment of approved AI trainings

After assignment, the generated development plan appears in the list of assigned development

plans. It contains the campaign, employee, title, actions, status, progress, checklist, start date, and end date.

Figure 4.40 presents the manager development plans interface.

#	Campaign	EMPLOYEE	TITLE	ACTIONS	STATUS	PROGRESS	CHECKLIST	START	END
1	campaign1	marion jobert	AI Development Plan	Review the detected competency gaps with the manager. Complete Java Training Course from Coursera. Reason: Skill gap detected java Link: https://www.coursera.org/search?query=java Apply the acquired knowledge in daily work tasks. Schedule a progress review with the manager before the end date. Update this development plan after completing each action.	COMPLETED	100%	5/5	2028-08-21	2028-09-21
2	campaign1	marion jobert	AI Development Plan Based on Evaluation Goals	Review the detected competency gaps with the manager. Complete 'Containerize and deploy a Java app to Azure' from Microsoft Learn. Reason: Skill gap detected java Link: https://learn.microsoft.com/en-us/training/modules/containerize-deploy-java-app-aks/?WT.mc_id=catagip1 Complete 'Introduction to Java on Azure' from Microsoft Learn. Reason: Skill gap detected java Link: https://learn.microsoft.com/en-us/training/modules/intro-to-java-azure/?WT.mc_id=catagip1 Apply the acquired knowledge in daily work tasks. Schedule a progress review with the manager before the end date. Update this development plan after completing each action.	IN PROGRESS	0%	0/5	2028-08-22	2028-08-29
3	campaign3	marion jobert	AI Development Plan Based on Evaluation Goals	Review the detected competency gaps with the manager. Complete 'MFA: Introduction to Programming Using Java' from Microsoft Learn. Reason: Skill gap detected java Link: https://learn.microsoft.com/en-us/training/modules/introduction-to-programming-using-java/?WT.mc_id=catagip1 Complete 'Containerize and deploy a Java app to Azure' from Microsoft Learn. Reason: Skill gap detected java Link: https://learn.microsoft.com/en-us/training/modules/containerize-deploy-java-app-aks/?WT.mc_id=catagip1 Apply the acquired knowledge in daily work tasks. Schedule a progress review with the manager before the end date. Update this development plan after completing each action.	IN PROGRESS	0%	0/5	2028-08-22	2028-08-24

Figure 4.40: Manager development plans interface

4.2.3.4 Employee Development Plan Tracking

The Employee can consult assigned development plans from the Employee Space. Each plan displays the title, related evaluation, campaign, progress, checklist status, start date, end date, actions, and training links.

Figure 4.41 presents the employee development plans interface.

AI Development Plan	Progress	Checklist	Start	End
Evaluation #1 - campaign1	100%	5/5	2028-08-21	2028-09-21
AI Development Plan Based on Evaluation Goals Evaluation #2 - campaign1	0%	0/5	2028-08-22	2028-08-29
AI Development Plan Based on Evaluation Goals Evaluation #3 - campaign3	0%	0/5	2028-08-22	2028-08-24

Figure 4.41: Employee development plans interface

The Employee can update progress using a checklist. Once actions are selected and saved, the progress percentage is automatically recalculated. When all actions are completed, the plan

status becomes COMPLETED.

Figure 4.42 illustrates the employee development plan progress update.

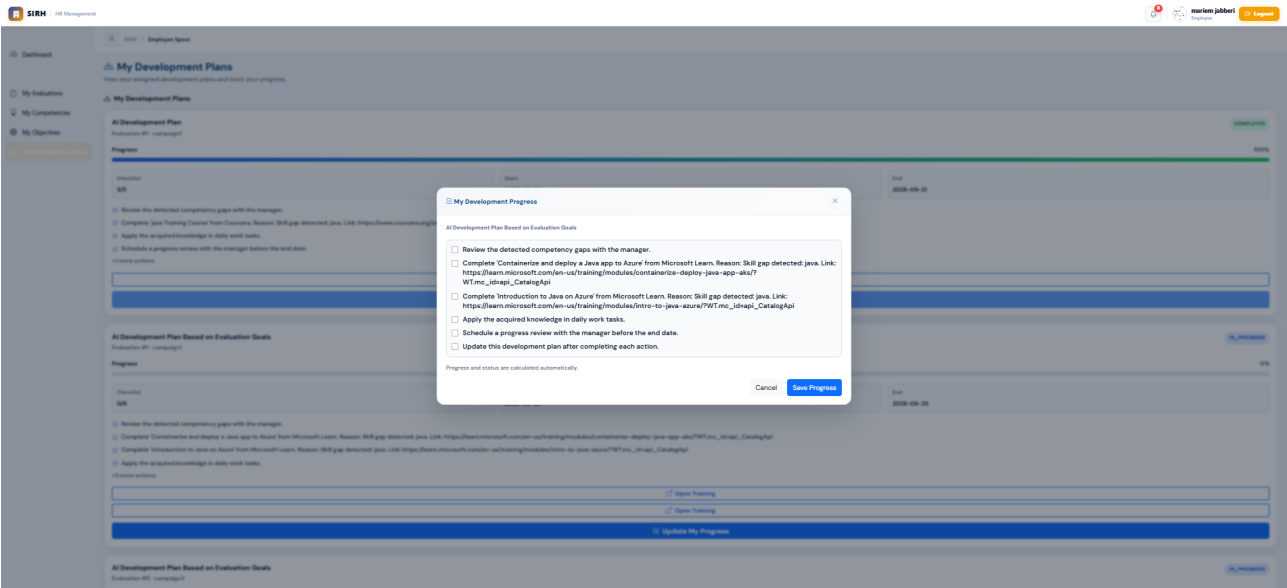


Figure 4.42: Employee development plan progress update

4.2.3.5 HR Admin Development Plan Monitoring

The HR Administrator can consult all assigned development plans. This interface allows HR to monitor the campaign, employee, plan title, status, progress, checklist, start date, and end date.

Figure 4.43 presents the HR Admin development plan monitoring interface.

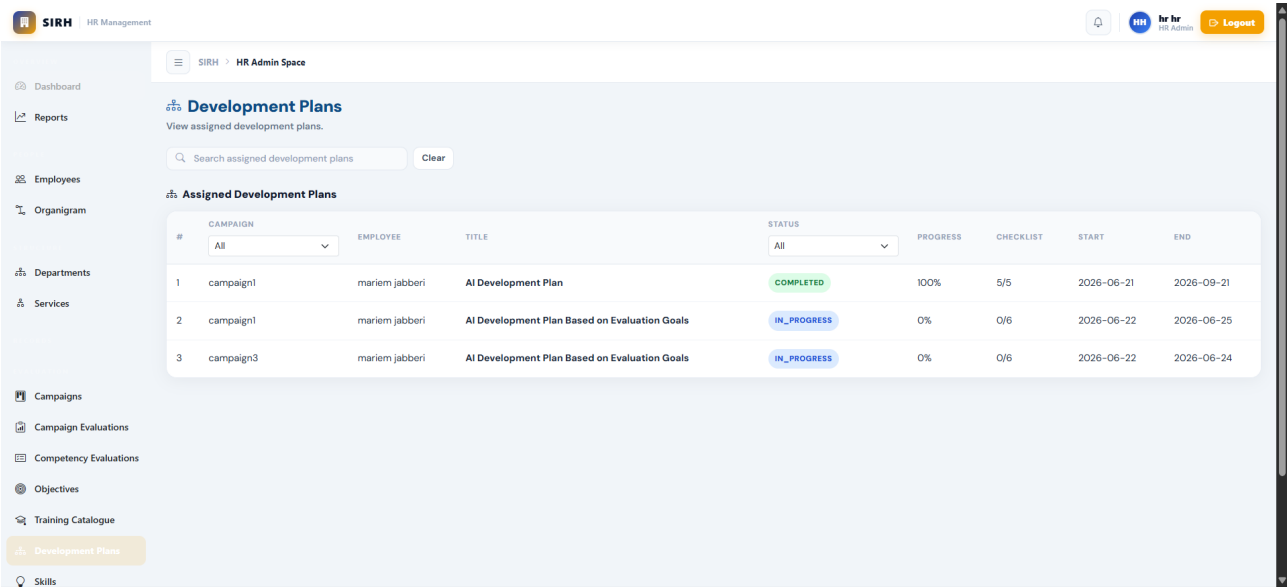


Figure 4.43: HR Admin development plan monitoring

4.2.3.6 Dashboards and Reports

Dashboards and reports support decision-making after the evaluation and development plan phases. They provide visual indicators for HR Administrators, Managers, and Employees according to their roles.

4.2.3.7 HR Admin Dashboard

The HR Admin dashboard provides a centralized view of the HR platform and offers real-time monitoring of workforce information, evaluation activities, and development planning processes. It enables HR administrators to supervise the overall state of the system through a collection of indicators, recent activities, and quick access panels.

The dashboard consolidates several categories of information:

- Workforce statistics, including the number of employees, departments, services, contracts, campaigns, evaluations, and skills.
- Recent employees and their current status.
- Workforce distribution according to roles and active users.
- Evaluation monitoring, showing validated, pending, and ongoing evaluations.
- Contract and business service overviews.
- Recent evaluation results and objective achievement indicators.
- Development plans and their completion progress.
- Department information and management shortcuts.
- Quick actions providing direct access to the main HR modules.

This dashboard allows HR administrators to monitor organizational data and quickly access the modules required to manage employees, evaluations, objectives, competencies, training catalogues, and development plans.

The two figures below present the HR Admin dashboard. the Figure 4.44 shows the upper part of the dashboard, including the main indicators, recent employees, workforce breakdown, contracts, and services.

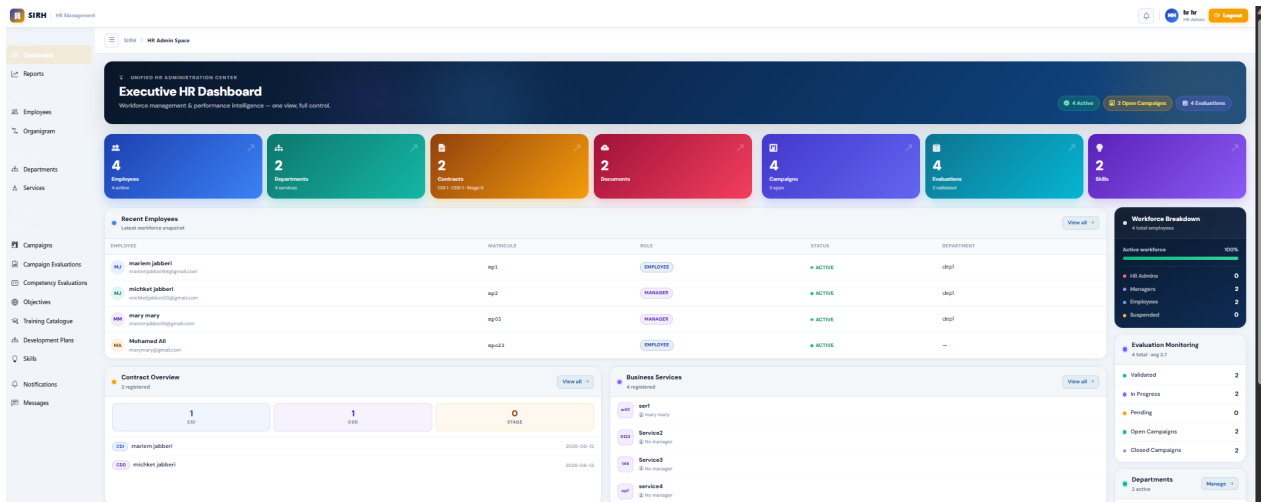


Figure 4.44: HR Admin Dashboard (A)

the Figure 4.45 shows the lower part, including recent evaluations, objectives, development plans, departments, and quick actions.

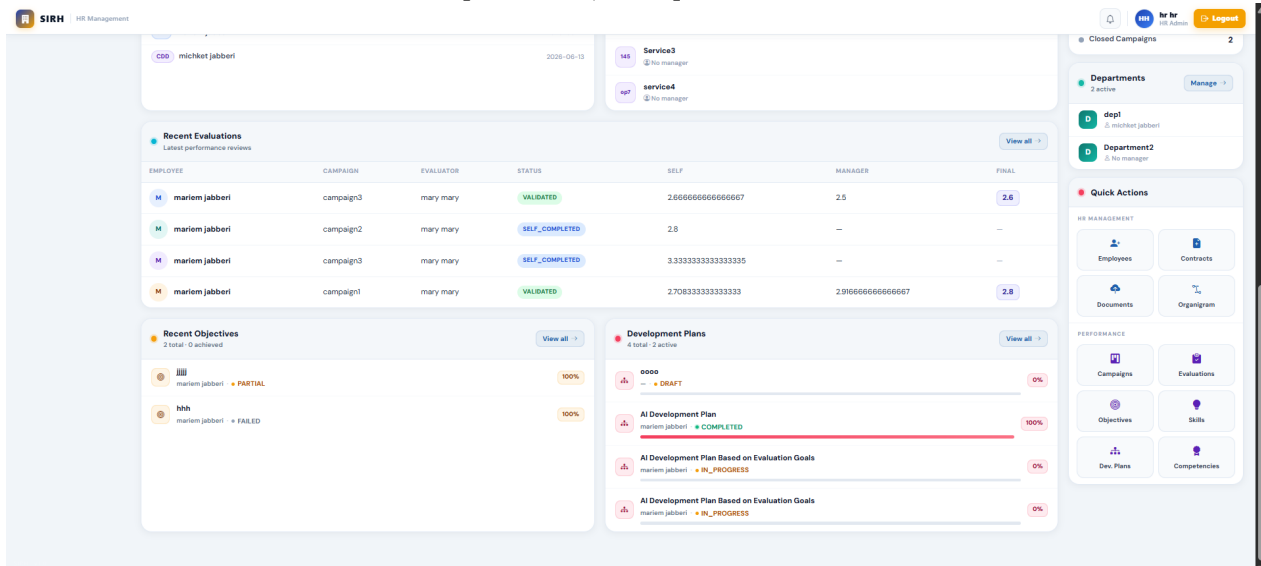


Figure 4.45: HR Admin Dashboard (B)

4.2.3.8 Manager Dashboard

The Manager dashboard focuses on team follow-up. It displays indicators related to team evaluations, competency assessments, objectives, assigned development plans, and employee progress.

The main indicators are:

- Number of team members.
- Number of team evaluations.
- Number of evaluations waiting for manager assessment.
- Number of evaluations in progress.

- Number of validated evaluations.
- Number of assigned objectives.
- Number of assigned development plans.
- Overview of recent team evaluations.

Figure 4.46 presents the Manager dashboard and its main team-monitoring indicators.

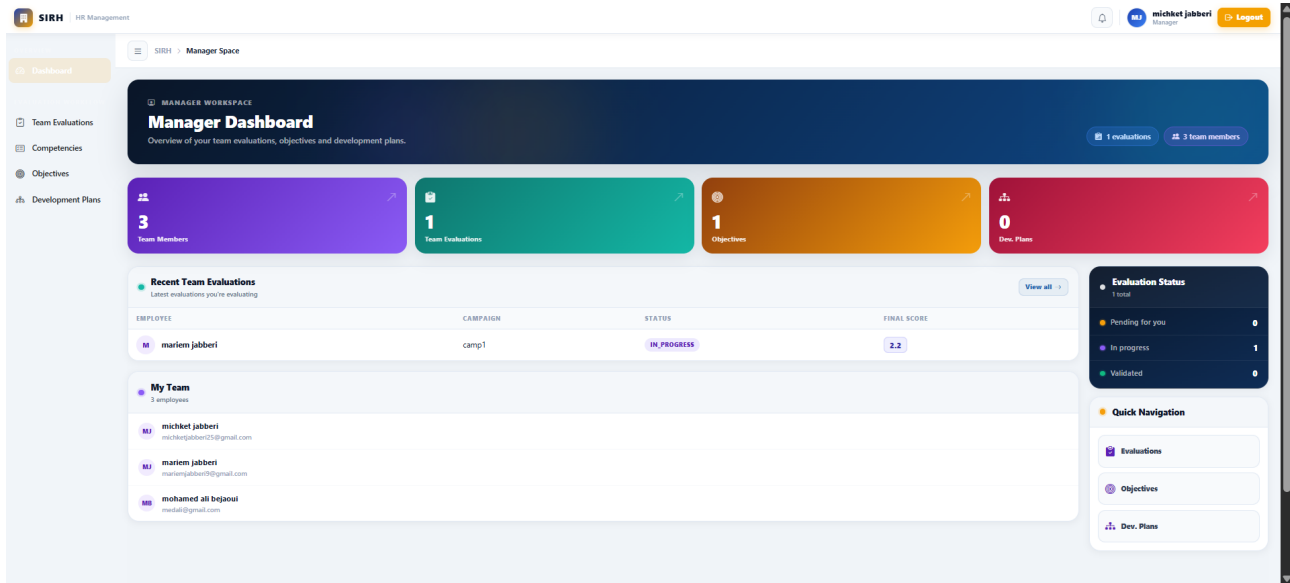


Figure 4.46: Manager Dashboard

4.2.3.9 Employee Dashboard

The Employee dashboard provides a personalized overview of the employee's evaluation progress, objectives, and development activities. It enables employees to monitor their current evaluations, follow the achievement of their objectives, review assigned development plans, and track their overall performance through a set of key indicators.

The main indicators are:

- Number of personal evaluations.
- Current evaluation status.
- Self-evaluation score.
- Final evaluation score.
- Number of assigned objectives.
- Objective achievement rate.
- Number of assigned development plans.

- Performance summary and quick access to pending actions.

Figure 4.47 illustrates the Employee dashboard and the main indicators available to employees for monitoring their evaluation and professional development activities.

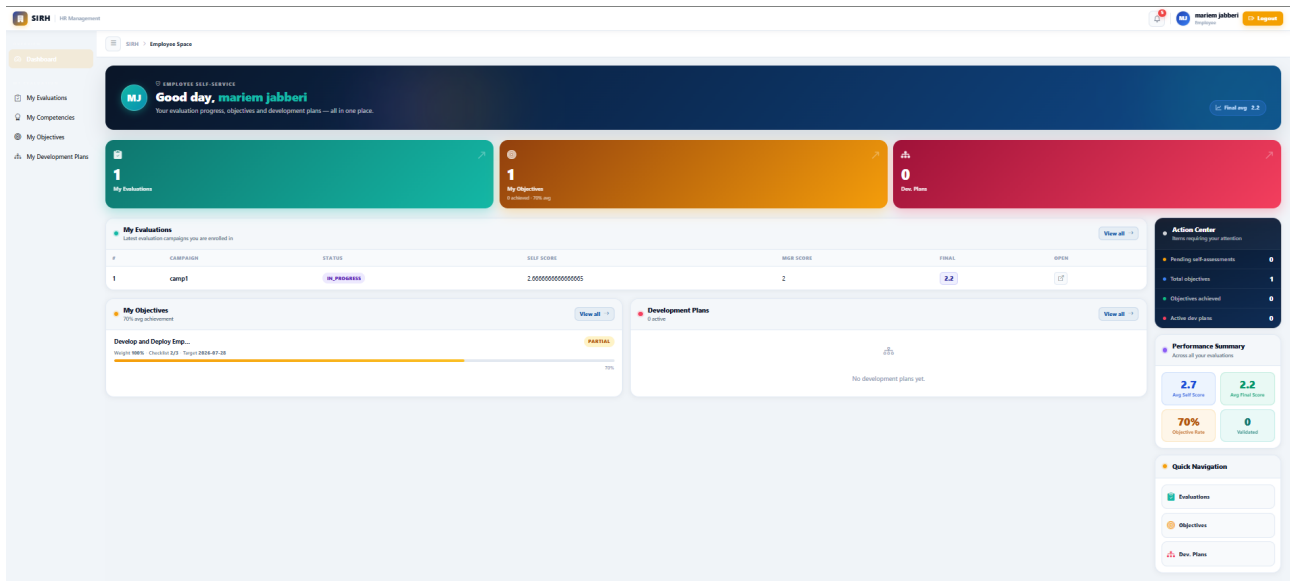


Figure 4.47: Employee Dashboard

4.2.3.10 Reports Module

The Reports module provides the HR Administrator with consolidated analytical views of the organization's Human Resources data. It transforms operational information into visual dashboards that support decision-making by presenting key performance indicators related to employees, organizational structure, evaluation campaigns, contracts, and development activities.

The reports allow HR administrators to answer questions such as:

- How many employees, departments, and services are currently active?
- How are employees distributed across organizational roles and departments?
- What is the distribution of employment contract types?
- What is the current status of evaluation campaigns and employee evaluations?
- What is the average final evaluation score across completed evaluations?
- How many objectives have been achieved and how many development plans are active?
- What is the overall organizational summary for HR decision making?

The reports are generated from data stored in the Employee Service and Evaluation Service databases and are presented through interactive Angular dashboards.

Figure 4.48 illustrates the first reporting dashboard, which provides Human Resources analytics related to employees, departments, roles, contracts, campaigns, and evaluations.

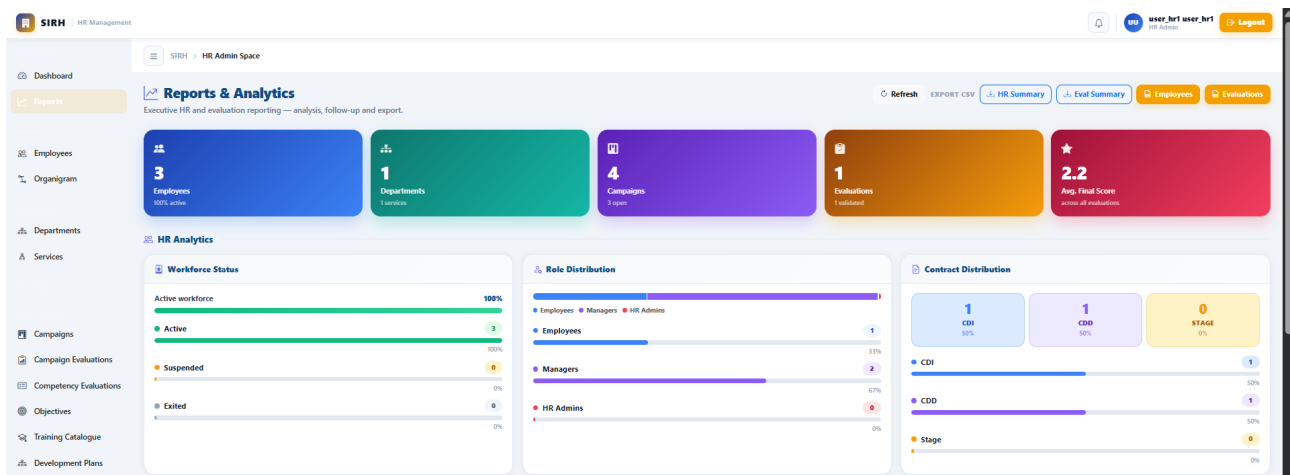


Figure 4.48: Reports Dashboard (A): Human Resources Analytics

Figure 4.49 presents the second reporting dashboard, which summarizes department distribution, document statistics, evaluation status, campaign progress, and executive performance indicators.

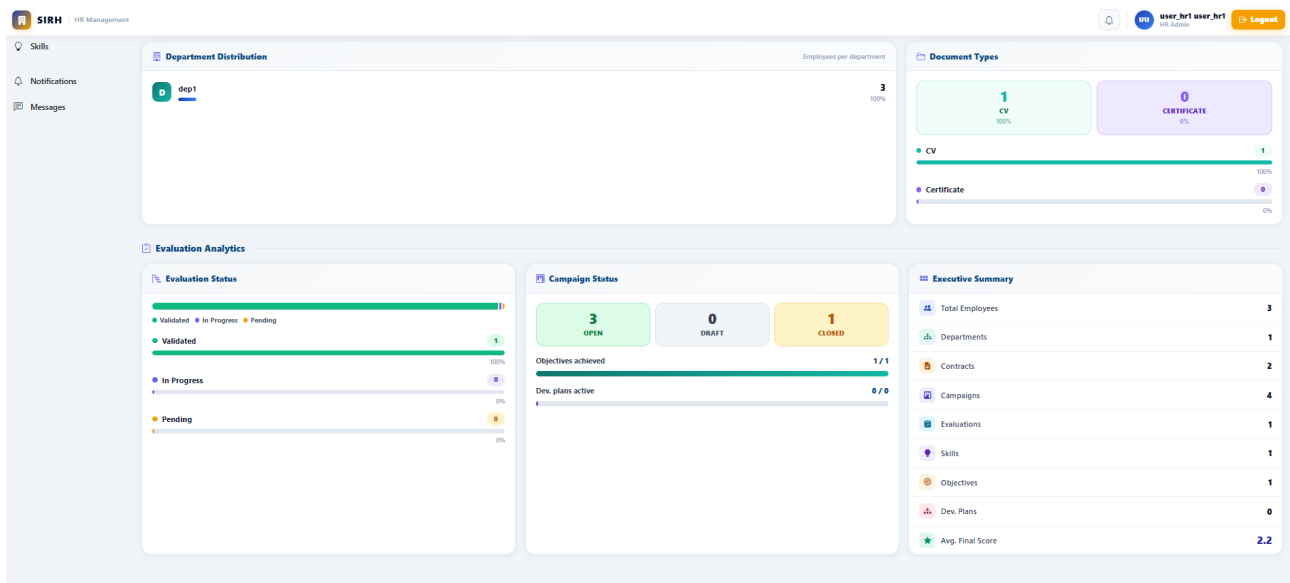


Figure 4.49: Reports Dashboard (B): Evaluation and Executive Analytics

Conclusion

This release completed the functional evaluation cycle of the SIRH platform. Sprint 3 implemented the evaluation management workflow, while Sprint 4 transformed validated evaluations into development actions through AI recommendations, development plans, dashboards, and reports. The next chapter will focus on Release 3, dedicated to DevSecOps and deployment, including Docker containerization, CI/CD pipelines, code quality analysis, security scanning, and deployment

automation.

RELEASE 3: DEVSECOPS AND DEPLOYMENT

Plan

Introduction	102
1 Sprint 5: Docker Containerization and Local Orchestration	102
2 Sprint 6: CI/CD Automation, Security and Observability	109
Conclusion	121

Introduction

Release 3 transforms the SIRH platform from a locally developed application into a reproducible, containerized, secure, observable, and deployment-ready solution. It is divided into two complementary sprints.

Sprint 5 focuses on containerization and local orchestration. It introduces Docker images for the Angular frontend, Spring Boot microservices, the FastAPI AI engine, Keycloak, and MySQL, together with a complete Docker Compose environment.

Sprint 6 builds on this foundation by automating testing, quality analysis, security scanning, Docker image publication, Kubernetes manifest validation, local Minikube deployment, metrics collection, dashboards, alert rules, and centralized logging.

Together, both sprints establish the DevSecOps lifecycle of the SIRH platform

5.1 Sprint 5: Docker Containerization and Local Orchestration

5.1.1 Sprint 5 Backlog

Sprint 5 establishes the containerized execution environment of the SIRH platform. The backlog is presented in Table 5.1.

Table 5.1: Sprint 5 backlog

ID	User Story	Priority	Estimate
US-01	As a developer, I want to containerize the Angular frontend so that the user interface runs in an isolated and reproducible environment.	High	2 days
US-02	As a developer, I want to containerize the Eureka Discovery Server and API Gateway so that service discovery and routing run inside containers.	High	2 days
US-03	As a developer, I want to containerize the Employee Service and Evaluation Service so that backend services can be deployed independently.	High	2 days
US-04	As a developer, I want to containerize the FastAPI AI Engine so that training recommendation logic remains independently deployable.	High	2 days

ID	User Story	Priority	Estimate
US-05	As a developer, I want to containerize MySQL and Keycloak so that persistence and identity management are included in the local environment.	High	2 days
US-06	As a developer, I want to orchestrate the complete platform with Docker Compose so that it can be started, stopped, and validated consistently.	High	2 days
US-07	As a developer, I want to configure health checks, persistent volumes, restart policies, and environment-based configuration so that the platform is robust and maintainable.	High	2 days
Total			14 days

5.1.2 Sprint 5 Analysis and Design

The objective of Sprint 5 is to package each major component of the SIRH platform as an independent container and to orchestrate the complete environment through Docker Compose.

5.1.2.1 Containerized Architecture

Figure 5.1 presents the Docker-based architecture.

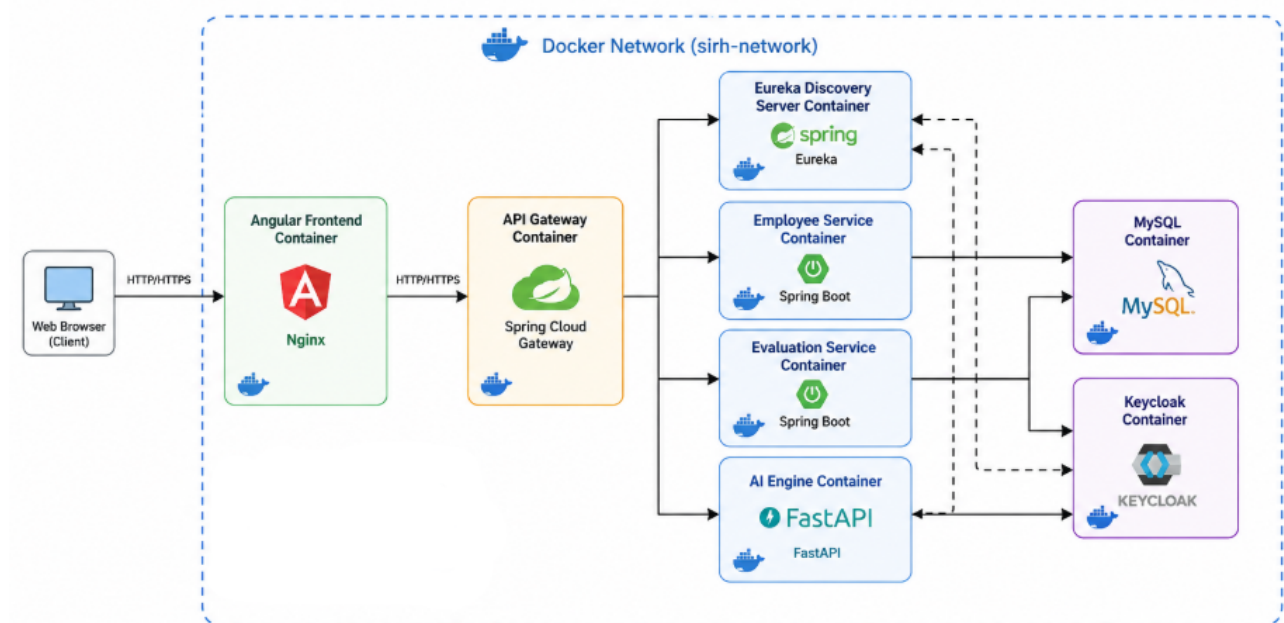


Figure 5.1: Sprint 5 containerized architecture

The containerized environment is composed of the following services:

- Angular frontend;
- API Gateway;
- Eureka Discovery Server;
- Employee Service;
- Evaluation Service;
- FastAPI AI Engine;
- MySQL;
- Keycloak.

Each component is isolated in its own container while remaining connected to the shared `sirh-network` bridge network.

5.1.2.2 Containerization Strategy

The adopted strategy applies one deployable image per major component. This separation improves portability, fault isolation, reproducibility, and future scalability. [W29]

Table 5.2 summarizes the containerized components of the SIRH platform together with their primary responsibilities within the overall architecture.

Table 5.2: Containerized components of the SIRH platform

Component	Role
Angular Frontend	Provides role-based interfaces for HR Administrator, Manager, and Employee users.
API Gateway	Acts as the platform entry point, validates JWT tokens, and routes requests to the appropriate microservices.
Eureka Discovery Server	Provides service registration and discovery for backend services.
Employee Service	Manages employees, departments, business services, contracts, documents, and organizational information.
Evaluation Service	Manages campaigns, evaluations, competencies, objectives, development plans, dashboards, and training recommendations.

Component	Role
FastAPI AI Engine	Generates AI-assisted training recommendations and exposes the open-course catalogue import endpoints.
MySQL	Stores application data and the Keycloak persistence schema.
Keycloak	Provides authentication, authorization, role-based access control, and JWT token issuance.

5.1.2.3 Runtime Configuration Strategy

Sensitive and environment-specific values are externalized through environment variables. Docker Compose reads the local `.env` file, which is excluded from source control.

The configuration includes:

- database names and credentials;
- Keycloak administrator and client configuration;
- Eureka service URL;
- JWT JWK Set URI;
- AI service URL;
- frontend URL;
- mail server configuration;
- JVM memory settings.

This approach avoids embedding environment-dependent values directly in source code or Dockerfiles.

5.1.3 Realization of Sprint 5

5.1.3.1 Infrastructure Organization

To simplify deployment and maintenance, the SIRH project separates the application source code from the infrastructure resources. The repository is organized into dedicated directories for the backend microservices, Angular frontend, Docker configuration, authentication resources, database initialization scripts, and monitoring components.

As illustrated in Figure 5.2, the `infra` directory contains the Docker Compose configuration, Keycloak customization, MySQL initialization scripts, Kubernetes manifests, monitoring resources

(Prometheus, Grafana, Loki, and Alloy), environment variables, and the SonarQube configuration used throughout the DevSecOps pipeline.

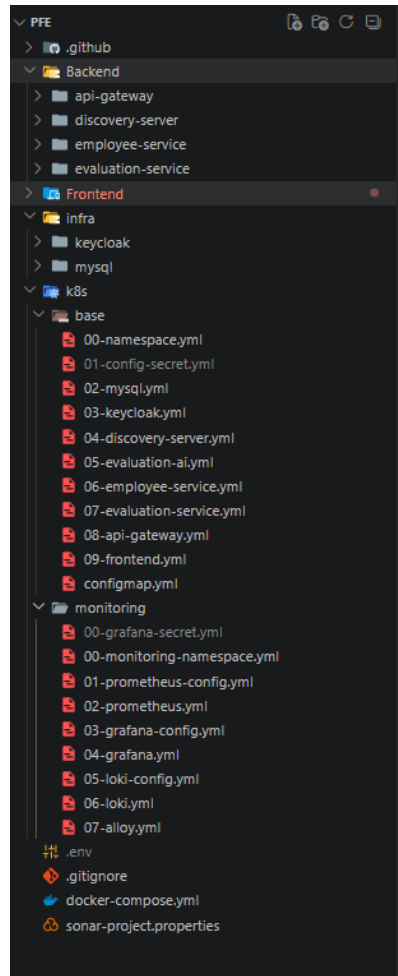


Figure 5.2: Infrastructure organization of the SIRH project

5.1.3.2 Angular Frontend Containerization

The Angular frontend uses a multi-stage Dockerfile.

- The build stage installs dependencies and generates the optimized production bundle.
- The runtime stage serves the generated static files through Nginx.

This strategy reduces the runtime image size because Node.js and build tools are not included in the final image.

The frontend container exposes port 80 internally and port 4200 on the host. A dedicated `/health` endpoint is used by the Docker health check.

5.1.3.3 Spring Boot Microservices Containerization

The Discovery Server, API Gateway, Employee Service, and Evaluation Service are packaged as executable JAR files and executed in dedicated Java runtime containers.

The containers receive their runtime configuration through environment variables. JVM heap and metaspace limits are explicitly configured to avoid excessive memory consumption in the local environment.

Container health is monitored using the Spring Boot Actuator readiness endpoint:

```
/actuator/health/readiness
```

5.1.3.4 FastAPI AI Engine Containerization

The AI component is containerized independently from the Java services. It exposes port 8000 and provides a dedicated `/health` endpoint.

Keeping the AI engine independent allows the Evaluation Service and the AI module to evolve separately and preserves technology isolation between Spring Boot and FastAPI.

5.1.3.5 MySQL Containerization

MySQL 8 is used as the relational database engine. The container is configured with:

- UTF-8 character encoding;
- a persistent `mysql_data` volume;
- initialization scripts mounted read-only;
- an explicit health check based on `mysqladmin ping`;
- a restart policy using `unless-stopped`.

The health check is used by dependent services to avoid starting before the database is ready.

5.1.3.6 Keycloak Containerization

Keycloak is built from the custom infrastructure directory and includes the SIRH authentication theme.

The container uses MySQL as its persistence backend and exposes port 8085 on the host. It provides authentication, authorization, role management, and JWT token issuance for the Angular frontend and backend resource servers.

5.1.3.7 Docker Compose Orchestration

The complete platform is orchestrated through `docker-compose.yml`. The file defines services, images, build contexts, networks, ports, persistent volumes, environment variables, dependencies, health checks, and restart policies.

The startup chain is controlled through health-based dependencies:

- Keycloak starts after MySQL becomes healthy.
- Employee Service starts after MySQL, Keycloak, and Eureka become healthy.
- Evaluation Service starts after MySQL, Keycloak, Eureka, Employee Service, and the AI Engine become healthy.
- API Gateway starts after Keycloak, Eureka, Employee Service, and Evaluation Service become healthy.
- Frontend starts after Keycloak and API Gateway become healthy.

5.1.3.8 Networks, Volumes and Restart Policies

All services communicate through the dedicated `sirh-network` bridge network.

Two persistent Docker volumes are defined:

- `mysql_data` for relational data;
- `employee_uploads` for employee photos and documents.

All long-running services use the `unless-stopped` restart policy, which improves local resilience while preserving explicit developer control.

5.1.3.9 Docker Compose Validation

The environment is started with:

```
docker compose up -d --build
```

The running containers are then verified using Docker Desktop together with Docker Compose commands.

Figure 5.3 shows the successful execution of the containerized SIRH platform. The screenshot confirms that all application services were built, started, and running correctly within the Docker Compose environment.

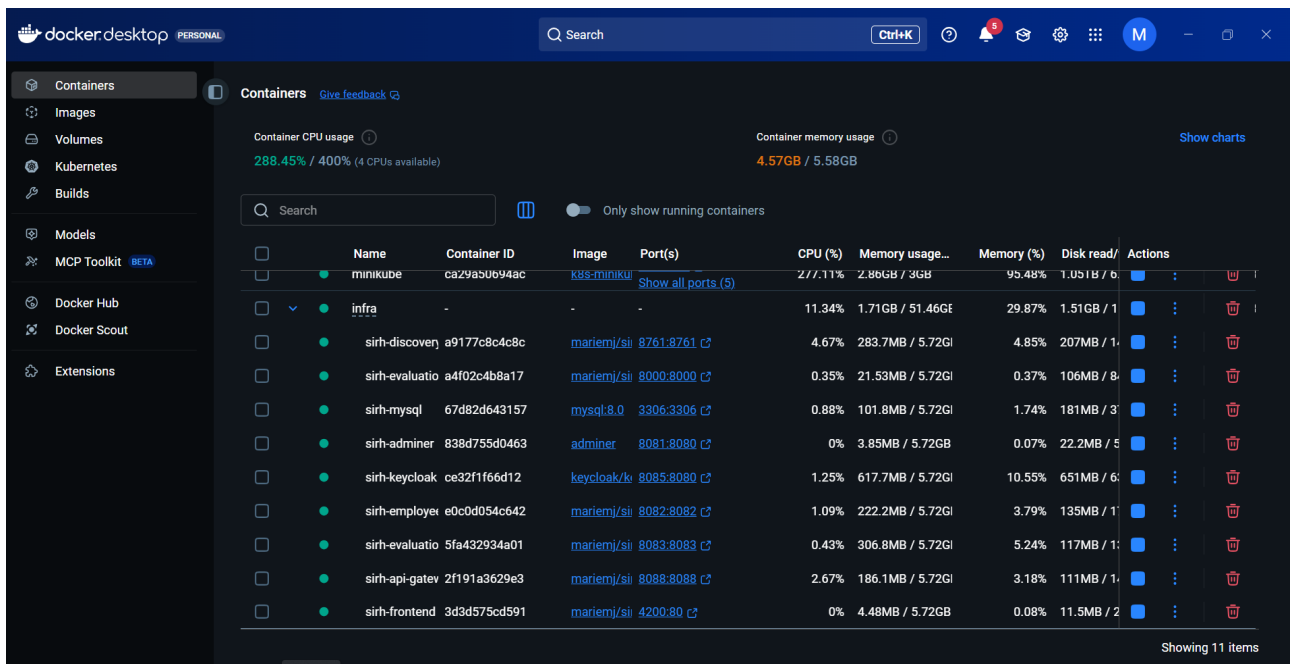


Figure 5.3: Successful Docker Compose execution of the SIRH platform

The validation confirmed that the frontend, API Gateway, Discovery Server, Employee Service, Evaluation Service, AI Engine, Keycloak, and MySQL were running and reachable through the configured ports.

5.2 Sprint 6: CI/CD Automation, Security and Observability

5.2.1 Sprint 6 Backlog

Sprint 6 builds on the containerized environment established in Sprint 5. It automates quality and security controls, publishes validated images, verifies Kubernetes manifests, deploys the platform to Minikube, and introduces observability.

Table 5.3: Sprint 6 backlog

ID	User Story	Priority	Estimate
US-01	As a developer, I want to configure GitHub Actions so that tests, analyses, security scans, Docker builds, and Kubernetes validation are automated.	High	2 days
US-02	As a developer, I want to generate backend and frontend coverage reports so that tested and untested code can be measured.	High	1 day

ID	User Story	Priority	Estimate
US-03	As a developer, I want to integrate SonarQube Cloud and its Quality Gate so that maintainability, reliability, security, duplication, and coverage are controlled automatically.	High	2 days
US-04	As a developer, I want to scan dependencies, source code, secrets, filesystems, and Docker images using OWASP Dependency-Check, npm audit, GitLeaks, and Trivy.	High	2 days
US-05	As a developer, I want to build and publish the seven SIRH Docker images to Docker Hub so that validated images are available for deployment.	High	1 day
US-06	As a developer, I want to validate application and monitoring manifests with Kubeconform so that invalid Kubernetes resources are detected before deployment.	High	2 days
US-07	As a developer, I want to deploy the platform to Minikube and verify all rollouts so that the Kubernetes environment is operational.	High	1 day
US-08	As an operator, I want to monitor metrics using Prometheus and Grafana so that service health and resource usage are visible.	High	1 day
US-09	As an operator, I want to centralize Kubernetes logs with Alloy and Loki so that logs can be consulted from Grafana.	High	1 day
US-10	As a project team, I want to archive coverage, security, and Docker build reports as workflow artifacts so that every pipeline execution is traceable.	Medium	1 day
Total			14 days

5.2.2 Sprint 6 Analysis and Design

The objective of Sprint 6 is to establish an automated and traceable DevSecOps process around the images created in Sprint 5.

5.2.2.1 CI/CD Architecture

Figure 5.4 presents the CI/CD architecture.

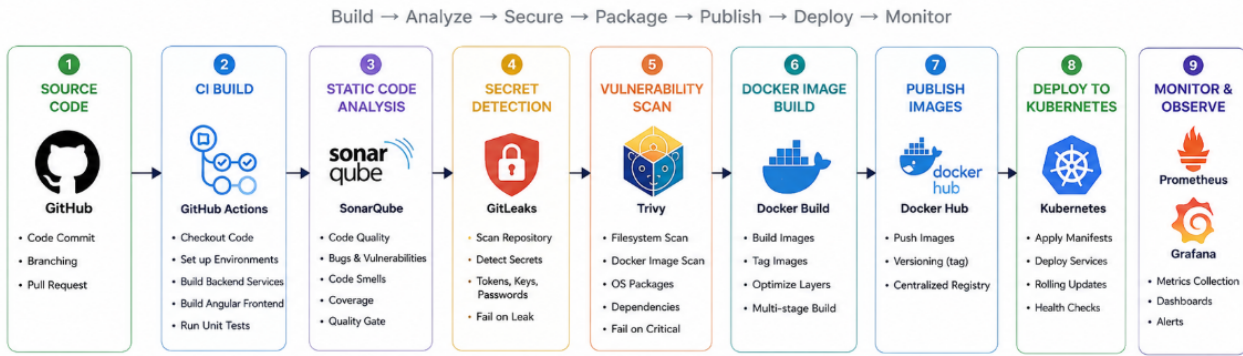


Figure 5.4: CI/CD and DevSecOps pipeline of the SIRH platform

The pipeline follows these stages:

1. source code checkout;
2. Java and Angular tests;
3. JaCoCo and LCOV coverage generation;
4. OWASP Dependency-Check and npm audit;
5. SonarQube Cloud analysis and Quality Gate;
6. GitLeaks and Trivy filesystem scans;
7. Parallel Docker image build;
8. Docker image vulnerability scan;
9. Docker Hub publication;
10. artifact archiving;
11. Kubeconform validation.

5.2.2.2 Deployment Strategy

The GitHub-hosted pipeline produces deployment-ready images and validated manifests. The final Minikube deployment is executed locally because the cluster runs on the developer workstation and cannot be reached directly by GitHub-hosted runners.

The adopted flow is:

GitHub Actions → Docker Hub → Validated Kubernetes Manifests → Local Minikube

5.2.2.3 Observability Architecture

The observability stack is divided into metrics and logs:

- **Metrics:** Spring Boot Actuator → Prometheus → Grafana.
- **Logs:** Kubernetes Pods → Alloy → Loki → Grafana.

5.2.3 Realization of Sprint 6

5.2.3.1 GitHub Actions Pipeline

The main workflow is defined in `.github/workflows/ci-cd.yml`. It is triggered automatically on code pushes, pull requests, and can also be executed manually.

The workflow is organized into three main jobs:

Test and Quality: Executes backend and frontend validation tasks, including dependency installation, application build, code quality analysis, vulnerability checks, SonarQube analysis, Quality Gate verification, and report generation.

Build and Scan: Uses a matrix strategy to build, scan, tag, and publish the Docker images corresponding to the platform services.

Validate Kubernetes: Validates the Kubernetes manifest files before deployment to ensure configuration consistency.

Figure 5.5 illustrates the overall GitHub Actions workflow executed during the CI/CD process.

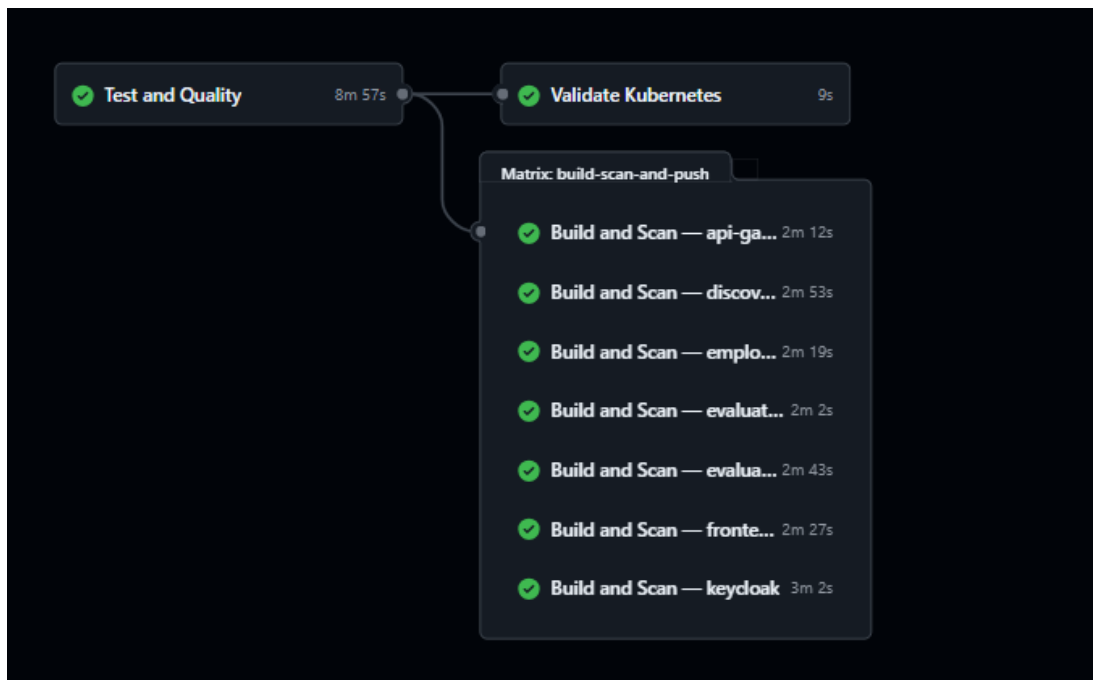


Figure 5.5: GitHub Actions CI/CD workflow

5.2.3.2 Automated Tests and Coverage

The Java microservices are tested using Maven, JUnit, and Mockito during the CI/CD pipeline execution. The Maven `verify` phase generates JaCoCo code coverage reports for the following backend services:

- Discovery Server;
- API Gateway;
- Employee Service;
- Evaluation Service.

The Angular frontend is tested using ChromeHeadless. LCOV and HTML coverage reports are generated and validated before the SonarQube analysis to ensure adequate frontend test coverage.

The CI/CD pipeline also archives the generated reports and build artifacts, including backend test reports, JaCoCo coverage reports, Angular production bundles, Angular coverage reports, dependency analysis reports, and security scan results for later inspection. [W38, W39, W40, W41, W42]

Figures 5.6 and 5.7 illustrate the successful execution of the testing and quality verification job together with the artifacts generated by the pipeline.

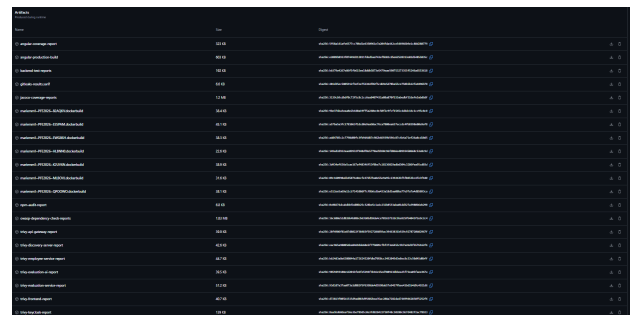
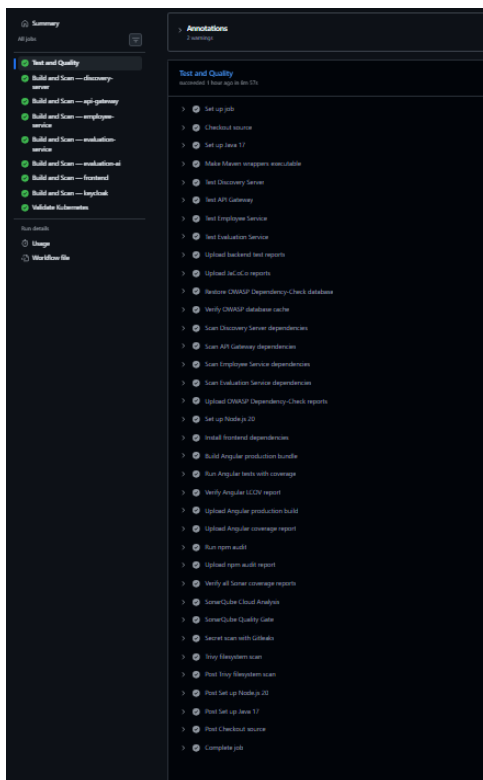


Figure 5.7: Generated pipeline artifacts (B)

Figure 5.6: Test and Quality job execution (A)

5.2.3.3 SonarQube Cloud Analysis

SonarQube Cloud evaluates the quality of the source code by analyzing maintainability, reliability, security, code duplication, code smells, and test coverage. The analysis imports the JaCoCo XML reports generated by the backend services together with the LCOV coverage report produced by the Angular frontend. [W42]

After the analysis is completed, SonarQube Cloud evaluates the Quality Gate. If the Quality Gate fails, the CI/CD pipeline is interrupted before the Docker images are published, ensuring that only code meeting the defined quality standards can proceed to the deployment stage.

Figure 5.8 presents the SonarQube Cloud project summary after a successful analysis. It highlights the overall Quality Gate status together with the project's security, reliability, maintainability, code coverage, and duplication metrics.

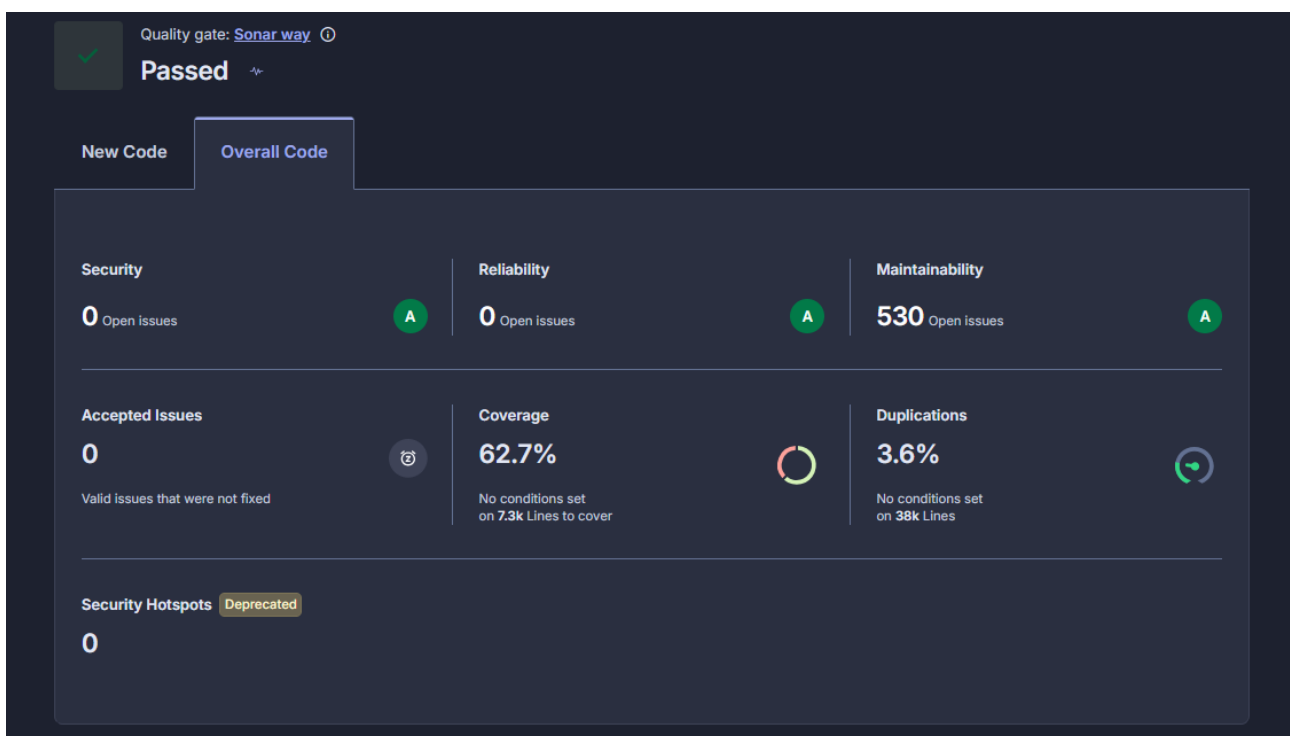


Figure 5.8: SonarQube Cloud project summary

5.2.3.4 OWASP Dependency-Check

OWASP Dependency-Check scans the Maven dependency trees of the Java services to identify known vulnerabilities in third-party libraries before deployment.

To improve the efficiency of the CI/CD process, a dedicated workflow, `./update-owasp-database.yml`, was created to maintain the OWASP Dependency-Check vulnerability database. The workflow is scheduled to run automatically every Monday and can also be triggered manually when

an immediate database update is required. During its execution, it restores the previously cached database, downloads the latest vulnerability information from the National Vulnerability Database (NVD) using the official API, verifies the generated files, and stores the updated cache for subsequent workflow executions. [W44, W45]

This separation significantly reduces the execution time of the main CI/CD pipeline by avoiding the download and regeneration of the complete vulnerability database during every code push.

Figures 5.9 and 5.10 present the successful execution of the dedicated OWASP Dependency-Check database update workflow. Figure A shows the workflow summary, while Figure B illustrates the detailed execution steps performed during the database update process.

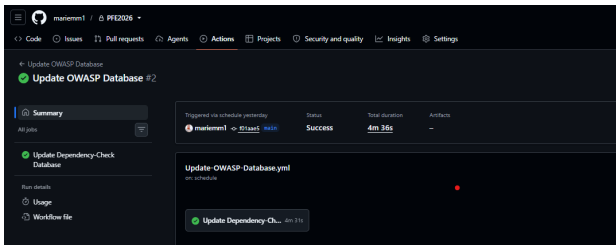


Figure 5.9: OWASP database update workflow (A)

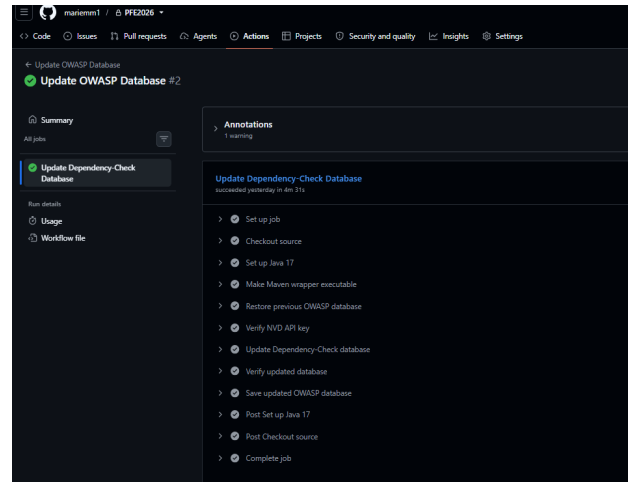


Figure 5.10: OWASP database update workflow (B)

5.2.3.5 GitLeaks Secret Detection

GitLeaks is integrated into the CI/CD pipeline to detect exposed secrets, API keys, passwords, tokens, and other sensitive information that may have been accidentally committed to the repository. The scan is performed automatically during every pipeline execution before the Docker images are built and published. [W47]

Figure 5.11 presents the successful execution of the GitLeaks scan during the CI/CD workflow. The analysis confirms that no secrets or sensitive credentials were detected in the project repository.

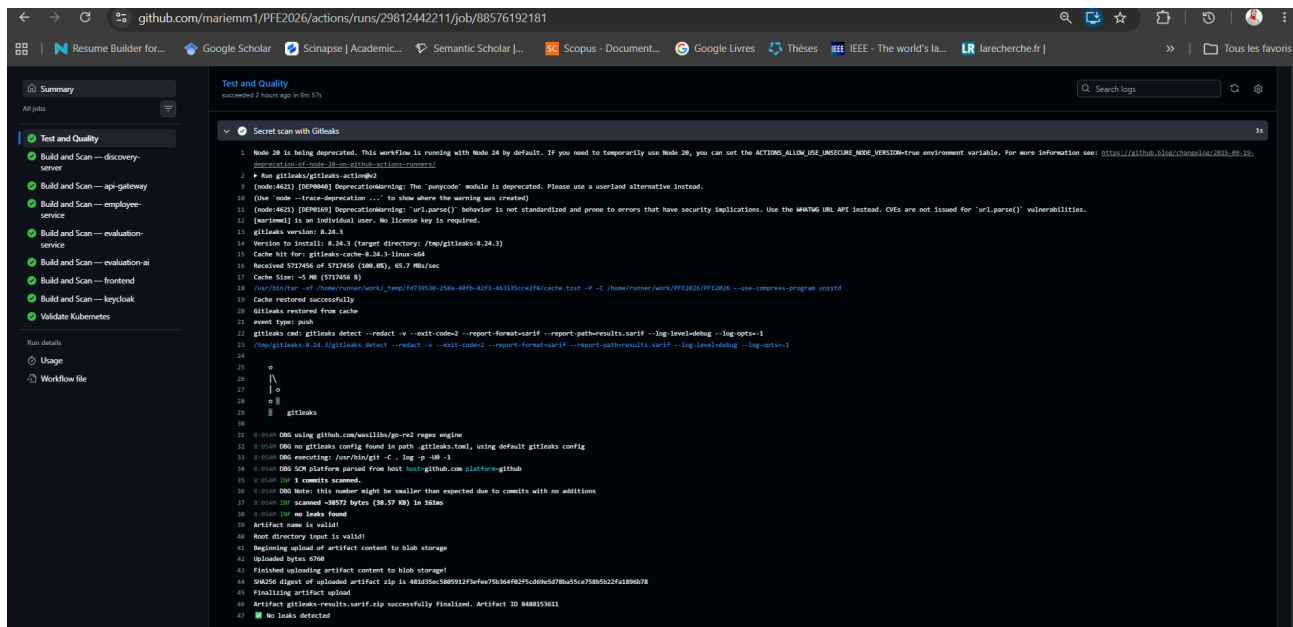


Figure 5.11: GitLeaks secret detection workflow showing that no leaks were detected

5.2.3.6 Trivy Filesystem and Image Scanning

Trivy is executed automatically during every CI/CD pipeline execution before the Docker images are published. This additional verification ensures that vulnerable dependencies and container images are detected early in the deployment process. [W48]

Trivy provides two complementary security controls:

- filesystem scanning of the repository, dependencies, and configuration files;
- vulnerability scanning of each Docker image.

The scans focus on HIGH and CRITICAL findings. JSON reports are generated for every image and uploaded as artifacts.

5.2.3.7 Docker Matrix Build and Publication

Docker Buildx is used together with a matrix strategy to build the Docker images of the SIRH platform. Each component is built independently, allowing the pipeline to execute multiple image builds in parallel while reducing the overall execution time. [W32]

The following Docker images are generated during the workflow:

- `sirh-discovery-server`;
- `sirh-api-gateway`;
- `sirh-employee-service`;

- `sirh-evaluation-service`;
- `sirh-evaluation-ai`;
- `sirh-frontend`;
- `sirh-keycloak`.

Before publication, every image is scanned for security vulnerabilities using Trivy. When all security and quality checks are successfully completed, the images are automatically pushed to Docker Hub. Each published image receives two tags: the corresponding Git commit SHA for traceability and the `latest` tag for deployment convenience.

Figure 5.12 presents the Docker Hub repository containing the Docker images generated and published automatically by the CI/CD pipeline.

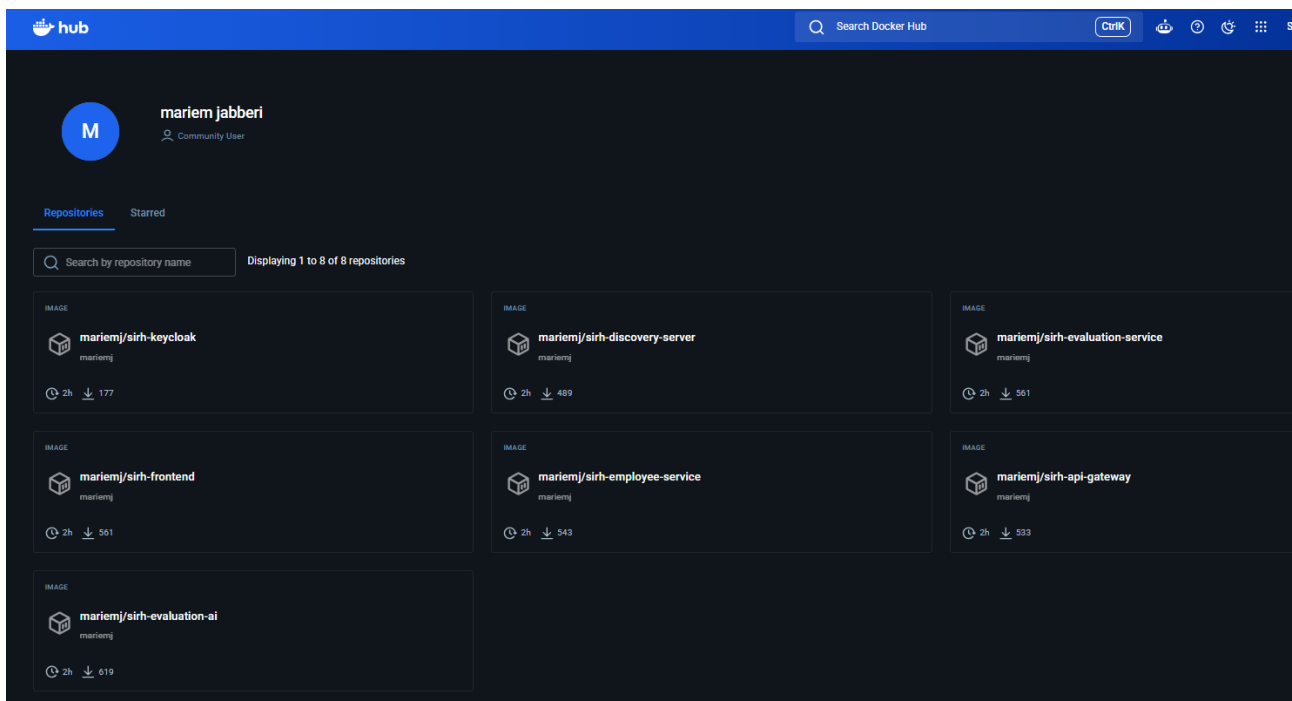


Figure 5.12: Docker images of the SIRH platform published to Docker Hub

5.2.3.8 Local Minikube Deployment

After the successful execution of the CI/CD pipeline, the validated Kubernetes manifests are deployed to a local Minikube cluster using the following commands: [W49, W51, W50]

```
kubectl apply -f k8s/base/
kubectl apply -f k8s/monitoring/
```

The deployment is then verified by checking the status of the deployments, services, and running pods:

```
kubectl get deployments -n sirh
kubectl get services -n sirh
kubectl get pods -n sirh
kubectl rollout status deployment/<name> -n sirh
```

All application deployments successfully reached the **Available** state, while the corresponding services and pods were correctly created and entered the **Running** state.

The deployed environment includes:

- ClusterIP services for internal communication;
- NodePort services for externally accessible components;
- ConfigMaps and Secrets for application configuration;
- PersistentVolumeClaims for persistent storage;
- Startup, readiness, and liveness probes;
- Resource requests and limits for container management.

Figures 5.13, 5.14, and 5.15 present the successful deployment verification on the local Minikube cluster. Figure A shows the deployed Kubernetes deployments, Figure B presents the created services, and Figure C illustrates the running pods after deployment.

```
PS C:\Users\Mariem\PFE> kubectl get deployments -n sirh
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
keycloak             1/1     1             1           11d
mysql                1/1     1             1           11d
sirh-api-gateway     1/1     1             1           11d
sirh-discovery-server 1/1     1             1           11d
sirh-employee-service 1/1     1             1           11d
sirh-evaluation-ai   1/1     1             1           11d
sirh-evaluation-service 1/1     1             1           11d
sirh-frontend        1/1     1             1           11d
```

Figure 5.13: Kubernetes deployments (A)

```
PS C:\Users\Mariem\PFE> kubectl get services -n sirh
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
api-gateway         NodePort    10.101.93.245 <none>         8088:30088/TCP   11d
discovery-server    ClusterIP   10.109.147.253 <none>         8761/TCP         11d
employee-service    ClusterIP   10.108.121.250 <none>         8082/TCP         11d
evaluation-ai       ClusterIP   10.104.116.131 <none>         8080/TCP         11d
evaluation-service   ClusterIP   10.104.146.58  <none>         8083/TCP         11d
frontend            NodePort    10.98.149.232  <none>         80:30090/TCP     11d
keycloak            ClusterIP   10.105.101.111 <none>         8080/TCP         11d
mysql               ClusterIP   10.108.114.182 <none>         3306/TCP         11d
```

Figure 5.14: Kubernetes services (B)

```
PS C:\Users\Mariem\PFE> kubectl get pods -n sirh
NAME                                READY   STATUS    RESTARTS   AGE
keycloak-865cb9d49b-q26d1           1/1     Running   8 (158m ago)  9d
mysql-b6c965ff6-69qn9               1/1     Running   8 (158m ago)  9d
sirh-api-gateway-7b9bbf48b4-4gccz    1/1     Running   8 (158m ago)  9d
sirh-discovery-server-b864996f5-tmvfk 1/1     Running   8 (158m ago)  9d
sirh-employee-service-5556c5d656-zffn2 1/1     Running   8 (158m ago)  9d
sirh-evaluation-ai-77796bb499-xnzzrq 1/1     Running   8 (158m ago)  9d
sirh-evaluation-service-68b45d9dbb-7d7g8 1/1     Running   10 (158m ago)  9d
sirh-frontend-597fc47889-qlfw9       1/1     Running   8 (158m ago)  9d
```

Figure 5.15: Running Kubernetes pods (C)

5.2.3.9 Prometheus Metrics Collection

Prometheus is used to continuously collect runtime metrics from the SIRH microservices through the Spring Boot Actuator `/actuator/prometheus` endpoint. The collected metrics provide real-time information about the health and performance of the deployed application. [W55,]

The following services expose Prometheus metrics:

- API Gateway;
- Discovery Server;
- Employee Service;
- Evaluation Service.

Figure 5.16 presents the Prometheus target status page after the deployment. The monitoring server successfully discovers all configured services and reports each target in the UP state, confirming that the metrics endpoints are reachable and operating correctly.

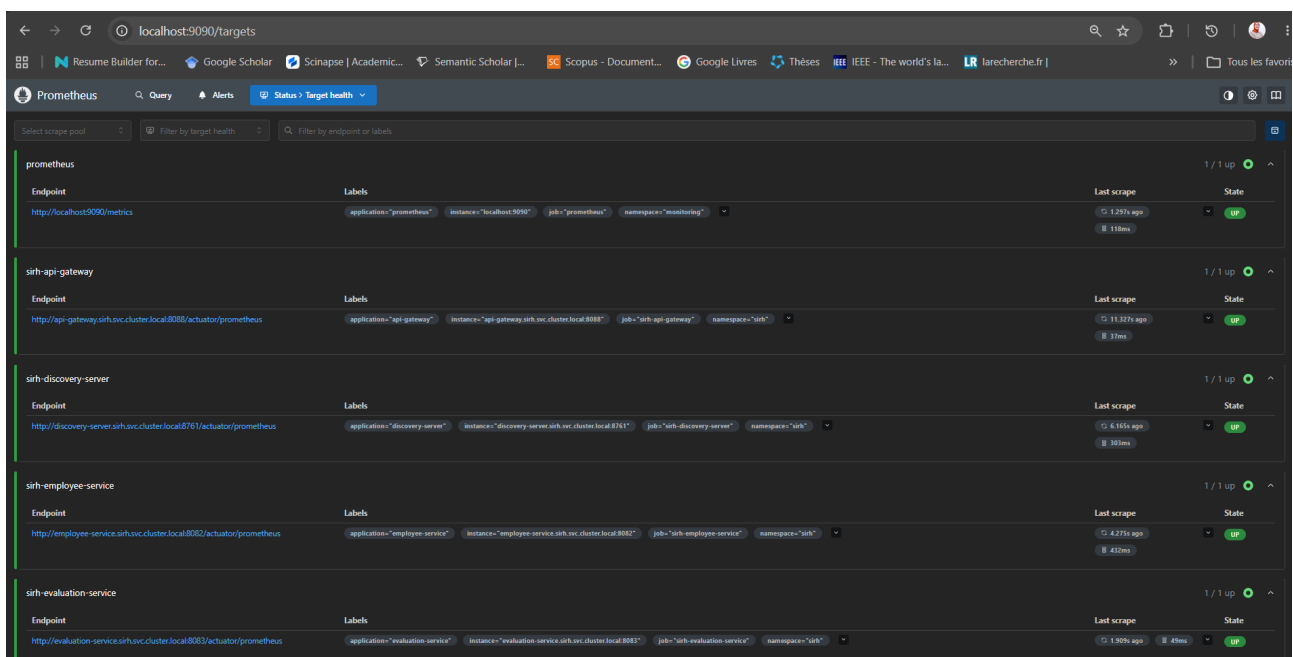


Figure 5.16: Prometheus monitoring targets reporting the UP state

5.2.3.10 Grafana Dashboard

Grafana is automatically provisioned with Prometheus and Loki as data sources to provide centralized visualization of the metrics and logs collected from the SIRH platform. The provisioning process also deploys a predefined *SIRH Overview* dashboard, allowing the monitoring environment to be available immediately after deployment. [W57, W55, W58]

The dashboard provides real-time visualization of the main operational indicators of the platform, including:

- overall service health;
- process CPU usage;
- JVM heap usage;
- HTTP requests per second;
- service uptime;
- centralized Kubernetes logs.

Figure 5.17 presents the *SIRH Overview* dashboard implemented for the platform. The dashboard confirms that all services are operational while providing real-time metrics related to resource utilization, application performance, request throughput, uptime monitoring, and centralized log aggregation.

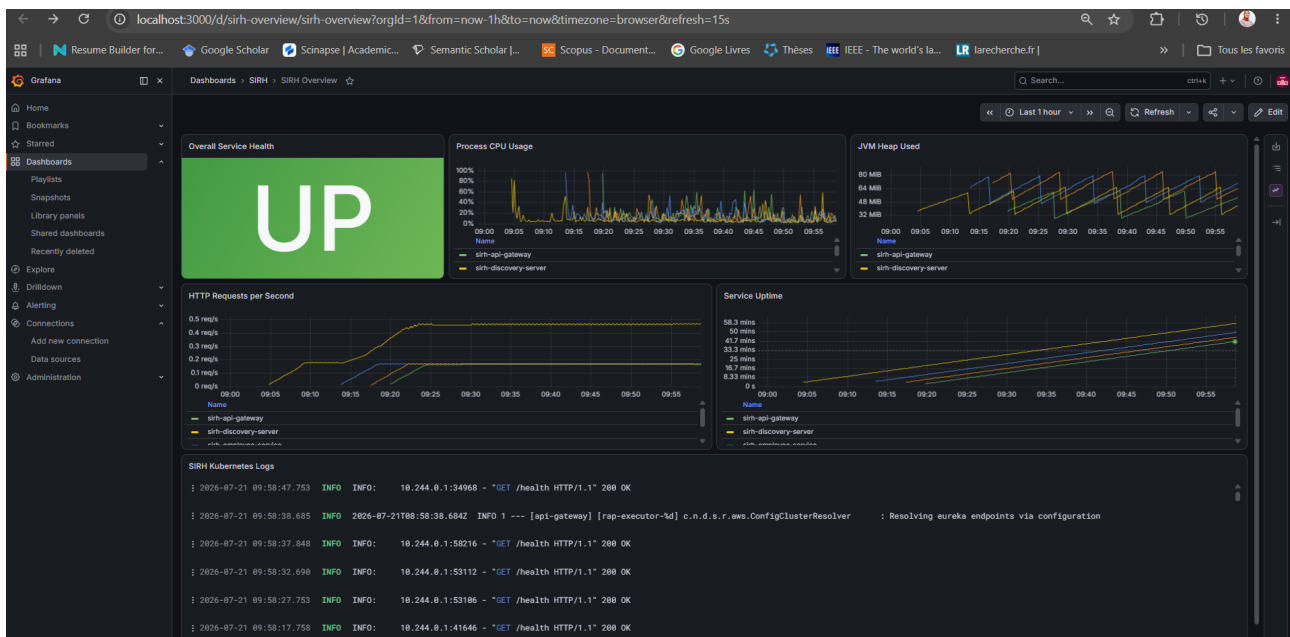


Figure 5.17: Grafana *SIRH Overview* observability dashboard

5.2.3.11 Centralized Logging with Alloy and Loki

Centralized log collection is implemented using Alloy and Loki. Alloy automatically discovers the SIRH pods through the Kubernetes API, collects their logs, enriches them with metadata, and forwards them to the Loki server for storage and indexing. [W59, W58]

A dedicated `ServiceAccount`, `ClusterRole`, and `ClusterRoleBinding` grant Alloy the permissions required to access pod logs across the Kubernetes cluster.

Each log stream is enriched with labels such as the namespace, pod, container, application, cluster, and project, making it possible to efficiently filter and analyze logs.

Grafana retrieves the logs stored in Loki using LogQL queries such as:

```
{namespace="sirh"}
```

Figure 5.18 presents the Grafana Explore interface connected to Loki. The screenshot shows the centralized logs collected from all SIRH services, together with the log volume timeline and metadata labels used for searching and filtering log entries.

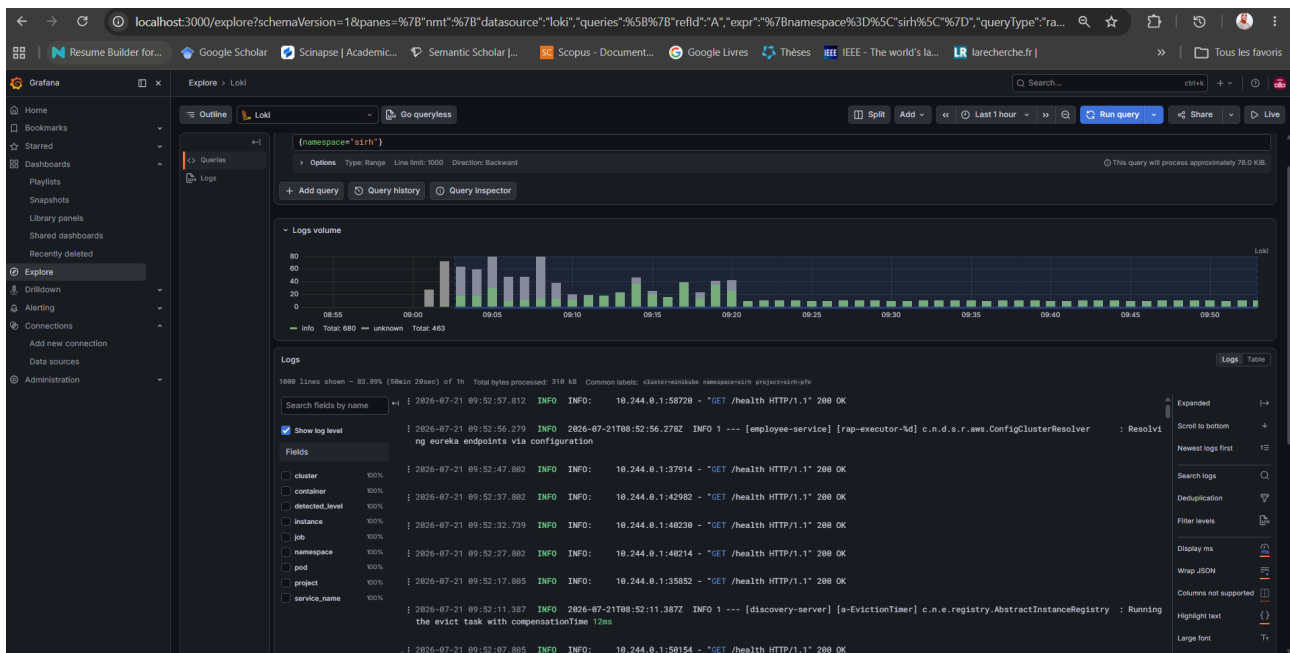


Figure 5.18: Grafana Explore displaying centralized logs collected by Loki

Conclusion

Release 3 completed the operationalization of the SIRH platform. Sprint 5 provided containerized and reproducible execution through Docker and Docker Compose. Sprint 6 automated the verification and publication lifecycle through GitHub Actions, SonarQube Cloud, OWASP Dependency-Check, npm audit, GitLeaks, Trivy, Docker Buildx, Docker Hub, Kubeconform, Prometheus, Grafana, Loki, and Alloy.

The resulting platform is portable, reproducible, secure, observable, and deployment-ready. The separation between automated GitHub-hosted CI and local Minikube deployment was intentionally adopted because the Kubernetes cluster is hosted on the developer workstation and is not directly accessible from GitHub-hosted runners.

General Conclusion and Perspectives

This final-year project presented the design, development, and validation of a secure Human Resources Information System (HRIS) based on a microservices architecture. The proposed solution was designed to overcome the limitations of traditional monolithic HR management systems by providing a modular, scalable, secure, and maintainable platform capable of supporting the digital transformation of Human Resources management.

The project combined several modern technologies to build a complete enterprise application. Spring Boot microservices were used to implement the backend business logic, while Angular provided a responsive and user-friendly web interface. Authentication and authorization were centralized using Keycloak with JWT-based security and role-based access control. Service registration and request routing were ensured through Netflix Eureka and Spring Cloud Gateway, enabling loose coupling and independent deployment of the different services.

From a functional perspective, the platform successfully implements employee administration, department and service management, contract and document management, organizational structure visualization, performance evaluation campaigns, competency assessment, objective tracking, development plans, dashboards, reporting, and AI-assisted training recommendations. The integration of a FastAPI-based Artificial Intelligence module further enhances the platform by generating personalized training recommendations according to employee competency gaps and evaluation results.

From a software engineering perspective, the project followed the Agile Scrum methodology and was developed through six consecutive sprints organized into three major releases. This iterative approach enabled the gradual implementation, validation, and continuous improvement of both functional and technical features throughout the project lifecycle.

The final release introduced DevSecOps practices that significantly improved the quality and maintainability of the platform. All application components were containerized using Docker and orchestrated locally with Docker Compose. A complete CI/CD pipeline was implemented using GitHub Actions to automate testing, code quality analysis, security scanning, Docker image publication, and deployment validation. Additional observability capabilities were integrated through Prometheus, Grafana, Loki, and Alloy, providing centralized monitoring, metrics collection, and log aggregation.

Overall, the objectives defined at the beginning of the project were successfully achieved. The

resulting platform combines modern software engineering practices, secure authentication, Artificial Intelligence, containerization, and DevSecOps into a coherent, reliable, and deployment-ready enterprise solution. Beyond fulfilling the functional requirements of a Human Resources Information System, the project demonstrates the practical application of contemporary cloud-native development principles and highlights the benefits of combining microservices architecture with automated software delivery and observability.

Although the proposed solution provides a complete and functional HRIS platform, several enhancements can be considered to further extend its capabilities:

- Deploy the platform on cloud infrastructures such as Microsoft Azure or Amazon Web Services to improve availability, scalability, and operational resilience.
- Extend the Artificial Intelligence module by integrating predictive machine learning models capable of estimating an employee's final performance score before the end of an evaluation campaign. Such a model could exploit historical evaluation scores, objective achievement rates, competency assessments, seniority, organizational information, and previous development plan completion. Algorithms such as Random Forest Regressor or XGBoost could be trained on historical evaluation data to predict future performance while providing confidence intervals. The models could be versioned using MLflow and automatically retrained after each completed evaluation campaign, enabling continuous improvement of prediction accuracy.

In conclusion, this project demonstrates that combining microservices architecture, secure authentication, Artificial Intelligence, containerization, DevSecOps, and observability makes it possible to develop a modern enterprise application that is secure, scalable, maintainable, and ready for future evolution. The proposed HRIS provides a solid technological foundation upon which additional business functionalities and cloud-native capabilities can be progressively built.

Webography

- [W1] Scrum.org, *The Scrum Guide*. Available at: <https://scrumguides.org/>. Accessed: 18 February 2026.
- [W3] Oracle, *Java Documentation*. Available at: <https://docs.oracle.com/en/java/>. Accessed: 22 February 2026.
- [W4] Spring, *Spring Framework Reference Documentation*. Available at: <https://docs.spring.io/spring-framework/reference/>. Accessed: 26 February 2026.
- [W5] Spring, *Spring Boot Reference Documentation*. Available at: <https://docs.spring.io/spring-boot/index.html>. Accessed: 2 March 2026.
- [W6] Spring, *Spring Security Reference Documentation*. Available at: <https://docs.spring.io/spring-security/reference/>. Accessed: 5 March 2026.
- [W7] Spring, *Spring Cloud Documentation*. Available at: <https://spring.io/projects/spring-cloud>. Accessed: 8 March 2026.
- [W8] Spring, *Spring Cloud Gateway Reference Documentation*. Available at: <https://docs.spring.io/spring-cloud-gateway/reference/>. Accessed: 10 March 2026.
- [W9] Netflix, *Eureka Service Discovery Documentation*. Available at: <https://github.com/Netflix/eureka/wiki>. Accessed: 12 March 2026.
- [W10] Spring, *Spring Data JPA Reference Documentation*. Available at: <https://docs.spring.io/spring-data/jpa/reference/>. Accessed: 15 March 2026.
- [W11] Hibernate, *Hibernate ORM Documentation*. Available at: <https://hibernate.org/orm/documentation/>. Accessed: 18 March 2026.
- [W12] Apache Software Foundation, *Apache Maven Documentation*. Available at: <https://maven.apache.org/guides/>. Accessed: 21 March 2026.
- [W13] Google, *Angular Documentation*. Available at: <https://angular.dev/>. Accessed: 24 March 2026.
- [W14] Microsoft, *TypeScript Documentation*. Available at: <https://www.typescriptlang.org/docs/>. Accessed: 27 March 2026.
- [W15] RxJS, *RxJS Documentation*. Available at: <https://rxjs.dev/>. Accessed: 30 March 2026.

- [W16] OpenJS Foundation, *Node.js Documentation*. Available at: <https://nodejs.org/docs/latest/api/>. Accessed: 2 April 2026.
- [W17] NGINX, *NGINX Documentation*. Available at: <https://nginx.org/en/docs/>. Accessed: 5 April 2026.
- [W18] Keycloak, *Keycloak Documentation*. Available at: <https://www.keycloak.org/documentation>. Accessed: 8 April 2026.
- [W19] Keycloak, *Server Administration Guide*. Available at: https://www.keycloak.org/docs/latest/server_admin/. Accessed: 11 April 2026.
- [W20] Keycloak, *Securing Applications Guide*. Available at: https://www.keycloak.org/docs/latest/securing_apps/. Accessed: 14 April 2026.
- [W21] OpenAPI Initiative, *OpenAPI Specification*. Available at: <https://spec.openapis.org/oas/latest.html>. Accessed: 17 April 2026.
- [W22] SmartBear, *Swagger Documentation*. Available at: <https://swagger.io/docs/>. Accessed: 20 April 2026.
- [W23] Postman, *Postman Documentation*. Available at: <https://learning.postman.com/docs/>. Accessed: 23 April 2026.
- [W24] Oracle, *MySQL Reference Manual*. Available at: <https://dev.mysql.com/doc/>. Accessed: 26 April 2026.
- [W25] Python Software Foundation, *Python Documentation*. Available at: <https://docs.python.org/3/>. Accessed: 29 April 2026.
- [W26] Sebastián Ramírez, *FastAPI Documentation*. Available at: <https://fastapi.tiangolo.com/>. Accessed: 3 May 2026.
- [W27] Scikit-learn Developers, *Scikit-learn User Guide*. Available at: https://scikit-learn.org/stable/user_guide.html. Accessed: 7 May 2026.
- [W28] MLflow, *MLflow Documentation*. Available at: <https://mlflow.org/docs/latest/>. Accessed: 10 May 2026.
- [W29] Docker Inc., *Docker Documentation*. Available at: <https://docs.docker.com/>. Accessed: 15 May 2026.

- [W30] Docker Inc., *Dockerfile Reference*. Available at: <https://docs.docker.com/reference/dockerfile/>. Accessed: 18 May 2026.
- [W31] Docker Inc., *Docker Compose Documentation*. Available at: <https://docs.docker.com/compose/>. Accessed: 22 May 2026.
- [W32] Docker Inc., *Docker Buildx Documentation*. Available at: <https://docs.docker.com/build/buildx/>. Accessed: 25 May 2026.
- [W33] Docker Inc., *Docker Hub Documentation*. Available at: <https://docs.docker.com/docker-hub/>. Accessed: 28 May 2026.
- [W34] Git, *Git Documentation*. Available at: <https://git-scm.com/doc>. Accessed: 1 June 2026.
- [W35] GitHub, *GitHub Documentation*. Available at: <https://docs.github.com/>. Accessed: 4 June 2026.
- [W36] GitHub, *GitHub Actions Documentation*. Available at: <https://docs.github.com/en/actions>. Accessed: 8 June 2026.
- [W37] GitHub, *Workflow Artifacts Documentation*. Available at: <https://docs.github.com/en/actions/using-workflows/storing-workflow-data-as-artifacts>. Accessed: 11 June 2026.
- [W38] JUnit, *JUnit 5 User Guide*. Available at: <https://junit.org/junit5/docs/current/user-guide/>. Accessed: 15 June 2026.
- [W39] Mockito, *Mockito Documentation*. Available at: <https://site.mockito.org/>. Accessed: 18 June 2026.
- [W40] JaCoCo, *JaCoCo Documentation*. Available at: <https://www.jacoco.org/jacoco/trunk/doc/>. Accessed: 21 June 2026.
- [W41] Karma, *Karma Documentation*. Available at: <https://karma-runner.github.io/latest/index.html>. Accessed: 24 June 2026.
- [W42] SonarSource, *SonarQube Cloud Documentation*. Available at: <https://docs.sonarsource.com/sonarqube-cloud/>. Accessed: 27 June 2026.
- [W43] OWASP Foundation, *OWASP*. Available at: <https://owasp.org/>. Accessed: 30 June 2026.
- [W44] OWASP Foundation, *Dependency-Check*. Available at: <https://jeremylong.github.io/DependencyCheck/>. Accessed: 2 July 2026.

- [W45] NIST, *National Vulnerability Database*. Available at: <https://nvd.nist.gov/>. Accessed: 4 July 2026.
- [W46] npm, *npm audit*. Available at: <https://docs.npmjs.com/auditing-package-dependencies-for-security-vulnerabilities>. Accessed: 6 July 2026.
- [W47] GitLeaks, *GitLeaks Documentation*. Available at: <https://github.com/gitleaks/gitleaks>. Accessed: 8 July 2026.
- [W48] Aqua Security, *Trivy Documentation*. Available at: <https://trivy.dev/latest/>. Accessed: 9 July 2026.
- [W49] CNCF, *Kubernetes Documentation*. Available at: <https://kubernetes.io/docs/>. Accessed: 10 July 2026.
- [W50] Kubernetes SIG, *Minikube Documentation*. Available at: <https://minikube.sigs.k8s.io/docs/>. Accessed: 11 July 2026.
- [W51] Kubernetes, *kubectl Documentation*. Available at: <https://kubernetes.io/docs/reference/kubectl/>. Accessed: 12 July 2026.
- [W52] Kubeconform, *Kubeconform Documentation*. Available at: <https://github.com/yannh/kubeconform>. Accessed: 12 July 2026.
- [W53] Spring, *Spring Boot Actuator Documentation*. Available at: <https://docs.spring.io/spring-boot/reference/actuator/>. Accessed: 13 July 2026.
- [W54] Micrometer, *Micrometer Documentation*. Available at: <https://docs.micrometer.io/micrometer/reference/>. Accessed: 13 July 2026.
- [W55] Prometheus Authors, *Prometheus Documentation*. Available at: <https://prometheus.io/docs/>. Accessed: 14 July 2026.
- [W56] Prometheus Authors, *PromQL Documentation*. Available at: <https://prometheus.io/docs/prometheus/latest/querying/basics/>. Accessed: 14 July 2026.
- [W57] Grafana Labs, *Grafana Documentation*. Available at: <https://grafana.com/docs/grafana/latest/>. Accessed: 15 July 2026.
- [W58] Grafana Labs, *Grafana Loki Documentation*. Available at: <https://grafana.com/docs/loki/latest/>. Accessed: 15 July 2026.

- [W59] Grafana Labs, *Grafana Alloy Documentation*. Available at: <https://grafana.com/docs/alloy/latest/>. Accessed: 15 July 2026.

Résumé

Ce rapport présente la conception et le développement d'un Système d'Information des Ressources Humaines (SIRH) sécurisé basé sur une architecture microservices. La plateforme intègre des microservices Spring Boot, une interface Angular, Keycloak pour l'authentification et la gestion des rôles, ainsi qu'un service FastAPI proposant des recommandations de formation basées sur l'intelligence artificielle. Afin d'assurer la qualité, la sécurité et l'automatisation du déploiement, une chaîne DevSecOps complète a été mise en œuvre à l'aide de Docker, GitHub Actions, SonarQube Cloud, OWASP Dependency-Check, GitLeaks, Trivy et Kubernetes. La supervision est assurée par Prometheus, Grafana, Alloy et Loki. Ce projet illustre la réalisation d'une application d'entreprise sécurisée, évolutive et intelligente selon la méthodologie Agile Scrum.

Mots clés : SIRH, Microservices, Spring Boot, Angular, Keycloak, Intelligence Artificielle, DevSecOps, Kubernetes.

Abstract

This report presents the design and implementation of a secure Human Resources Information System (HRIS) based on a microservices architecture. The platform integrates Spring Boot microservices, an Angular frontend, Keycloak for authentication and role-based access control, and a FastAPI service that provides AI-based training recommendations. To ensure software quality, security, and deployment automation, a complete DevSecOps pipeline was implemented using Docker, GitHub Actions, SonarQube Cloud, OWASP Dependency-Check, GitLeaks, Trivy, and Kubernetes. Monitoring is achieved through Prometheus, Grafana, Alloy, and Loki. The project demonstrates the development of a secure, scalable, and intelligent enterprise application following Agile Scrum practices.

Keywords : HRIS, Microservices, Spring Boot, Angular, Keycloak, Artificial Intelligence, DevSecOps, Kubernetes.